Software Engineering

Chapter 4: System Modeling & Unified Architecture

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University

Chapter 4: Roadmap & Core Curriculum

Part 1: Foundations & Core Models

- **4.1 Foundations of Modeling:** Abstraction, 4 perspectives, 5 essential diagrams, and the elephant parable.
- **4.2 The UML:** The Method Wars, Three Amigos trinity (Jacobson, Rumbaugh, Booch), and OMG standardization.
- **4.3 Functional & Use Case Models:** Actor goals, system boundaries, include vs. extend, and AI prompt guidelines.
- **4.4 Structural & Class Models:** Domain classes, visibility, multiplicity, whole-part coupling, and AI prompt guidelines.
- **4.5 Interaction & Sequence Diagrams:** Chronological messages, lifelines, activations, and AI prompt guidelines.

Part 2: Workflows, Architecture & Synthesis

- **4.6 Process & Activity Diagrams:** Concurrent workflows, decisions, forks/joins, swimlanes, and AI prompt guidelines.
- **4.7 Behavioral & State Machines:** Reactive systems, event triggers, actions vs. activities, and AI prompt guidelines.
- **4.8 Text-Based Modeling with PlantUML:** Code-as-architecture, syntax quick reference, and multi-diagram pipelines.
- **4.9 Conceptual Recap & Synthesis:** Core principles review, fill-in-the-blank quiz, and references.

4.1 Foundations of System Modeling

> *"A language that doesn't affect the way you think about programming is not worth knowing."*
> *— Alan Perlis*

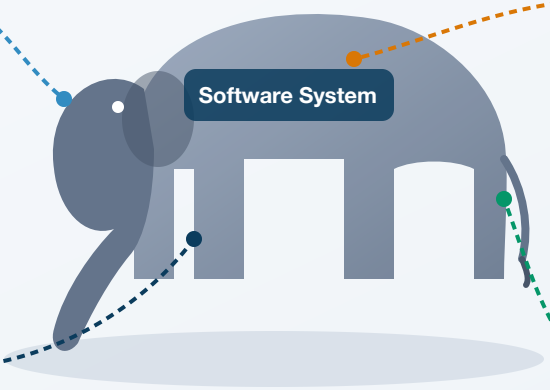
What is System Modeling?

> *"System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system."*
> *— Ian Sommerville, Software Engineering (10th ed.)*

- **Essential Principles of Modeling:**
  - **Abstraction: Hides** non-essential implementation details to highlight critical architectural structures, data flows, and dependencies.
  - **Multiple Perspectives:** No single diagram can explain a complex software system. Different stakeholders require different views.
  - **Communication & Verification:** Serves as a universal **visual** grammar between product managers, system architects, software developers, and QA engineers.
  - **Blueprint for Implementation:** Provides the formal structural and behavioral specification from which executable code and database schemas are developed.

The Parable of the Elephant & Multiple Perspectives

The Blind Men and the Elephant: Why Multi-Perspective Modeling is Essential



① Feeling Trunk: "It's like a water pipe!"

Interaction Perspective

- Focus: Message timing & invocation flows
- Blind Spot: Misses static data schemas

UML Model: Sequence Diagram

② Feeling Leg: "It's like a solid pillar!"

Structural Perspective

- Focus: Class attributes, methods & relations
- Blind Spot: Misses dynamic runtime behavior

UML Model: Class Diagram

③ Feeling Body: "It's like a high wall!"

External / Context Perspective

- Focus: System boundary, actors & external APIs
- Blind Spot: Cannot see internal decomposition

UML Model: Context Diagram

④ Feeling Tail: "It's like a swinging rope!"

Behavioral Perspective

- Focus: Business workflows & state transitions
- Blind Spot: Cannot capture static entity models

UML Model: Activity / State Diagram

Takeaway: Every model is a partial projection; only multi-perspective modeling reveals the full architecture!

4 Core System Modeling Perspectives

- **1. External Perspective:**
  - Models the environment, external partners, and operational context of the system.
  - Defines the strict system perimeter: what is built internally vs. what is delegated to third parties.
- **2. Interaction Perspective:**
  - Models dynamic communications between external actors and the system, or message exchanges between internal collaborating objects.
- **3. Structural Perspective:**
  - Models the static architecture of system data, object classes, attributes, methods, and relationships independent of runtime execution order.
- **4. Behavioral Perspective:**
  - Models the dynamic execution behavior, sequential business workflows, and reactive discrete state transitions in response to external events.

5 Essential UML Models in Modern Practice

Model / Diagram	Section	Perspective	Nature	Primary Engineering Role
1. Use Case Model	4.3	Interaction	Functional Contract	Defines system boundary, actor goals, and scope
2. Class Model	4.4	Structural	Static Backbone	Specifies domain entities, attributes, and relationships
3. Sequence Diagram	4.5	Interaction	Dynamic Chronology	Traces runtime message passing and BCE responsibilities
4. Activity Diagram	4.6	Behavioral	Dynamic Workflow	Models business processes, fork/join concurrency & swimlanes
5. State Machine Diagram	4.7	Behavioral	Reactive Lifecycle	Captures event-driven states, transitions & invariants

> ***Implementation & Acceleration:*** *Unified via* ***4.8 PlantUML*** *(Code-as-Architecture) with* ***AI Prompt Guidelines*** *integrated in each model.*

Concept Check Question 1

A software architect wants to define **how domain entity data is structured and how classes inherit and associate with each other**, completely independent of runtime execution order. Which modeling perspective should the architect adopt?

- **A.** External Perspective
- **B.** Interaction Perspective
- **C.** Structural Perspective
- **D.** Behavioral Perspective

4.2 The Unified Modeling Language (UML)

> *"A standardized visual grammar transformed software engineering from proprietary silos into a global discipline."*

The Need for Standardization: The 1990s "Method Wars"

- **The Rise of Object-Oriented Programming (Late 1980s – Early 1990s):**
  - The software industry transitioned from procedural code (C, Pascal) to object-oriented paradigms (C++, Smalltalk).
  - Software engineers urgently needed visual notations to represent classes, objects, and relationships.
- **The "Method Wars" Era:**
  - Over **50 competing OO modeling notations** flooded the commercial market.
  - Engineers argued fiercely over whether classes should be clouds, rectangles, or ovals; whether inheritance should be an open triangle, filled arrow, or dashed line.
  - **Severe Industry Fragmentation:** Companies could not exchange models, CASE tools were incompatible, and developers had to relearn notation whenever they switched jobs.

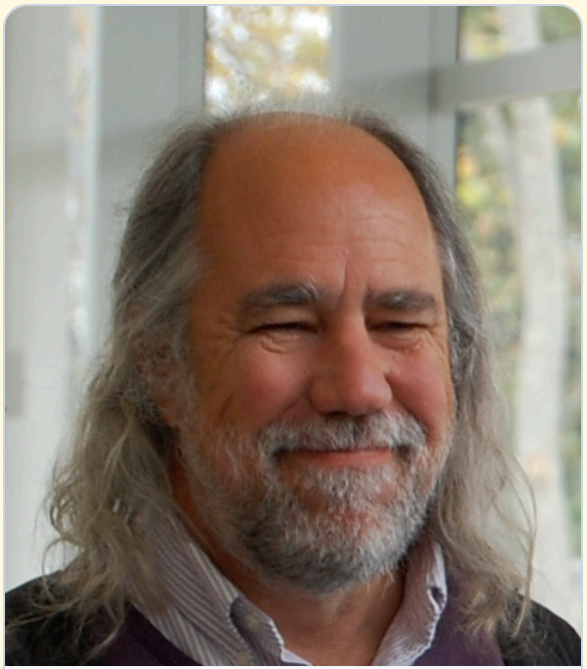
Unification & Standardization: From Rational to OMG

- **1994 – Unification Begins at Rational Software:**
  - Jim Rumbaugh joined Grady Booch at Rational to merge the Booch Method and OMT into the "Unified Method" (v0.8).
- **1995 – The Three Amigos Assemble:**
  - Ivar Jacobson joined Rational, bringing his revolutionary **Use Case** methodology (OOSE).
- **1997 – OMG International Standardization:**
  - Submitted to the **Object Management Group (OMG)**; unanimously adopted as **UML 1.1** in November 1997.
- **2005 – UML 2.0 Major Architecture Overhaul:**
  - Expanded from 9 to 13 (and later 14) diagram types with formal execution metamodels.

Pioneers of UML: Grady Booch

- **Role & Distinctions:**
  - Chief Scientist, Rational Software; IBM Fellow; ACM Fellow.
- **Pioneered Methodology:**
  - **The Booch Method** and seminal text: *Object-Oriented Analysis and Design with Applications*.
- **Core Contribution to UML:**
  - Focused heavily on **concrete software design**, module decomposition, class abstractions, and architectural patterns.
  - Championed visual expressiveness for implementation-level object structures and code mapping.
- **Famous Architectural Maxim:**

> *"Clean code always looks like it was written by someone who cares."*
>
> ”

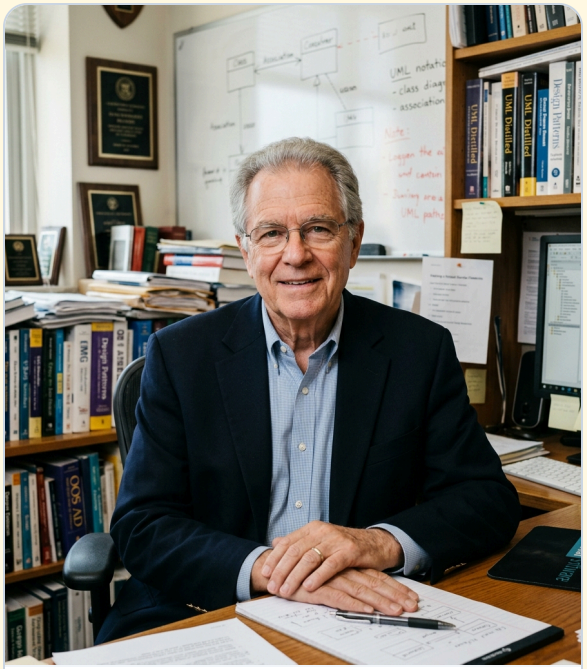


Grady Booch
Rational Software / IBM Fellow

Pioneers of UML: James Rumbaugh

- **Role & Distinctions:**
  - Lead Researcher, General Electric (GE) Global R&D; Rational Software; IBM.
- **Pioneered Methodology:**
  - **Object Modeling Technique (OMT)** and book: *Object-Oriented Modeling and Design*.
- **Core Contribution to UML:**
  - Emphasized rigorous **domain analysis**, semantic data modeling, and entity-relationship mapping.
  - Pioneered the synthesis of object structure with David Harel's **Statecharts** to model dynamic reactive systems.
- **Famous Architectural Maxim:**

> *"You cannot build great software without understanding domain truth."*



James Rumbaugh
GE Research / Rational Software

Pioneers of UML: Ivar Jacobson

- **Role & Distinctions:**
  - Lead Architect, Ericsson; Founder, Objectory AB; Rational Software; SEMAT Pioneer.
- **Pioneered Methodology:**
  - **Object-Oriented Software Engineering (OOSE)**.
- **Core Contribution to UML:**
  - **Inventor of Use Cases (1986):** Revolutionized requirements by anchoring system architecture to measurable user goals.
  - Introduced the **Boundary–Control–Entity (BCE)** robustness analysis pattern and component-based architecture.
- **Famous Architectural Maxim:**

> *"A system that has no users has no reason to exist. Model user goals first."*

Ivar Jacobson
Ericsson / Objectory / Rational

The Three Amigos: The Trinity of Software Modeling

Ivar Jacobson

The "Why" (Goal / User)

- **Use Case Driven:** What goals do external actors achieve?
- **BCE Robustness:** Separating UI boundaries from data entities.
- **Component Contracts:** Reusable subsystem architectures.

James Rumbaugh

The "What" (Domain Data)

- **Domain Analysis:** What real-world concepts exist?
- **Class & Data Schemas:** Structural entity relationships.
- **State Machines:** Reactive event-driven lifecycles.

Grady Booch

The "How" (Design & Code)

- **Object-Oriented Design:** How do classes collaborate?
- **Macro Architecture:** Layered module decomposition.
- **Implementation Map:** Direct code translation into OOP.

> 📌 ***Architectural Synthesis:*** *Jacobson captures* ***Why*** *the system exists; Rumbaugh models* ***What*** *domain concepts exist; Booch designs* ***How*** *components execute in code.*

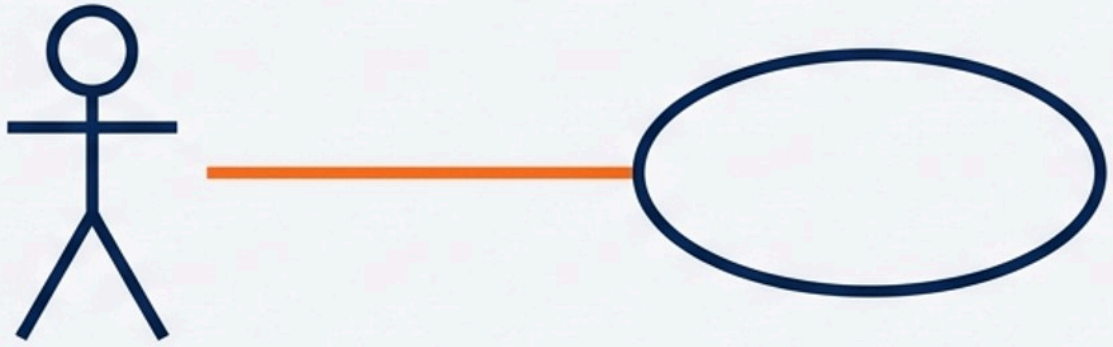
Concept Check Question 2

Which of the "Three Amigos" was specifically renowned for inventing **Use Cases (1986)** to anchor software architecture to tangible user goals?

- **A.** Grady Booch
- **B.** James Rumbaugh
- **C.** Ivar Jacobson
- **D.** Martin Fowler

4.3 Functional & Use Case Models

> *"A use case is a contract for behavior between the system and its actors to achieve a measurable business goal."*
> *— Alistair Cockburn*



Beginner's Guide to UML Use-Case Diagram Modeling

Transforming abstract requirements into clear system designs.

Focus: Business Value & User Goals

High-level visual representation of how users or external systems interact with the application.

💡 **The Golden Rule:**
Build use-case models from the project stake perspective, not the developer's.

Exclude: Technical Implementation

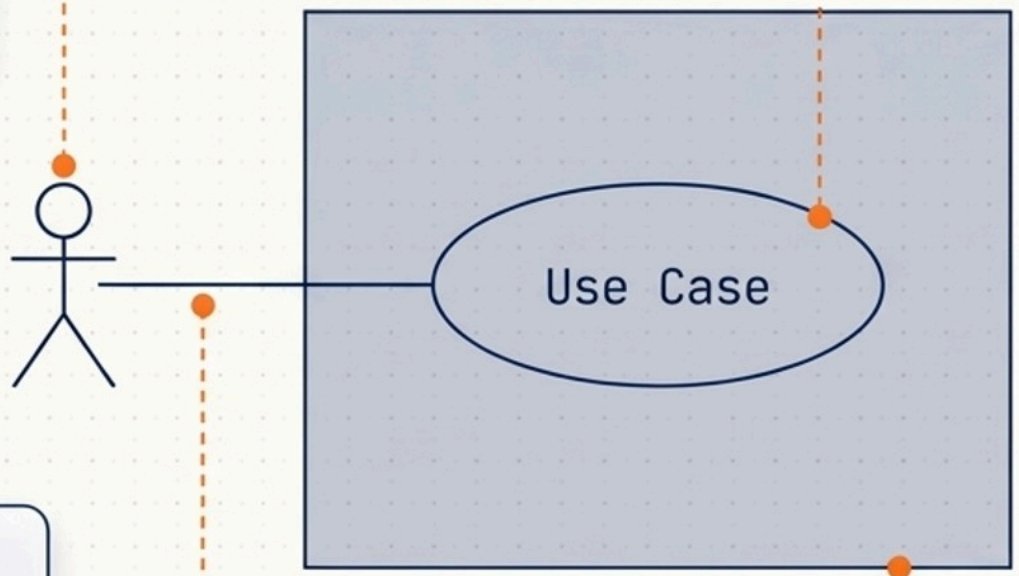
Do not map data flows, functional decomposition, or database processes.

Actor

Any person, organization, external system, or timer interacting with the system.

Use Case

A sequence of actions delivering measurable value to the actor.

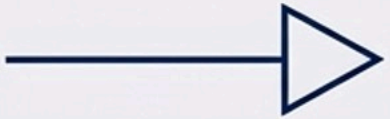


Relationship

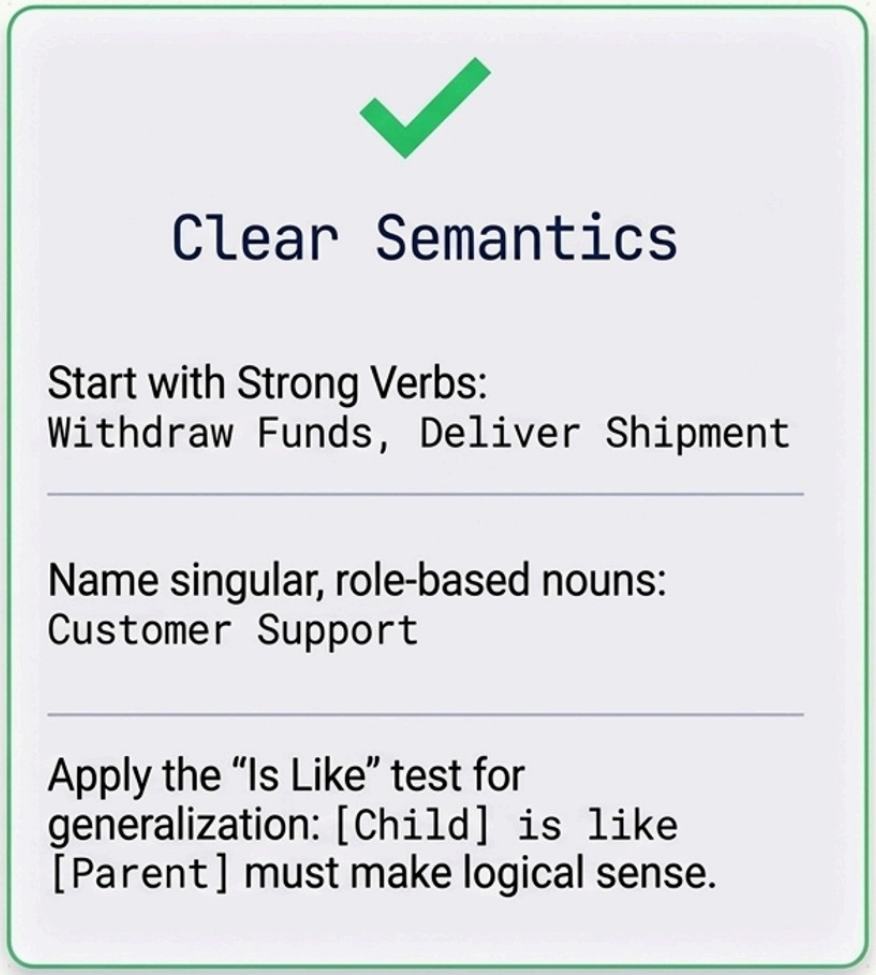
Shows how actors and use cases connect, depend on, or generalize each other.

System Boundary

Defines what is inside your system vs. what is outside your control.

Relationship	UML Notation	Behavior	Real-World Analogy
Association		Actor participates in a use case.	A customer walks into a store.
<<include>>		Mandatory, reusable logic. Runs at a specific point.	Calling a function: `checkout()` → `calculateTax()`
<<extend>>		Optional/conditional logic. Might run during certain steps.	A pop-up coupon or conditional error check.
Generalization		Is a relationship. Child inherits logic from parent.	International Student is like a Student, but with extra steps.

Clear Use Case Naming



Rules of Building Use Case Models

NESTING LIMIT

Max 2 Levels of Nesting

Avoid functional decomposition (e.g., A includes B includes C).
Keep it requirements-focused, not design-focused.

ACTOR INTERACTIONS

No Actor-to-Actor Lines

Actors do not interact directly on the diagram.
Detail these interactions in your text specifications.

SYSTEM BOUNDARIES

Use Boundaries with Purpose

Only use the boundary box to define system scope or distinct release phases (Phase 1, Phase 2). Omit if redundant.

Note: Call out external systems with <<system>> and use a Time actor (clock icon) for scheduled triggers.

Case Study: University Enrollment System

Actors (The "Who")

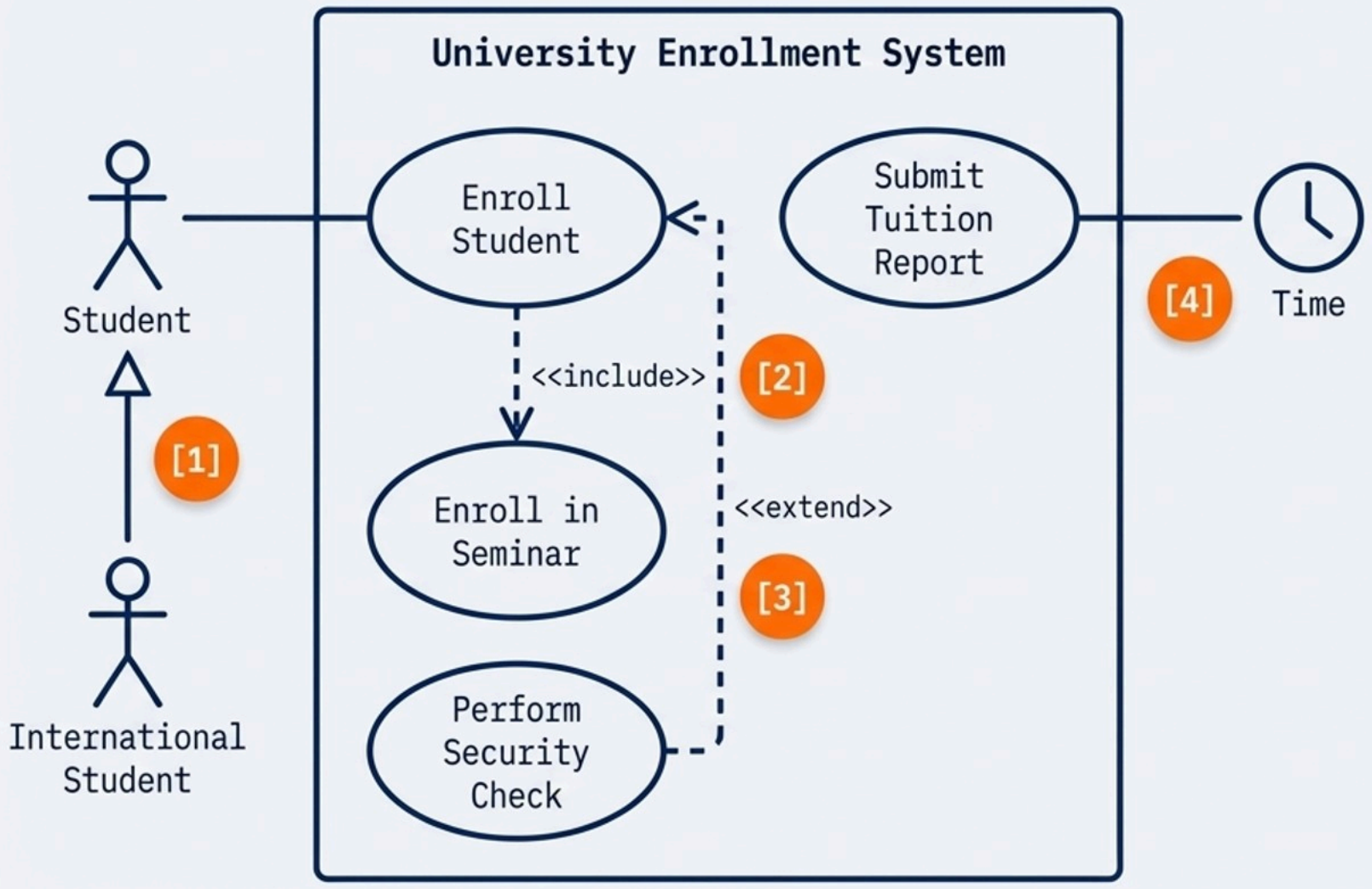
Student
Primary actor

International Student
Specialized actor

Time
Triggers periodic tasks

Core Use Cases (The "What")

- Enroll Student
- Enroll in Seminar
- Perform Security Check
- Submit Tuition Report (Monthly)



Blueprint Legend

[1] Generalization:
Int'l Student inherits base student logic.

[2] <<include>>:
Enrolling always requires a seminar (Mandatory).

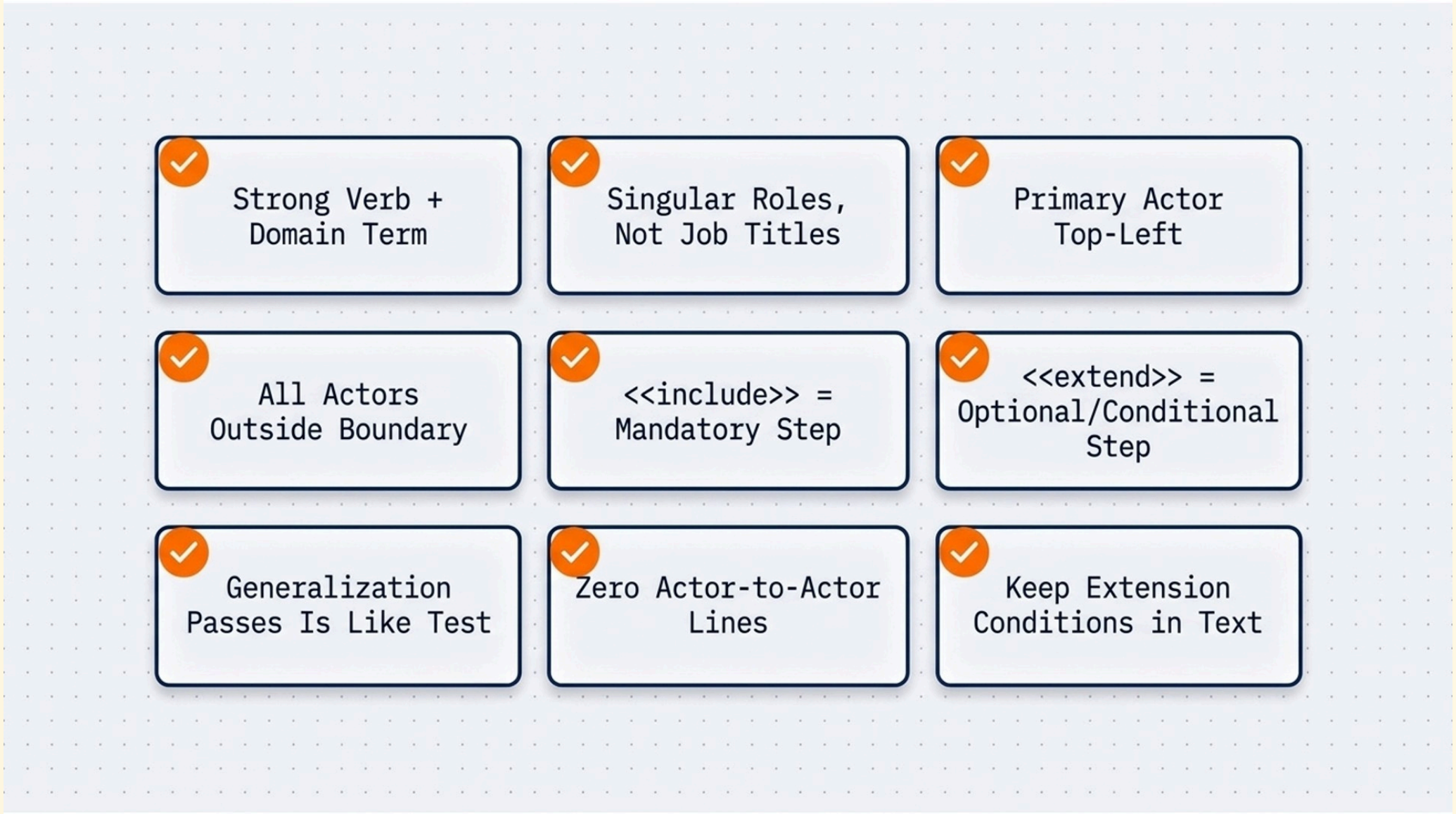
[3] <<extend>>:
Security check only runs for certain students (Conditional).

[4] Time Actor:
Triggers the monthly tuition report.

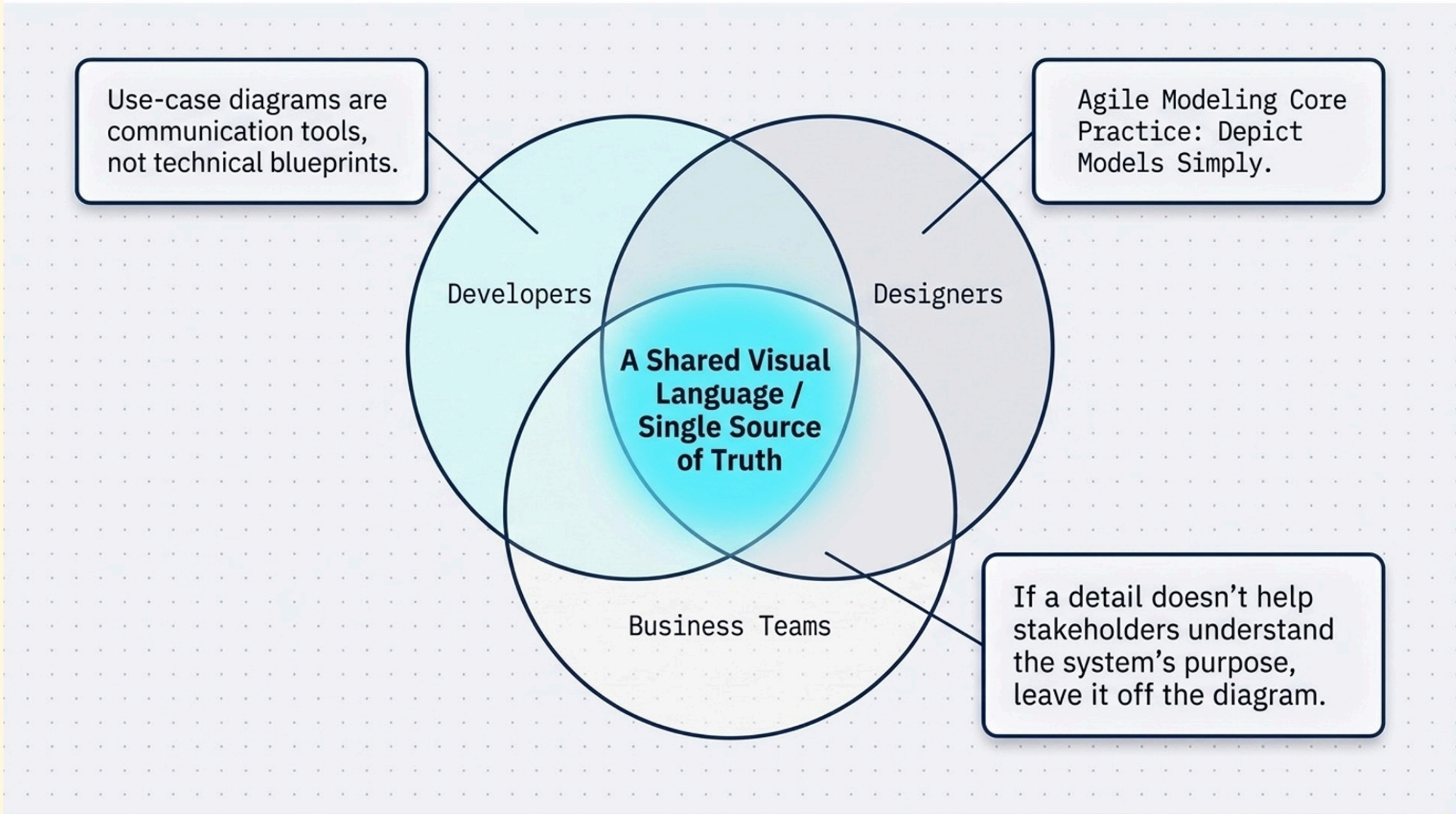
Common Anti-Patterns of Use Case Models

The Mistake (Symptom)	Why It Fails (Diagnosis)	The Fix (Cure)
✗ Technical names (`Process_DB_Transaction`)	Stakeholders don't understand implementation details.	✓ Use domain-friendly names (`Withdraw Funds`, `Generate Report`).
✗ Drawing arrows showing data flow	Associations show initiation, not data movement.	✓ Remove arrowheads unless showing passive actors.
✗ Overusing `<<extend>>` lines	Creates messy, unreadable diagrams.	✓ Move optional/conditional logic into textual use-case specs.
✗ Connecting Customer to Admin directly	Actors don't interact on use-case diagrams.	✓ Describe human interaction in the text scenario.

Best Practices Checklists



Communication First Mindset



Use Case Specification: UC-01 Enroll Student

Field	Specification Details
Use Case ID / Name	**UC-01: Enroll Student in Course**
Primary Actor	Student (authenticated via campus Single Sign-On)
Preconditions	Student is matriculated with active status; enrollment registration window is open
Postconditions	Student is officially enrolled in course section; seat quota decremented by 1
Main Success Scenario	**1.** Student searches for open course sections by department or course code. **2.** System displays matching course sections, schedules, and remaining seat quotas. **3.** Student selects target section and submits an enrollment request. **4.** System executes `<<include>> Verify Prerequisites` against academic transcript. **5.** System reserves seat, updates enrollment ledger, and generates confirmation notice. **6.** Student views updated schedule and receives confirmation notification.
Extensions (Alternate)	**4a. Prerequisite Deficiency:** System displays missing prerequisite courses and halts. **4b. Schedule Conflict:** System alerts student of timeslot overlap with existing class. **5a. Section at Capacity:** System offers `<<extend>> Add to Waitlist` queue.

AI Assistance: Guidelines for Use Case Modeling

- **1. Define Clear System Boundaries:**
  - Clearly separate what is inside the system from external users and third-party services (e.g., payment gateways).
- **2. Identify Real User Goals (Actors & Use Cases):**
  - Focus on complete, meaningful user goals (e.g., `Place Order`, `Track Delivery`).
  - Avoid breaking steps down into trivial UI interactions (no "Click Button" or "Enter Password").
- **3. Clarify Relationship Semantics:**
  - **Mandatory Sub-flows:** Identify common steps that occur every single time (shared building blocks).
  - **Optional or Conditional Steps:** Explicitly distinguish optional extensions (e.g., applying vouchers) from normal flows.
- **4. Human Review & Reality Check:**
  - Are use cases named with clear active verbs?
  - Are external partner systems kept outside the core system boundary?

AI Prompt Example: Food Delivery Use Case Model

1. Role & Task Instruction:

```
Act as a Lead Systems Analyst.
Generate a clean PlantUML Use Case
diagram for a Food Delivery System.
```

2. Modeling Guidance:

- Clearly define system boundary vs. external services.
- Focus on meaningful user goals (avoid trivial UI button clicks).
- Separate mandatory sub-flows from optional extensions.
- Identify primary human actors and external partner systems.

3. Input Requirements Statement:

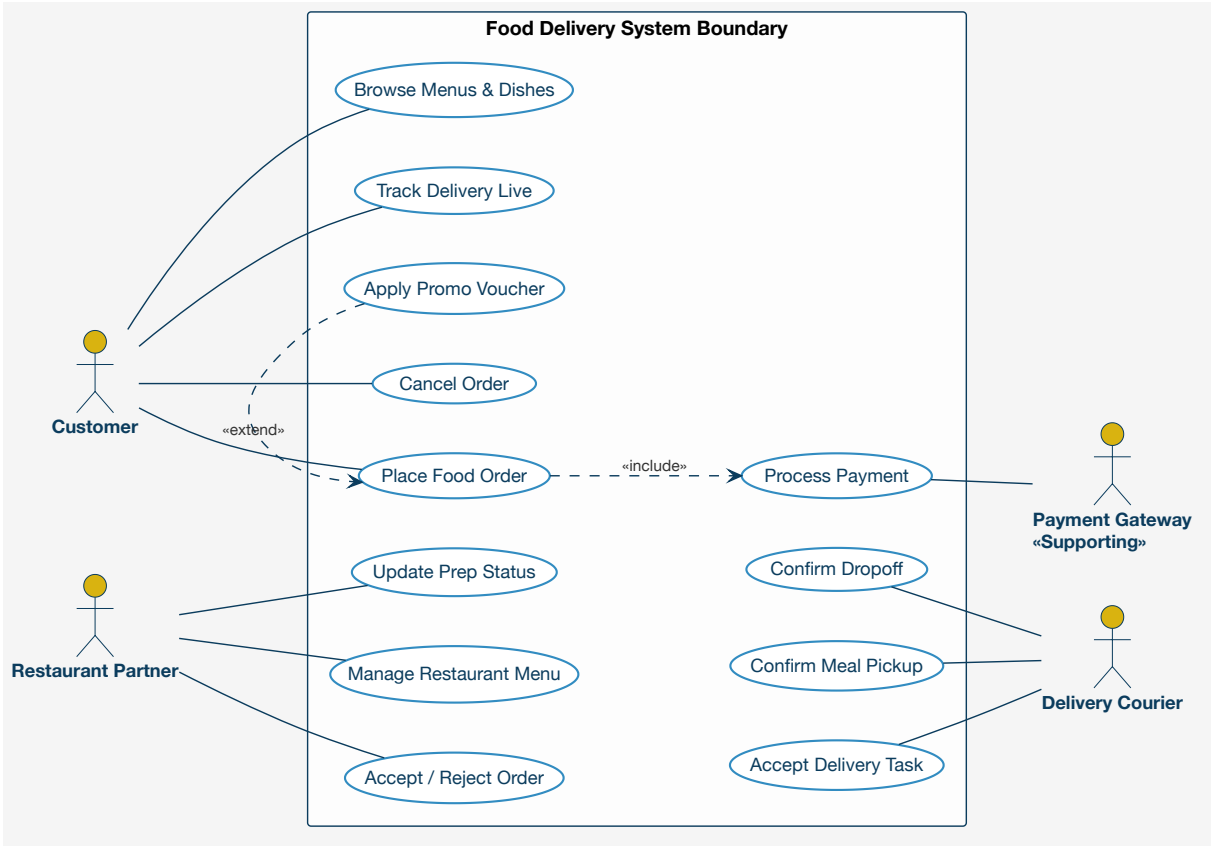
> *"Our platform allows **Customers** to browse restaurants, add food items to cart, and place orders.*
> *When placing an order, the system must always authorize payment through an external **Payment Gateway** and validate delivery address.*
> *A customer may optionally enter a discount coupon code (* `Apply Promo Voucher` *) or select contactless doorstep delivery.*
> ***Restaurant Staff** review incoming orders, accept or reject them, and update kitchen prep status.*
> ***Delivery Couriers** accept available delivery tasks, pick up orders, and complete deliveries."*
>
> ”

Generated PlantUML & Diagram: Food Delivery Use Case Model

Generated PlantUML Specification:

```
@startuml
left to right direction
actor Customer
actor "Restaurant Partner" as Restaurant
actor "Payment Gateway\n«Supporting»" as Payment
actor "Delivery Courier" as Courier

rectangle "Food Delivery System" {
  usecase "Browse Menus" as UC_Browse
  usecase "Place Food Order" as UC_PlaceOrder
  usecase "Apply Promo Voucher" as UC_Voucher
  usecase "Track Delivery Live" as UC_Track
  usecase "Manage Menu" as UC_Menu
  usecase "Accept / Reject Order" as UC_AcceptOrder
  usecase "Process Payment" as UC_Payment
  usecase "Accept Delivery Task" as UC_AcceptTask
}

Customer -- UC_Browse
Customer -- UC_PlaceOrder
Customer -- UC_Track
Restaurant -- UC_Menu
Restaurant -- UC_AcceptOrder

UC_PlaceOrder ..> UC_Payment : <<include>>
UC_Voucher .> UC_PlaceOrder : <<extend>>
UC_Payment -- Payment
```

Rendered Diagram Output:



Interactive Activity: Evolving Use Cases with AI (Pair Hands-On)

Hands-on Experiment: Mutate Requirements & Audit AI Output (4 Mins)

- **1. Add ONE New Feature to the Use Case Prompt:**
  - *Scheduled Ordering:* Allow `Customer` to pre-order food for future delivery.
  - *Live Chat Support:* Connect `Customer` with `Support Agent` or `Courier`.
  - *Group Ordering:* Multiple users add items into a single pooled order.
- **2. Ask AI to Generate:** Prompt an LLM to update the PlantUML use case diagram.
- **3. Render & Critique:** Paste into PlantText and audit:
  - Did AI keep external actors/gateways **outside** the system boundary?
  - Did AI correctly distinguish **mandatory steps** from **optional extensions**?
  - Did AI avoid trivial button clicks (e.g., no "Click Confirm")?

Concept Check Question 3

In our Food Delivery Use Case Model, why does `Apply Promo Voucher` point to `Place Food Order` with `<<extend>>`, while `Place Food Order` points to `Process Payment` with `<<include>>`?

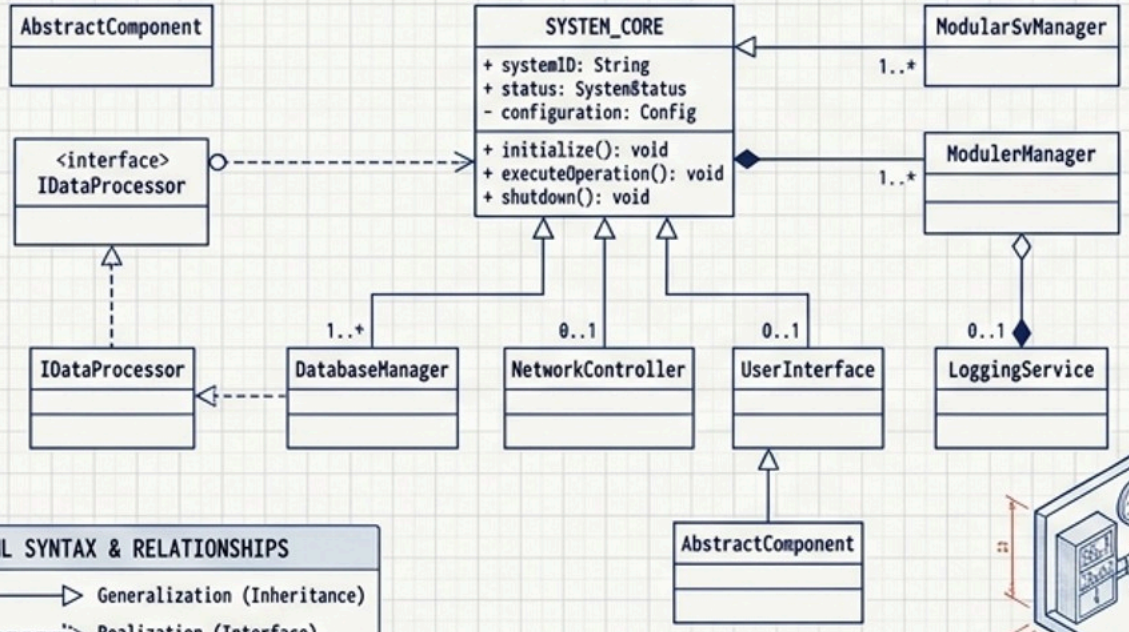
- **A.** Voucher application is mandatory for all orders; payment is optional.
- **B.** Vouchers are conditional optional behavior; payment is mandatory shared execution.
- **C.** Vouchers are executed by supporting actors; payment is executed by primary actors.
- **D.** Vouchers represent class inheritance; payment represents object composition.

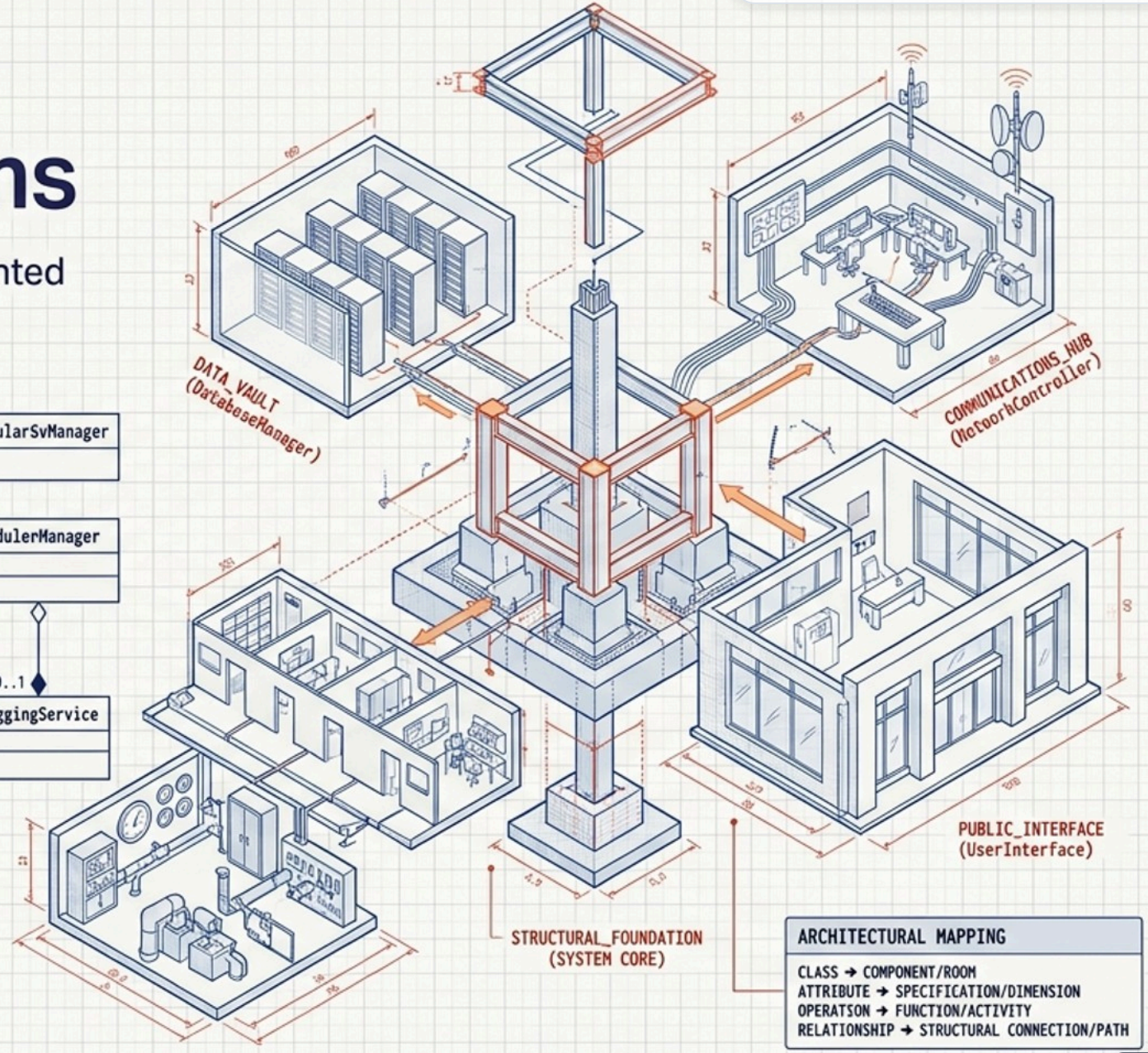
4.4 Structural & Class Models

> *"Classes are the static building blocks; objects are the living runtime instances."*

The Anatomy of UML Class Diagrams

A visual reference guide to mapping object-oriented systems, syntax, and relationships.





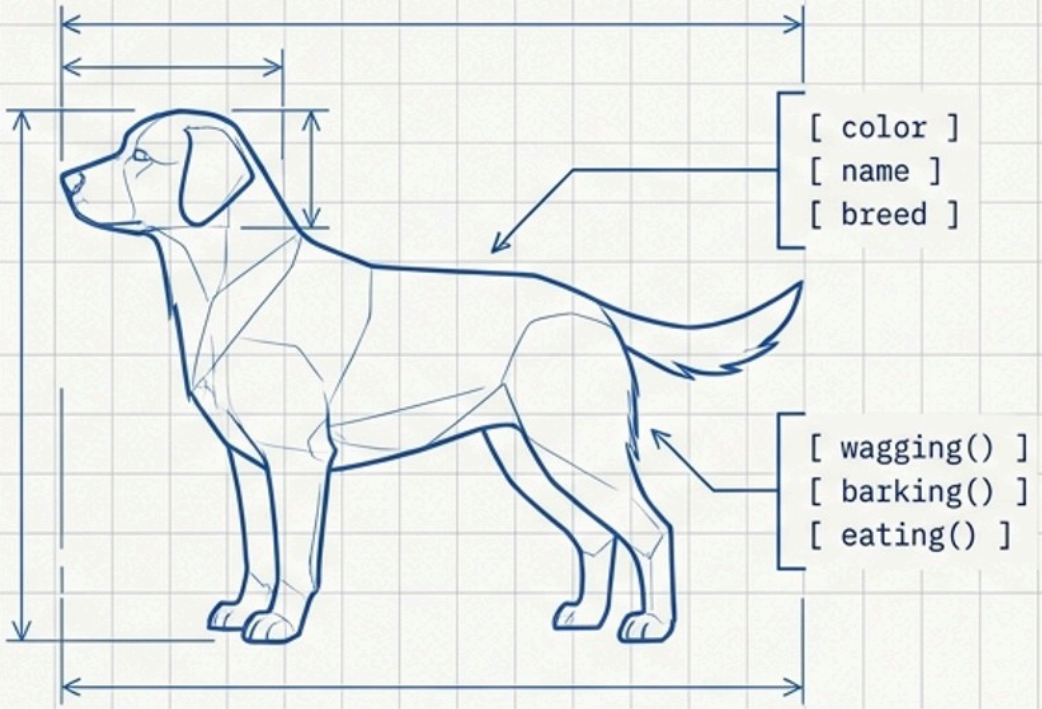
UML SYNTAX & RELATIONSHIPS

- Generalization (Inheritance)
- Realization (Interface)
- Aggregation (Shared) — Shared Ownership
- Composition (Composite) — Strong Ownership
- Association

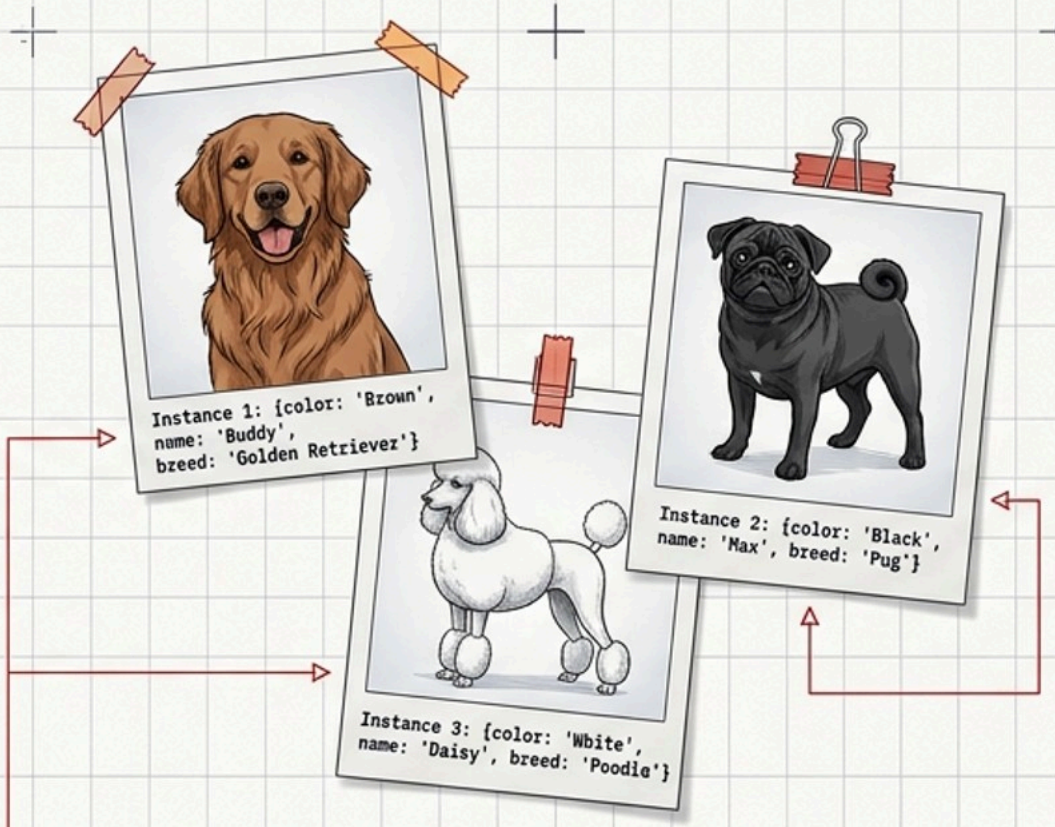
ARCHITECTURAL MAPPING

CLASS → COMPONENT/ROOM
ATTRIBUTE → SPECIFICATION/DIMENSION
OPERATSON → FUNCTION/ACTIVITY
RELATIONSHIP → STRUCTURAL CONNECTION/PATH

Blueprint vs. **Instance**: The Foundation of OOD



A **Class** is a blueprint. It describes the type of objects and encapsulates state (**attributes**) and behavior (**operations**). It is not the object itself.



Objects are usable instances of classes. Built from the same blueprint, they contain the same components but hold distinct, real-world states.

Instance

Define an instance's entry from cluapcst iantan

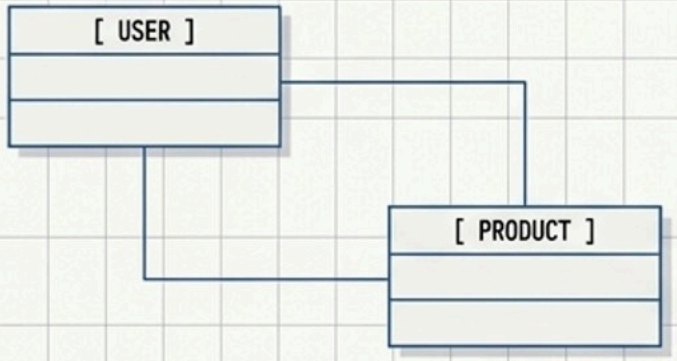
The **Three Perspectives** of Class Modeling

DEVELOPMENT TIME

Conceptual Perspective

Focus: Broad domain concepts.

Detail Level: Low. Seldom moves past class names and basic associations.

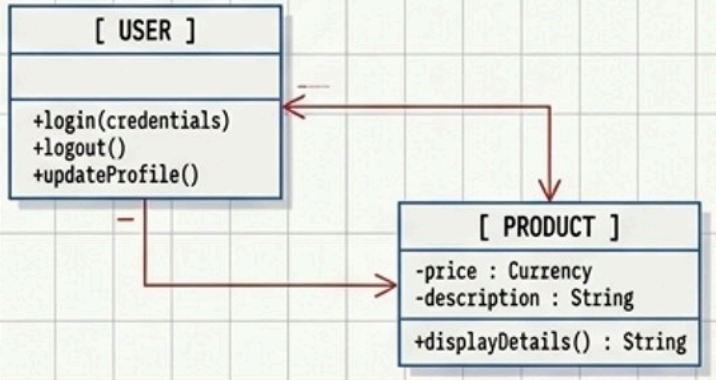


Simplified view for initial understanding.

Specification Perspective

Focus: Interfaces of Abstract Data Types (ADTs).

Detail Level: Medium. Emphasizes what the software does, independent of implementation.

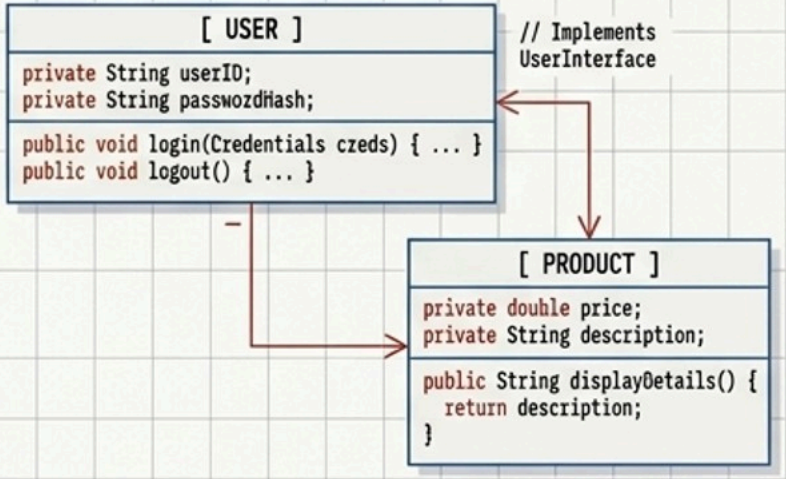


Defines the 'what' without the 'how'.

Implementation Perspective

Focus: Physical implementation of interfaces.

Detail Level: High. Exact replication of written code.



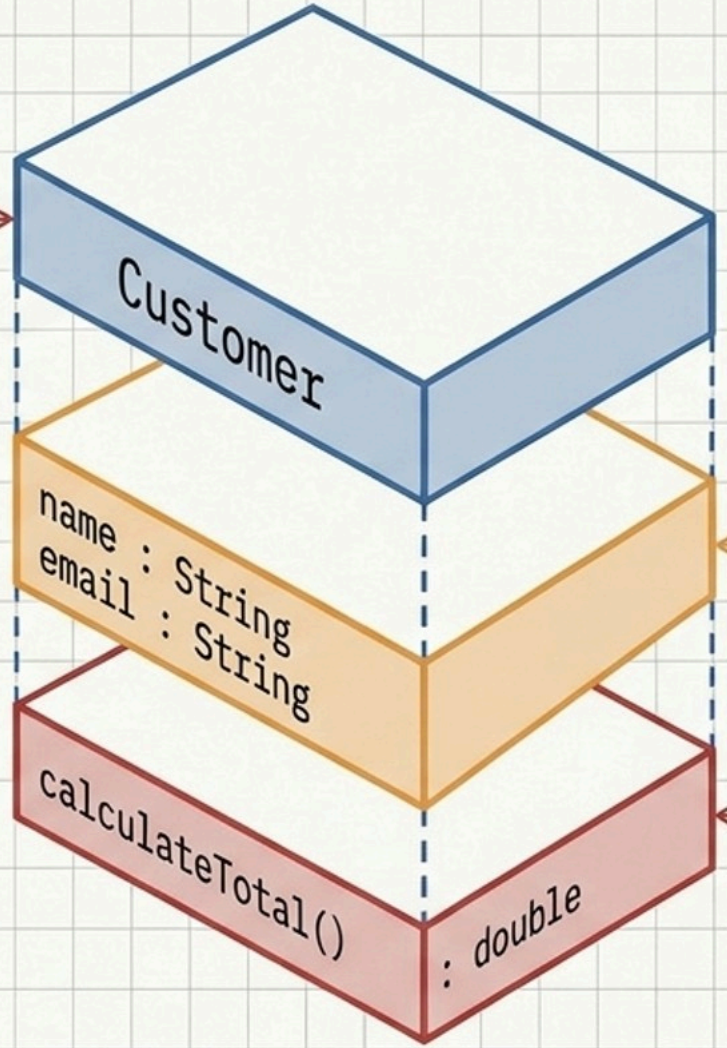
Represents the exact code structure.

Instance

Define at Instance's entwy
fzom cluapcst lantar

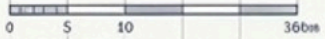
Anatomy of the Class Box

Class Name: The only mandatory information. Represents the core concept.



Attributes:
Encapsulates state.
Maps to member variables in code.
Type is shown after the colon.

Operations / Methods:
Encapsulates behavior.
Services the class provides.
Return type shown at the end.

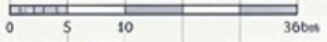


Instance

Define at Instance's entwy
$zom cluapcst lantar

The **Visibility Matrix**

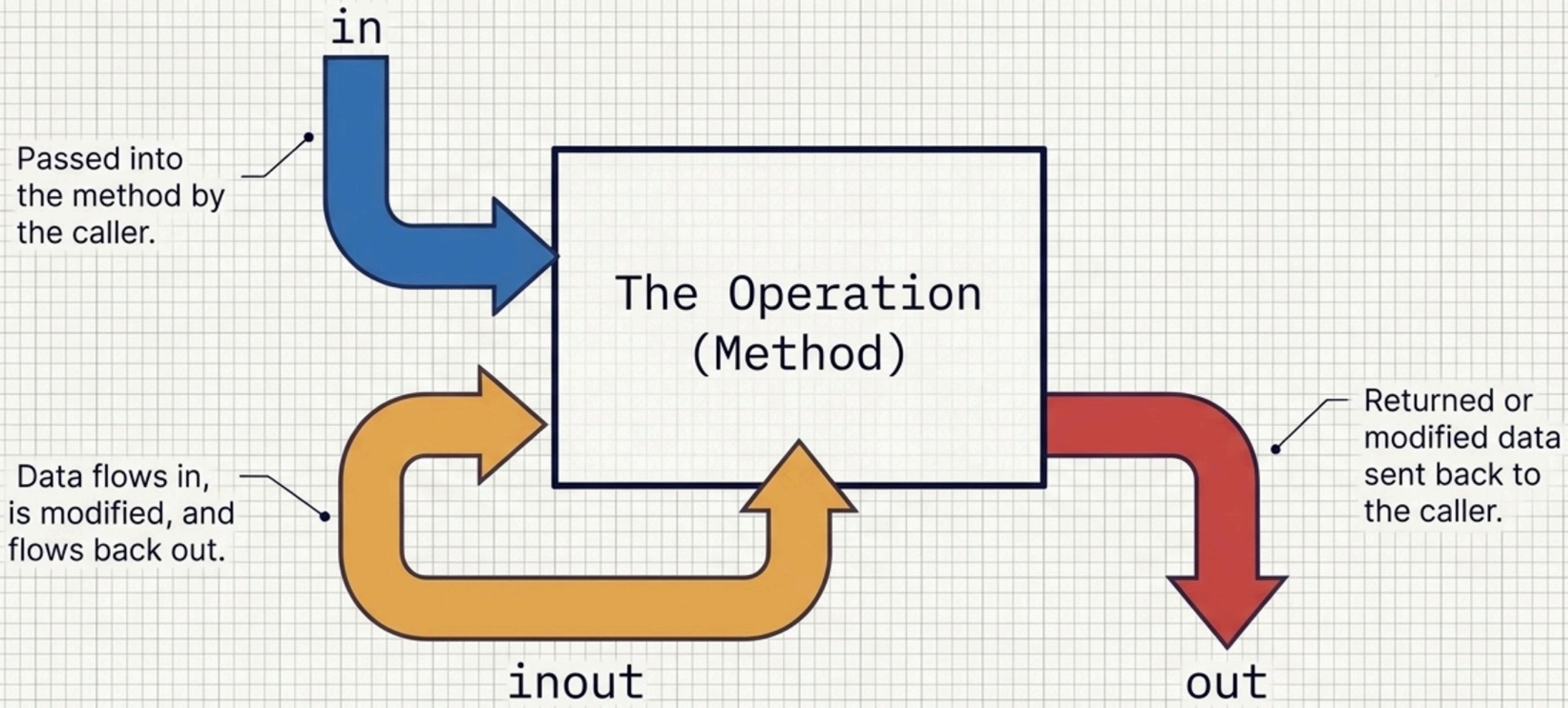
Symbol	Label	Meaning	Code Snippet
	Public	Accessible from anywhere.	`public String name;`
	Private	Accessible only within the class itself.	`private int id;`
	Protected	Accessible within the class and its subclasses.	`protected double balance;`



Instance

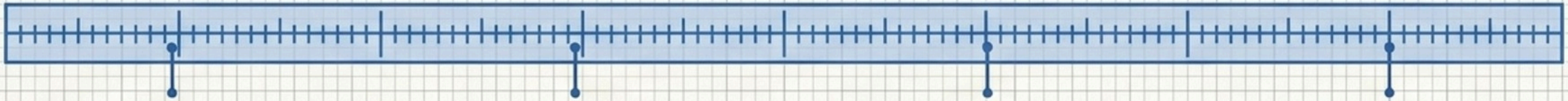
Define at Instance's entwy
Szom eluapcst lantan

Parameter Directionality Dashboard

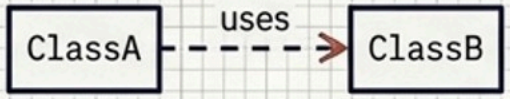


Taxonomy of Relationships

Loosest Connection — Tightest Structure

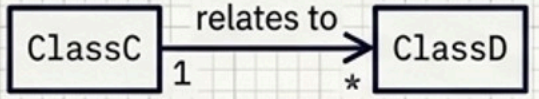


Dependency



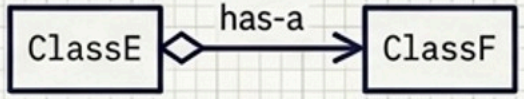
Class A uses Class B temporarily (e.g., as a parameter). No structural ownership.

Association (Simple)



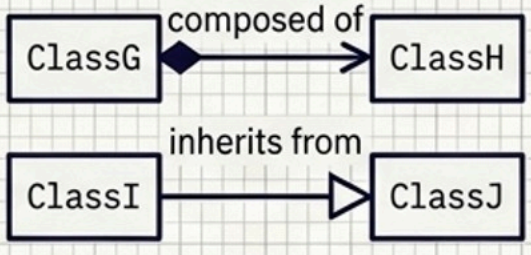
Classes know about each other. Can be unidirectional or bidirectional. Loose coupling.

Aggregation



A whole-part relationship, but parts can exist independently. E.g., a library has books.

Composition & Inheritance

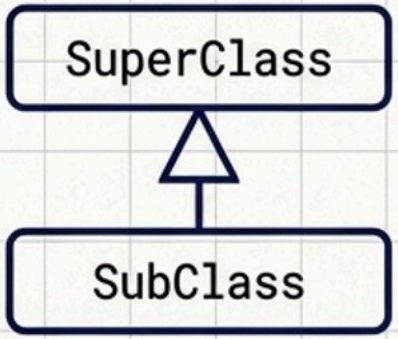


Composition: Strict whole-part; parts die with the whole.
Inheritance: Structural `is-a` hierarchy.

UML precisely conveys how code should be implemented. Connections range from temporary usage to strict structural life-and-death bindings.

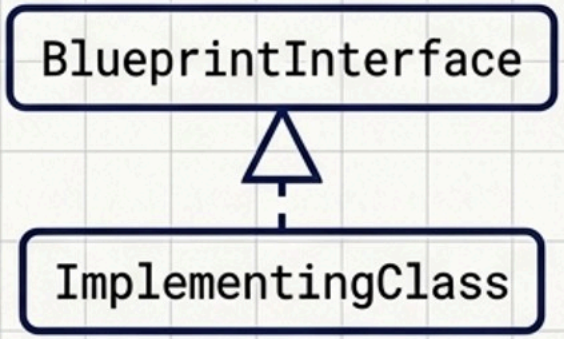
The Connector Cheat Sheet

Inheritance (Generalization)



Represents an “is-a” relationship. Subclass inherits features.

Realization



Relationship between interface and implementation.

Dependency

A “uses” relationship. Object used in a method, not stored in a field.

Simple Association

A structural link between two peer classes.

Lifecycle Diagnostic: Aggregation vs. Composition

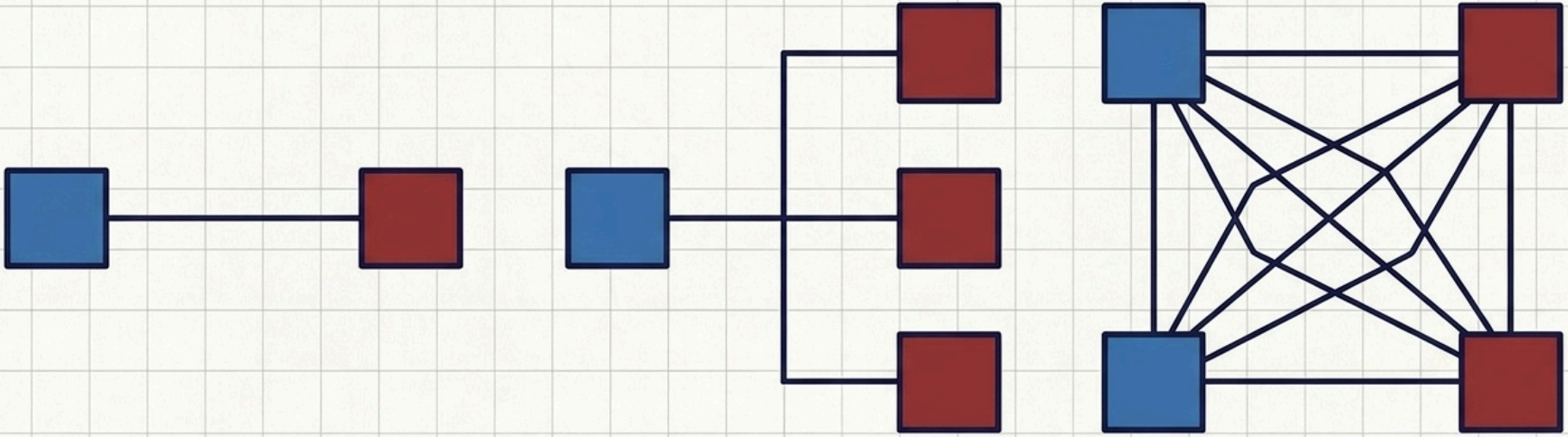
Aggregation	Composition
• **Concept:** A ‘part of’ relationship. • **Lifespan:** Separate lifetimes. • **Analogy:** Team ◇— Player (If the team disbands, players still exist)	• **Concept:** Strict containment. • **Lifespan:** Live and die together. Part cannot stand alone. • **Analogy:** House ◆— Room (If the house is demolished, rooms cease to exist)

Cardinality & Constraints

One to One (1 to 1)

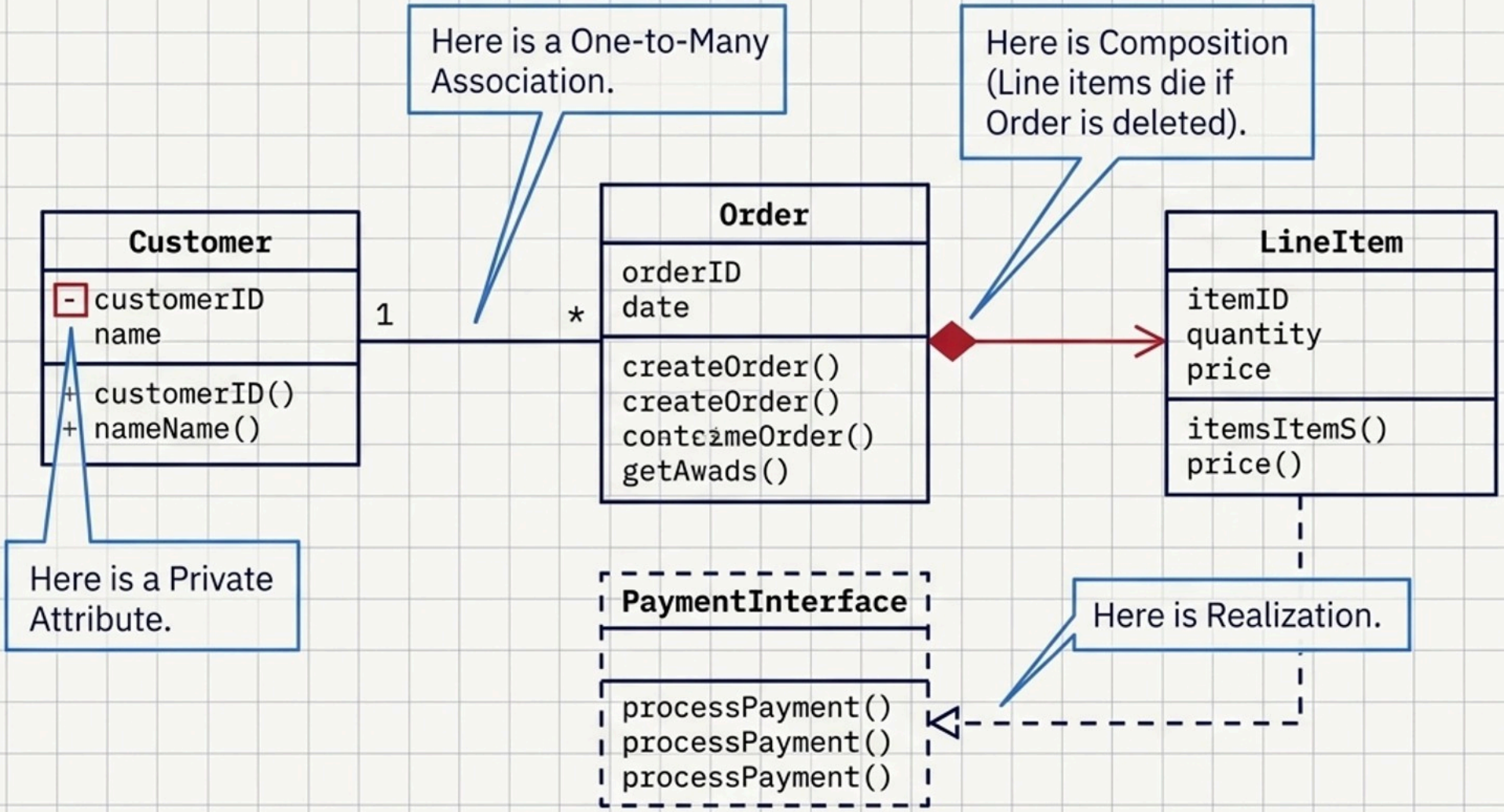
One to Many (1 to *)

Many to Many (* to *)



Cardinality defines the numerical limits of an association, dictating exactly how many instances of one class can connect to another.

Syntax to System: The Order Model



AI Assistance: Guidelines for Class Modeling

- **1. Define Core Domain Entities:**
  - Instruct AI to identify key business concepts (e.g., `Order`, `Customer`, `Item`) rather than raw database tables or UI widgets.
- **2. Clarify Object Relationships:**
  - **Whole vs. Part:** Guide AI to distinguish tight lifecycle ownership from loosely shared parts.
  - **Multiplicity:** Ensure AI specifies cardinality on both ends (e.g., 1 customer, many orders).
  - **Prefer Composition over Inheritance:** Prevent AI from creating deep "is-a" class hierarchies where simple composition fits better.
- **3. Encapsulation & Responsibilities:**
  - Keep internal state private; expose meaningful domain operations as methods.
- **4. Human Reality Check:**
  - Verify AI didn't hallucinate extra classes, invert arrows, or lazily draw plain associations.

AI Prompt Example: Food Delivery Class Model

1. Role & Task Instruction:

```
Act as a Principal Software Architect.
Generate a formal PlantUML Class Diagram
for a Food Delivery Domain Model.
```

2. Modeling Guidance:

- Identify core entities: `Customer`, `Restaurant`, `Order`, `OrderItem`, `Courier`.
- Distinguish whole-part relationships (an `Order` tightly owns its items).
- Clarify multiplicities on both ends of connections.
- Apply encapsulation (hide attributes, expose business operations).
- Keep external payment processing decoupled via an interface.

3. Input Requirements Statement:

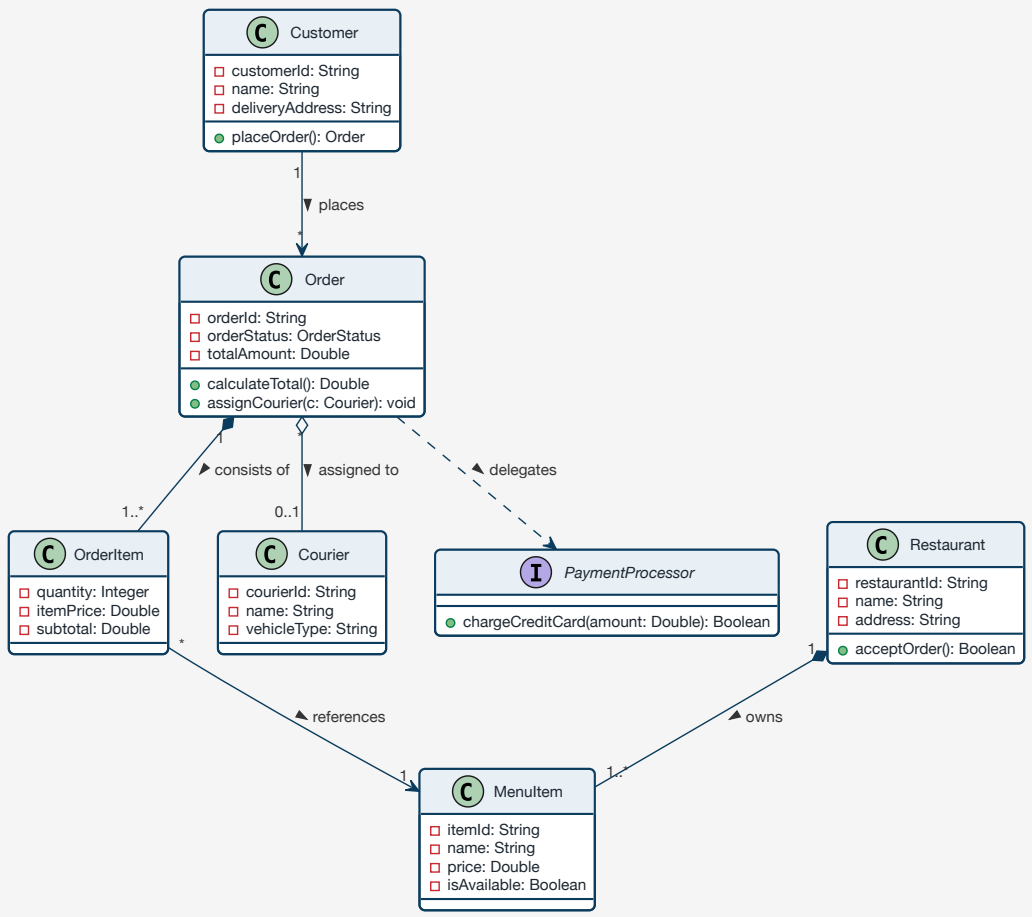
> *"A **Customer** has customerId, name, email, phone, and deliveryAddress.*
> *A **Restaurant** has restaurantId, name, address, and owns a collection of **MenuItems** (itemId, name, price, isAvailable).*
> *A Customer can place multiple **Orders**. Each **Order** has orderId, orderStatus (OrderStatus enum: Placed, Preparing, Delivered), orderTime, and totalAmount.*
> *An Order consists of one or more **OrderItems** (quantity, itemPrice, subtotal). If an Order is deleted, its OrderItems must be cascade-deleted immediately.*
> *A **Courier** (courierId, name, phone, vehicleType) can be assigned to deliver an Order.*
> *An Order delegates credit card charging to a **PaymentProcessor** interface."*

Generated PlantUML & Diagram: Food Delivery Class Model

Generated PlantUML Specification:

```
@startuml
class Customer {
  - customerId: String
  - deliveryAddress: String
  + placeOrder(): Order
}
class Restaurant {
  - restaurantId: String
  - address: String
  + acceptOrder(): Boolean
}
class MenuItem {
  - itemId: String
  - price: Double
}
class Order {
  - orderId: String
  - totalAmount: Double
  + calculateTotal(): Double
  + assignCourier(c: Courier): void
}
class OrderItem {
  - quantity: Integer
  - subtotal: Double
}
class Courier {
  - courierId: String
  - vehicleType: String
}
interface PaymentProcessor {
  + chargeCreditCard(amount: Double): Boolean
}

Restaurant "1" *-- "1..*" MenuItem : owns >
```

Rendered Diagram Output:



Interactive Activity: Evolving Food Delivery with AI (Pair Hands-On)

Hands-on Experiment: Mutate Requirements & Audit AI Output (4 Mins)

- **1. Add ONE New Rule to the Food Delivery Prompt:**
  - *Multi-Vendor:* Allow `Order` to bundle items from multiple `Restaurant`s.
  - *Tipping:* Allow `Customer` to tip and rate the assigned `Courier`.
  - *VIP Membership:* Add subscription tier (`Customer` gets free delivery).
- **2. Ask AI to Generate:** Prompt an LLM to update the PlantUML class diagram.
- **3. Render & Critique:** Paste into PlantText and audit:
  - Did AI preserve **Composition** (`*--`) vs. **Aggregation** (`o--`)?
  - Are **multiplicities** (`1..*`, `0..1`) logically sound for the new rule?
  - Did AI hallucinate redundant classes or violate `- private` fields?

Concept Check Question 4

In our Food Delivery Class Diagram, why is the relationship between `Order` and `OrderItem` modeled as **Composition (◆)**, whereas `Order` and `Courier` is modeled as **Aggregation (◇)**?

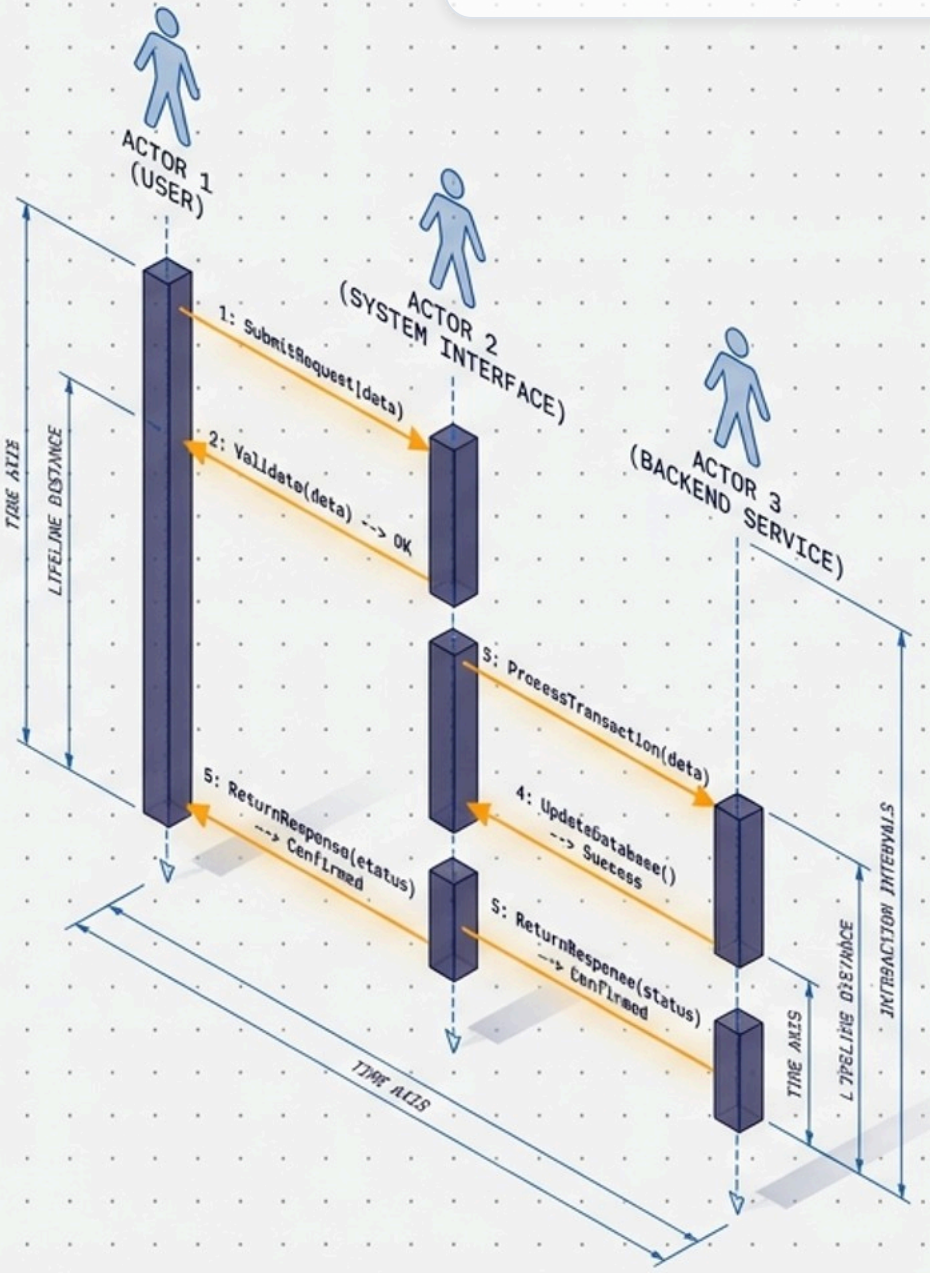
- **A.** An `OrderItem` can exist independently, but a `Courier` cannot exist without an `Order`.
- **B.** An `OrderItem` is destroyed with its parent `Order`, whereas a `Courier` maintains an independent lifecycle.
- **C.** Composition represents inheritance between classes, while aggregation represents method invocation.
- **D.** Composition requires zero-to-one multiplicity, while aggregation requires one-to-many multiplicity.

4.5 Interaction & Sequence Diagrams

> *"Interaction modeling shows how objects collaborate chronologically over time to fulfill the promise of a use case."*

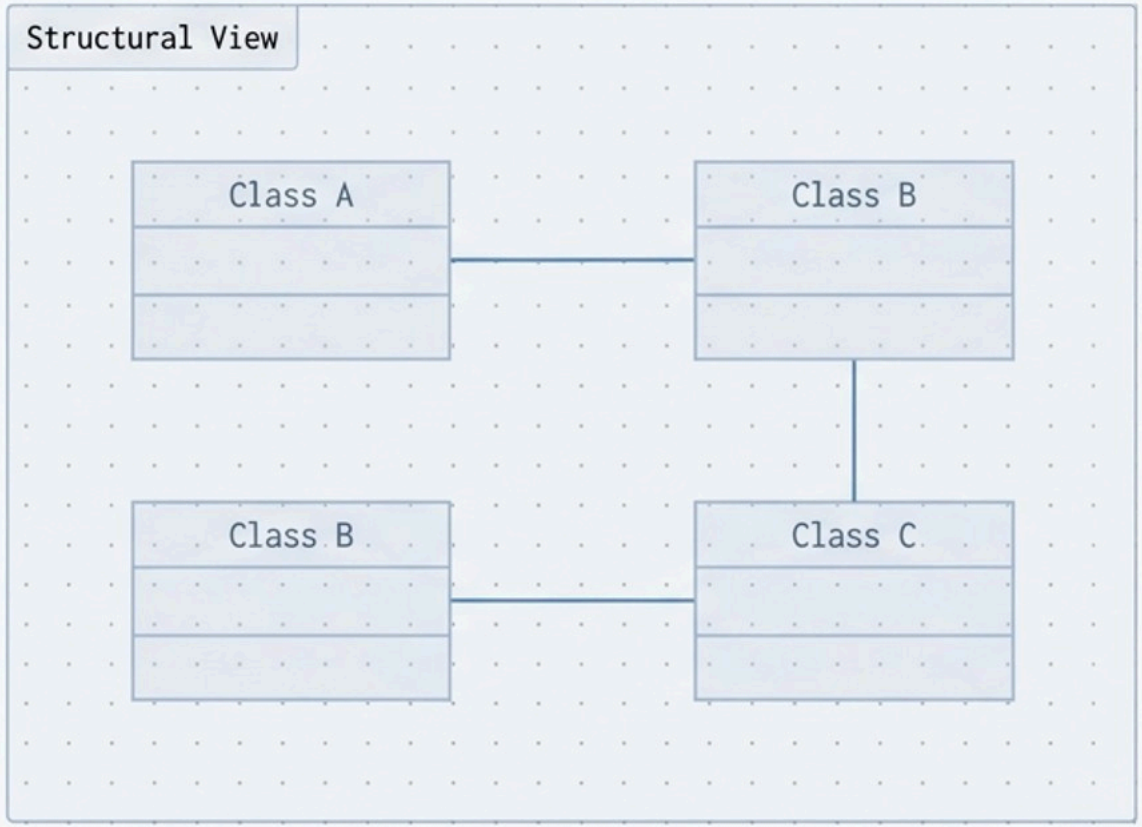
Sequence Diagrams: The Architectural Blueprint

A visual reference for modeling system interactions and logic.

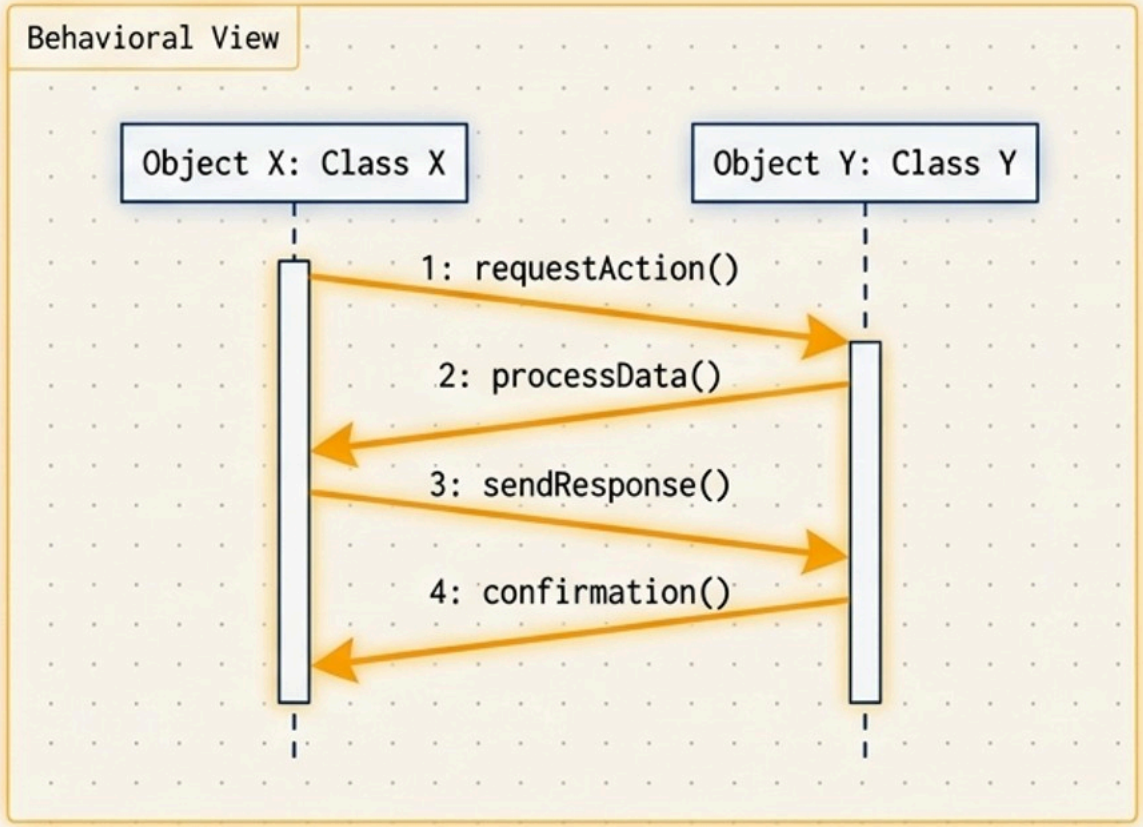


Mapping Dynamic Collaboration

STATIC MODEL

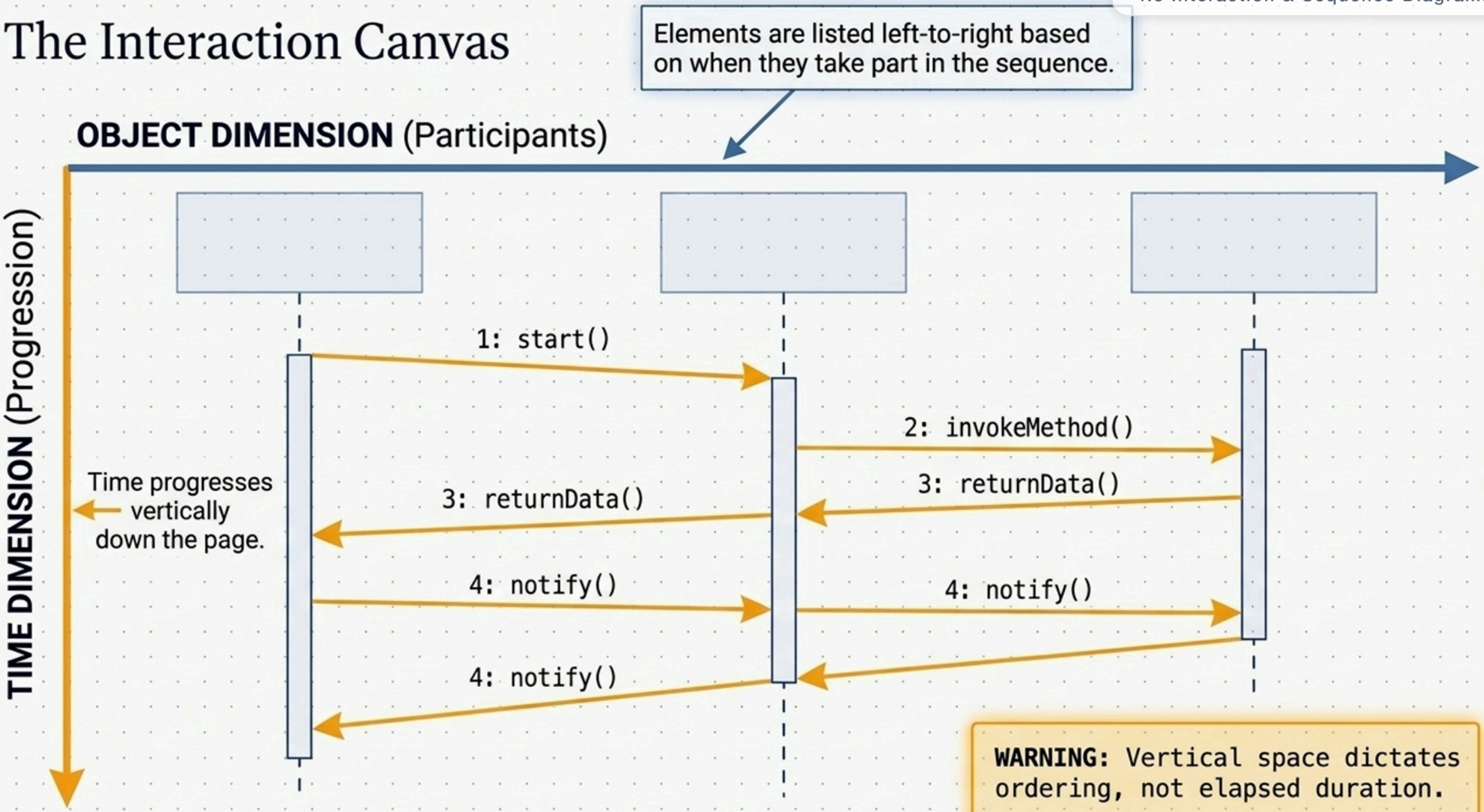


DYNAMIC MODEL

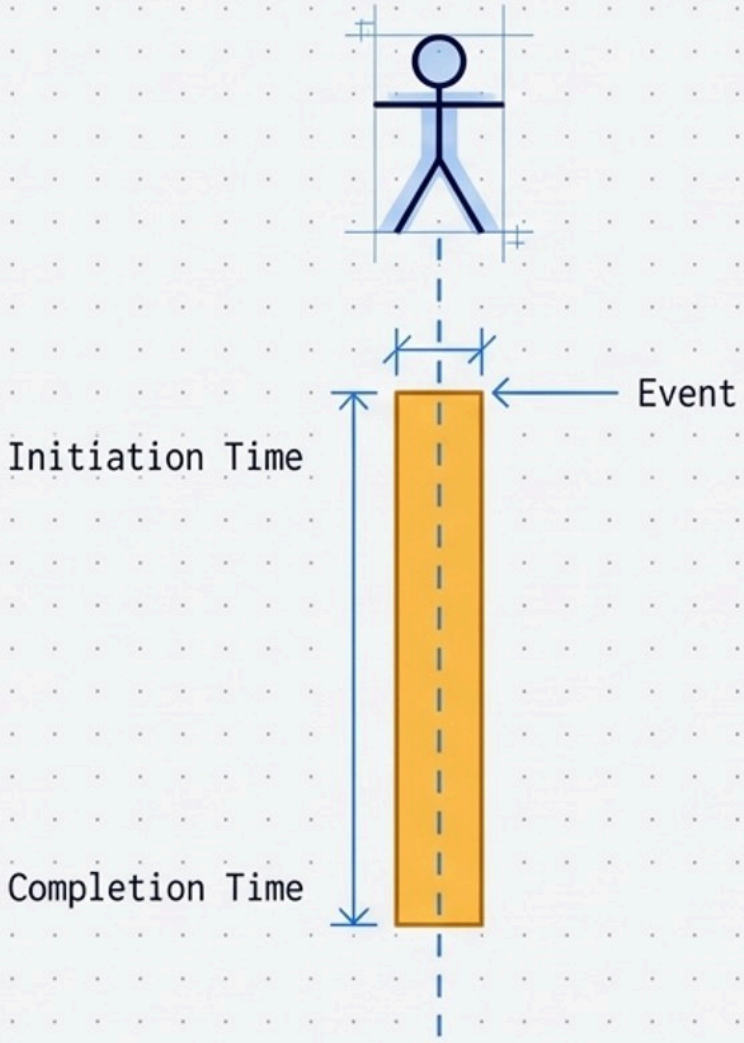


UML Sequence Diagrams detail how operations are carried out over time. Unlike class or object diagrams that provide static views, sequence diagrams capture the dynamic flow of messages sent and received within a collaboration.

The Interaction Canvas



Structural Anatomy



Actor

External entity interacting with the subject. A role, not necessarily a physical person.

Lifeline

An individual participant in the interaction.

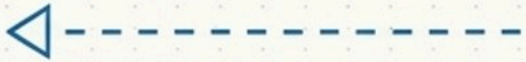
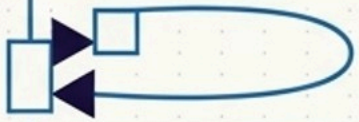
Focus of Control

The period during which an element performs an operation. Visualized by the activation rectangle.

Event

Any precise point where an occurrence happens (e.g., sending or receiving a message).

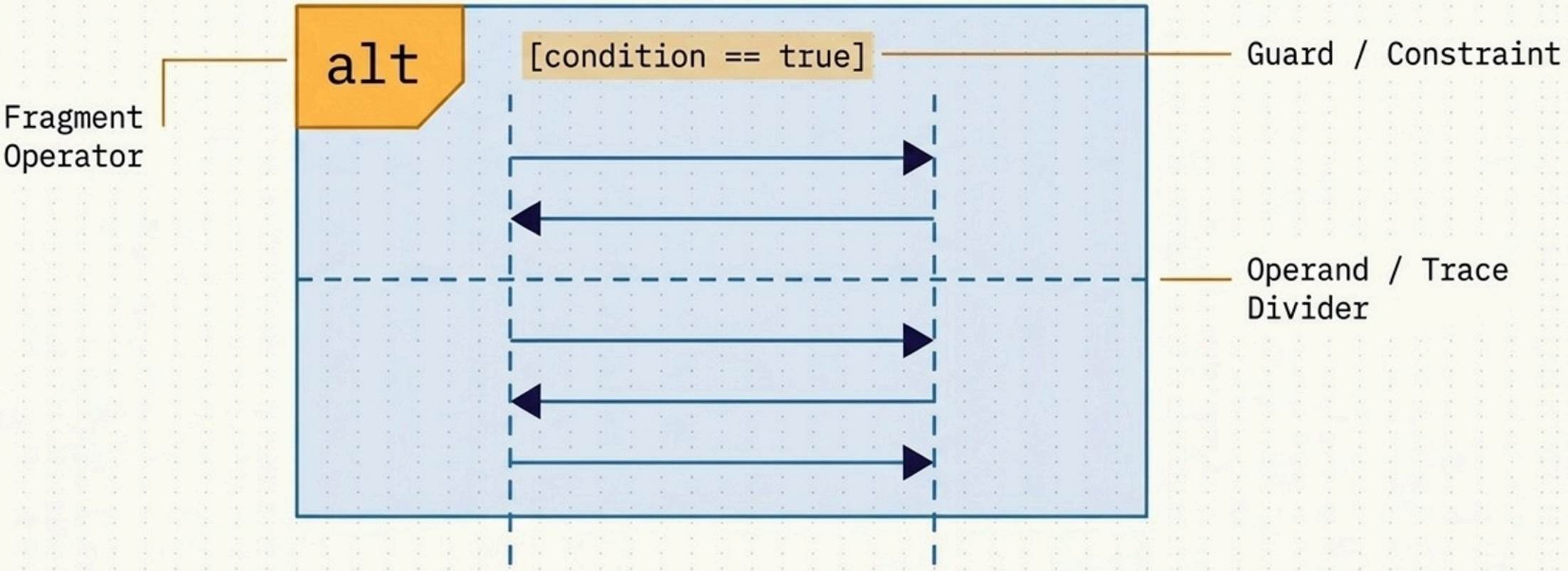
The Messaging Matrix

Message Type	Visual Notation	Semantic Meaning
`Call`		Invokes an operation on a target lifeline.
`Return`		Passes information back to the caller of a former message.
`Self`		Invokes a message on the same lifeline.
`Recursive`		Invokes a message targeting a new activation stacked on the caller.
`Create`		Instantiates a new target lifeline.
`Destroy`		Requests the destruction of the target lifeline's lifecycle.
`Duration`		Shows the distance between two time instants for an invocation.

Note: A comment carries no semantic force but attaches useful remarks.

UML 2.0 Sequence Fragments

Combined Fragments enclose a portion of the interaction to capture branching, loops, and parallel execution, moving beyond single linear paths.

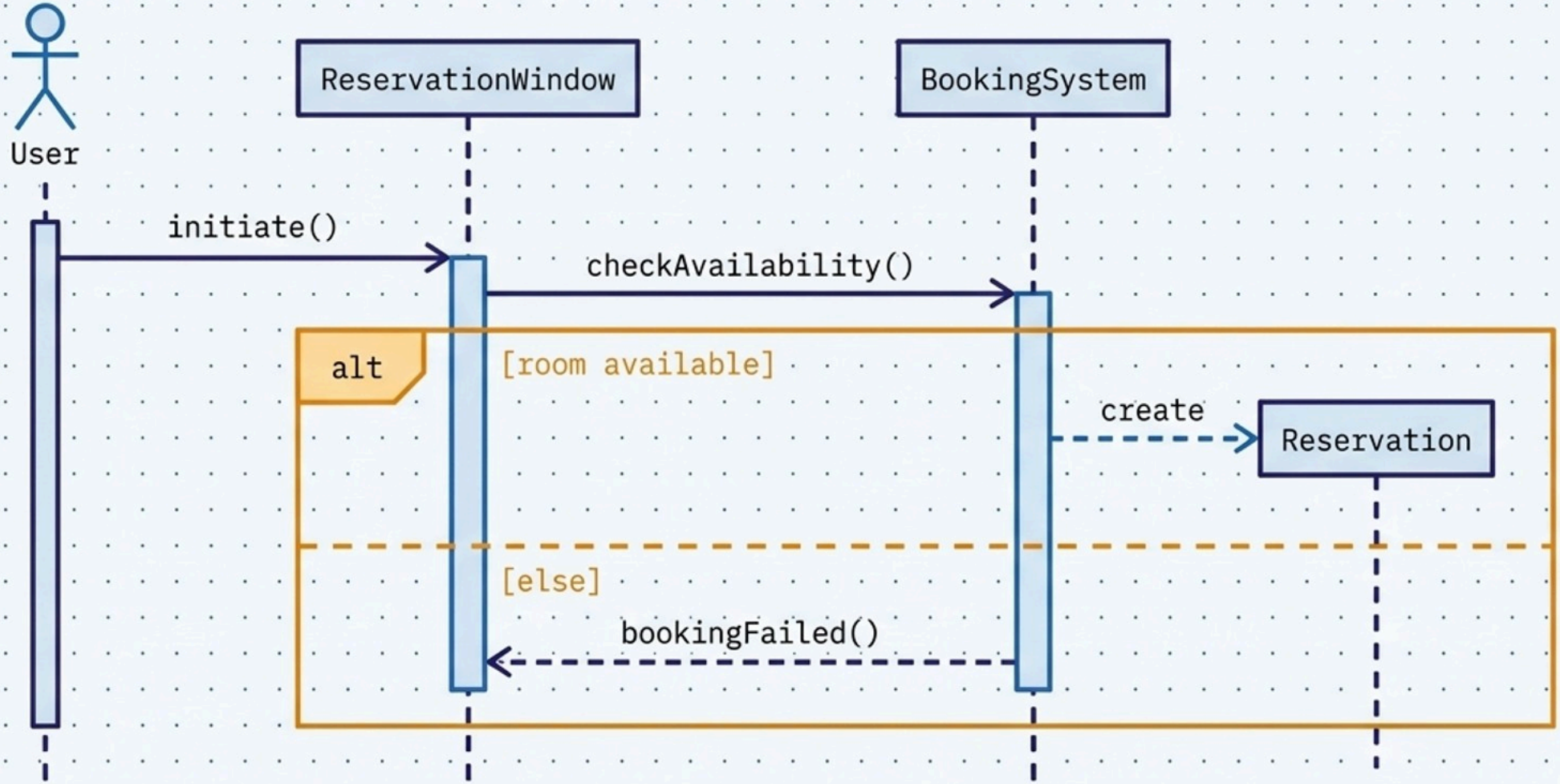


Fragment Operators to Code Logic

Operator	Definition	Code Equivalent
`alt`	**Alternative:** Multiple fragments; only the one with a true condition executes.	`if / else if / else`
`opt`	**Optional:** Executes only if the supplied condition is true. Single trace.	`if (without else)`
`loop`	**Loop:** Executes multiple times based on a guard condition.	`while / for loop`
`par`	**Parallel:** Each fragment runs concurrently.	`multi-threading / async`
`region`	**Critical Region:** Only one thread can execute this fragment at once.	`Mutex lock / sync`
`neg`	**Negative:** Shows an explicitly invalid interaction.	`Error path / Exception`
`ref`	**Reference:** Refers to an interaction defined on another diagram.	`Function / Module call`
`sd`	**Sequence Diagram:** Surrounds an entire root sequence diagram.	`Root program`

System Model: Hotel Reservation

Notice how the ReservationWindow initiates the sequence, progressing downward through an “alt” fragment to handle availability logic dynamically.



The Requirements-to-Code Pipeline

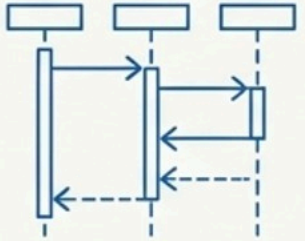
1. Use Case

The specification of a sequence of actions a system can perform with external actors.

2. Scenario

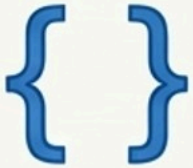
One specific path or flow through that high-level use case.

3. Sequence Diagram



The structural interaction modeled precisely over time.

4. Implementation



The final compiled algorithms and system code.

A scenario is a sequence of events occurring during one execution of a system—the Sequence Diagram is its exact architectural blueprint.

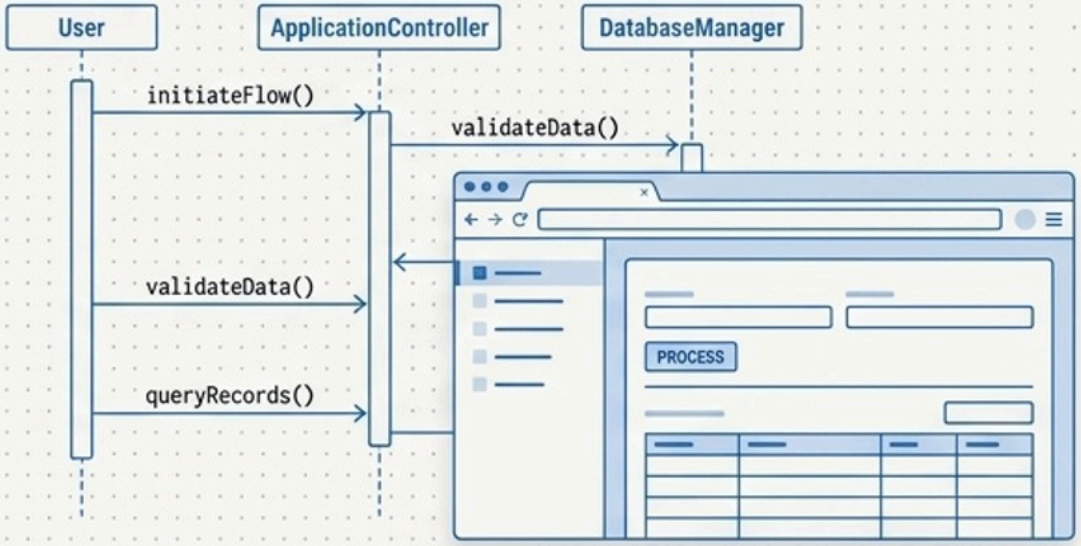
Model Before Code

CHAOTIC RAW CODE

```
if(data){
  return;
} else {
  ...
}
  function calculateRisk(input){
    //todo: implement
  }
       FOR user IN db.users
       SELECT * FROM...
   class SystemController {
     private int id; ...
   }
```

```
{
users {
  ...
  ...
}
{ }
ERROR: null pointer
 exception in module X
/> </
[ ]
[]
```

PRISTINE MODEL & BLUEPRINT



Language Neutral
Design systems without being locked into Java, Python, or C++ constraints.

Cross-Functional
Readable by non-coders; aligns business analysts and developers early.

Team Consensus
Easier to collaborate and debate a visual flow than to rewrite pull requests.

Test & UX Ready
Perfect foundational asset for writing accurate test cases and UI/UX.

Architecture is cheaper to change than code.
Draft the blueprint first.

AI Assistance: Guidelines for Sequence Modeling

- **1. Enforce Architectural Layering:**
  - Instruct AI to follow a clean layered structure (UI → Controller / Business Logic → Entities & Databases).
  - Prevent AI from taking shortcuts like connecting UI directly to database storage.
- **2. Capture Interaction Dynamics:**
  - **Synchronous Requests & Replies:** Clearly show calls and corresponding response acknowledgments.
  - **Branching (Success vs. Failure):** Require AI to handle both normal success and alternative exception scenarios (e.g., payment failure).
  - **Decoupled / Asynchronous Tasks:** Distinguish immediate blocking actions from background notifications (e.g., alert queues).
- **3. Human Reality Check:**
  - Did AI bypass business logic layers?
  - Are alternate scenarios mutually exclusive with clear conditions?

AI Prompt Example: Food Delivery Sequence Model

1. Role & Task Instruction:

```
Act as a Backend Systems Architect.
Generate a clean PlantUML Sequence Diagram
for a Food Delivery Checkout flow.
```

2. Modeling Guidance:

- Follow layered architecture (UI → Controller → Entities / External APIs).
- Explicitly model both paths: **Payment Approved** vs. **Payment Declined**.
- Show asynchronous handoff for background kitchen notification.
- Show active execution lifespans during processing.

3. Input Requirements Statement:

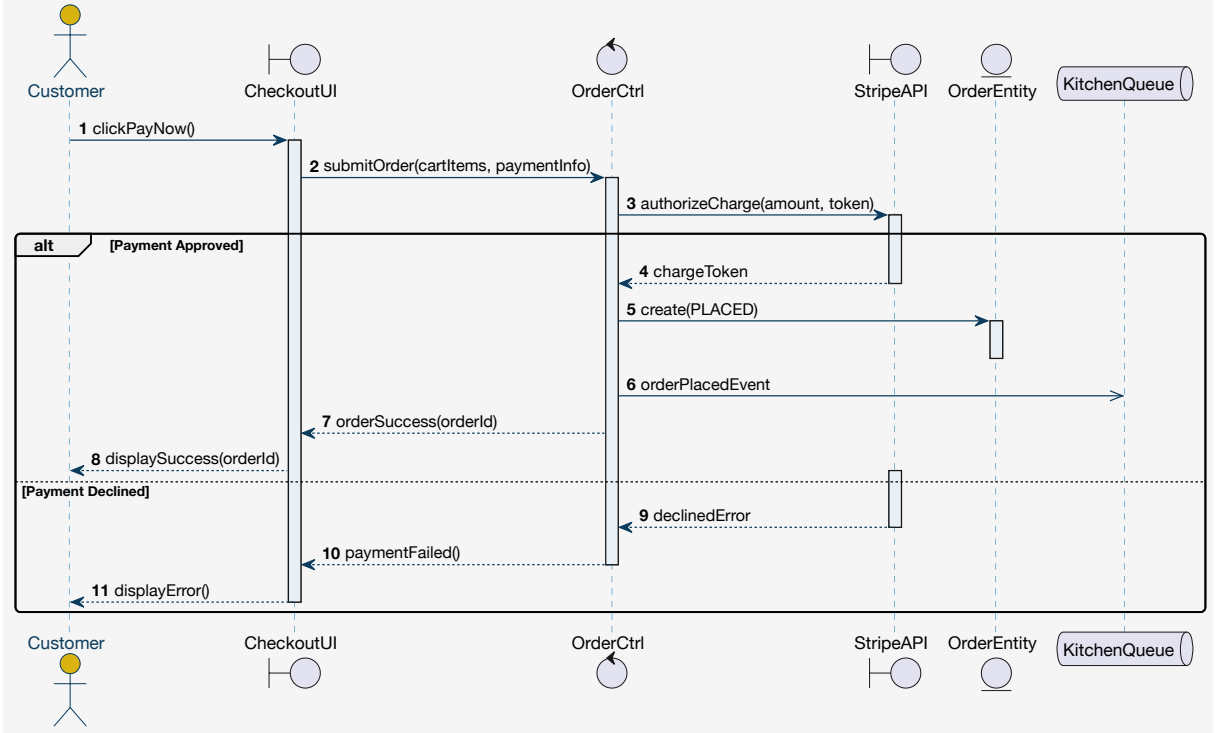
> *"A Customer initiates checkout by clicking 'Pay Now' on the **CheckoutUI**.*
> *The **CheckoutUI** forwards* `submitOrder(cartItems, paymentInfo)` *to the **OrderController**.*
> *The **OrderController** calls **PaymentGateway*** `authorizeCharge(amount, token)`.
>
> ***If payment is approved:***
>
> - *PaymentGateway returns* `chargeToken`.
> - *OrderController persists **Order** as* `PLACED`*, publishes async* `orderPlacedEvent` *to **KitchenOrderQueue**, and returns* `orderSuccess(orderId)` *to CheckoutUI.*
>
> ***If payment is declined:***
>
> - *PaymentGateway returns* `declinedError`.
> - *OrderController logs failure and returns* `paymentFailed()` *to CheckoutUI without creating an order."*

Generated PlantUML & Diagram: Food Delivery Sequence Model

Generated PlantUML Specification:

```
@startuml
autonumber
actor Customer
boundary CheckoutUI
control OrderCtrl
boundary StripeAPI
entity OrderEntity
queue KitchenQueue

Customer -> CheckoutUI: clickPayNow()
activate CheckoutUI
CheckoutUI -> OrderCtrl: submitOrder(cart, payment)
activate OrderCtrl
OrderCtrl -> StripeAPI: authorizeCharge(amount, token)
activate StripeAPI

alt Payment Approved
  StripeAPI --> OrderCtrl: chargeToken
  deactivate StripeAPI
  OrderCtrl -> OrderEntity: create(PLACED)
  OrderCtrl ->> KitchenQueue: orderPlacedEvent
  OrderCtrl --> CheckoutUI: orderSuccess(orderId)
  CheckoutUI --> Customer: displaySuccess()
else Payment Declined
  StripeAPI --> OrderCtrl: declinedError
  OrderCtrl --> CheckoutUI: paymentFailed()
  deactivate OrderCtrl
```

Rendered Diagram Output:



Interactive Activity: Evolving Sequences with AI (Pair Hands-On)

Hands-on Experiment: Mutate Flow & Audit AI Output (4 Mins)

- **1. Add ONE Edge Case to the Checkout Flow:**
  - *Gateway Timeout:* Payment gateway takes >5s to respond (trigger retry or fallback).
  - *Split Payment:* Allow customer to pay with partial Wallet Balance + Credit Card.
  - *Out of Stock:* Restaurant runs out of an item during checkout.
- **2. Ask AI to Generate:** Prompt an LLM to update the PlantUML sequence diagram.
- **3. Render & Critique:** Paste into PlantText and audit:
  - Did AI preserve **layering** (UI → Controller) without UI querying DB directly?
  - Are **success vs. failure branches** clearly partitioned with return messages?
  - Are **background alerts** (e.g., kitchen notifications) properly decoupled?

Concept Check Question 5

In a Boundary-Control-Entity (BCE) sequence diagram, which object should receive the customer's `submitOrder()` event from the checkout UI boundary?

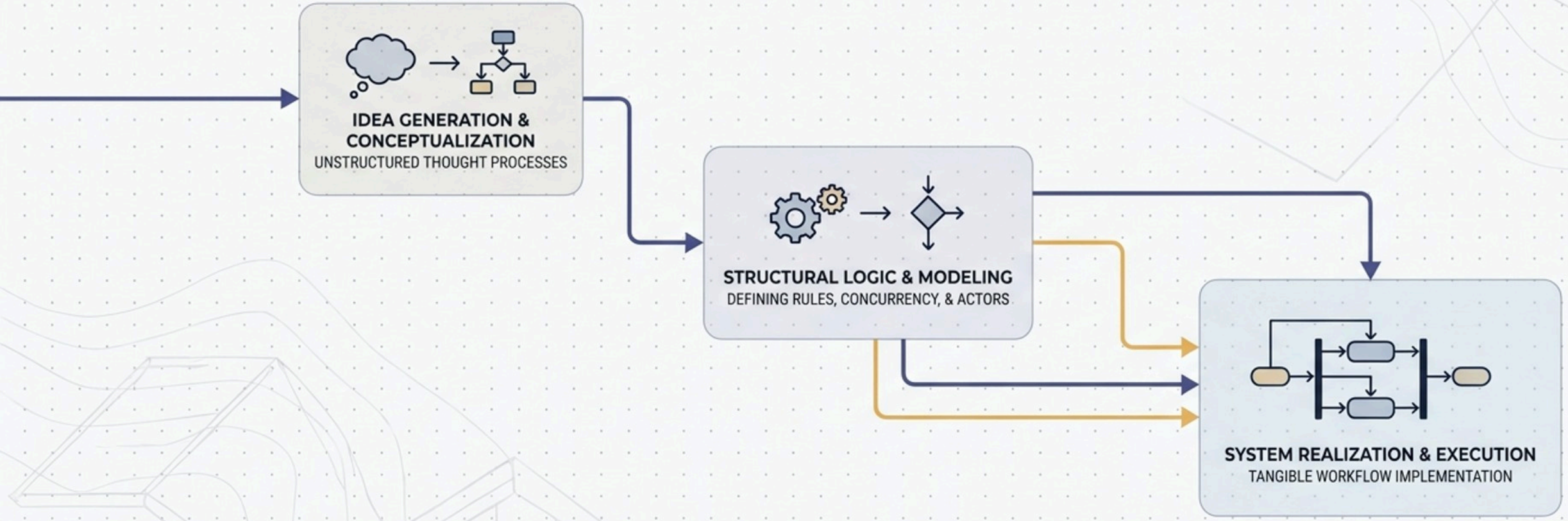
- **A.** The `Order` entity object to immediately save database state.
- **B.** The `PaymentGatewayAPI` boundary object to process payment immediately.
- **C.** The `OrderController` control object to orchestrate business validation and service calls.
- **D.** The `Restaurant` entity object to confirm kitchen capacity.

4.6 Process & Activity Diagrams

> *"Activity diagrams map the dynamic flow of control, concurrency, and data across collaborating participants."*

Mapping the Dynamic System

A guide to modeling complex, concurrent, and multi-actor workflows with UML Activity Diagrams.



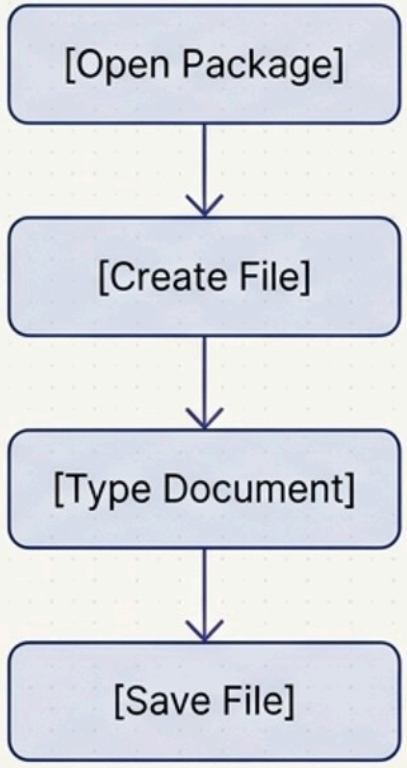
Moving beyond static structures to model dynamic behavior

An Activity Diagram is a crucial behavioral diagram in the Unified Modeling Language (UML). It is an advanced version of a flowchart designed to describe the dynamic aspects of a system, mapping the precise flow from one activity to the next.

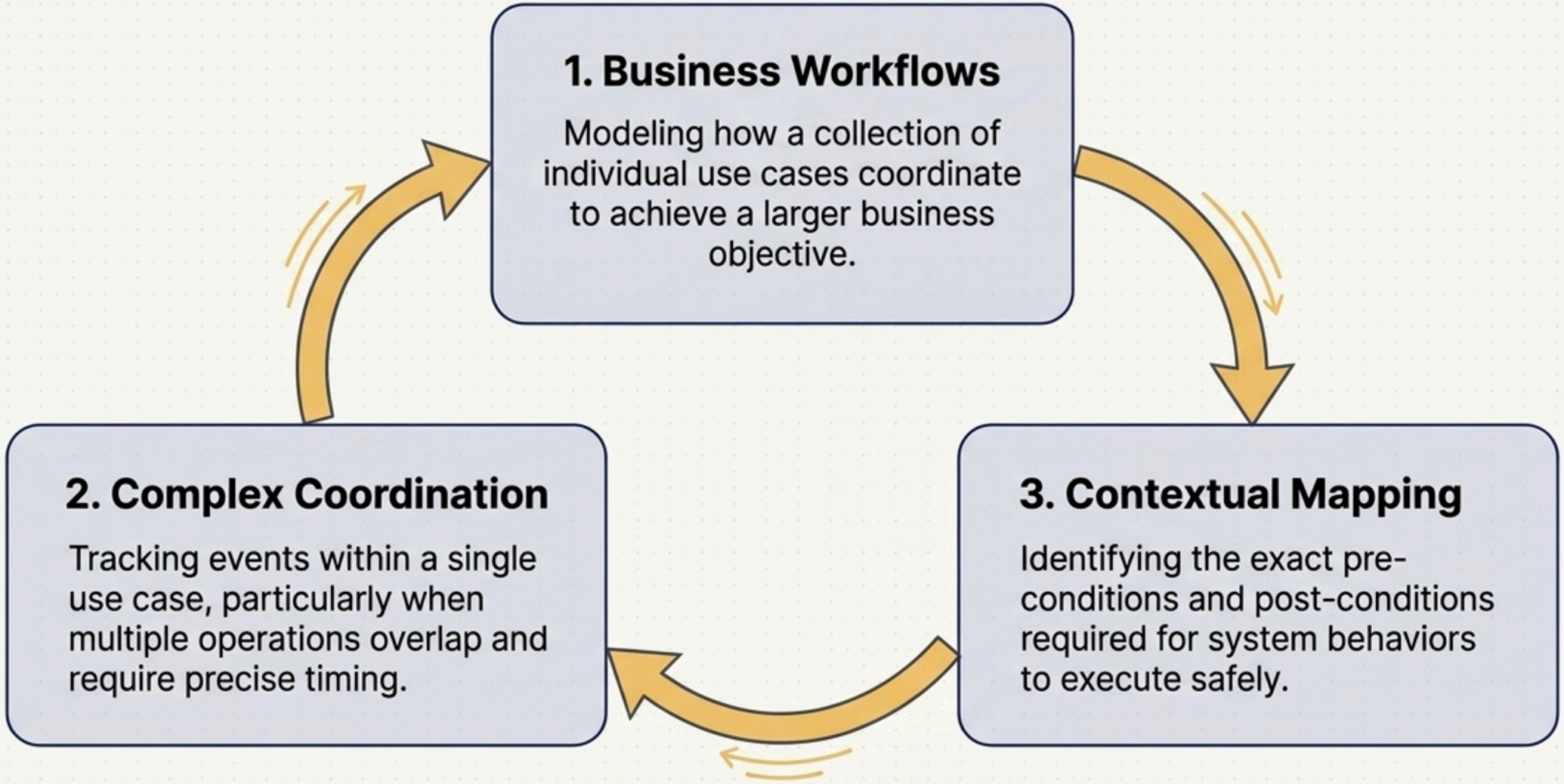
The Chaos: Text-Based Processes

Process starts when the user opens the package and then needs to initialize the system, following which they must create must create a new file and ensure all settings are correct, after which they proceed to type the document, remembering to format it according to the guidelines, and finally, after reviewing, they must save the file in the appropriate directory before closing the application and moving on to the next task.

The Clarity: Visual Logic



Three triggers for behavioral modeling



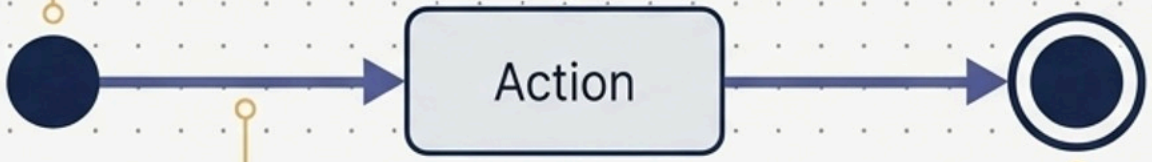
Standard flowcharts fail at systemic complexity

Capability	Standard Flowchart	UML Activity Diagram
Sequential Logic	✓	✓
Conditional Splits	✓	✓
Parallel / Concurrent Processes	✗	✓
Actor Accountability (Swimlanes)	✗	✓
Object States & Transformation	✗	✓

The core vocabulary of system flow

Initial Node
Portrays the spark—the beginning of a set of actions or activities.

Action
A specific, singular task to be performed by the system or actor.

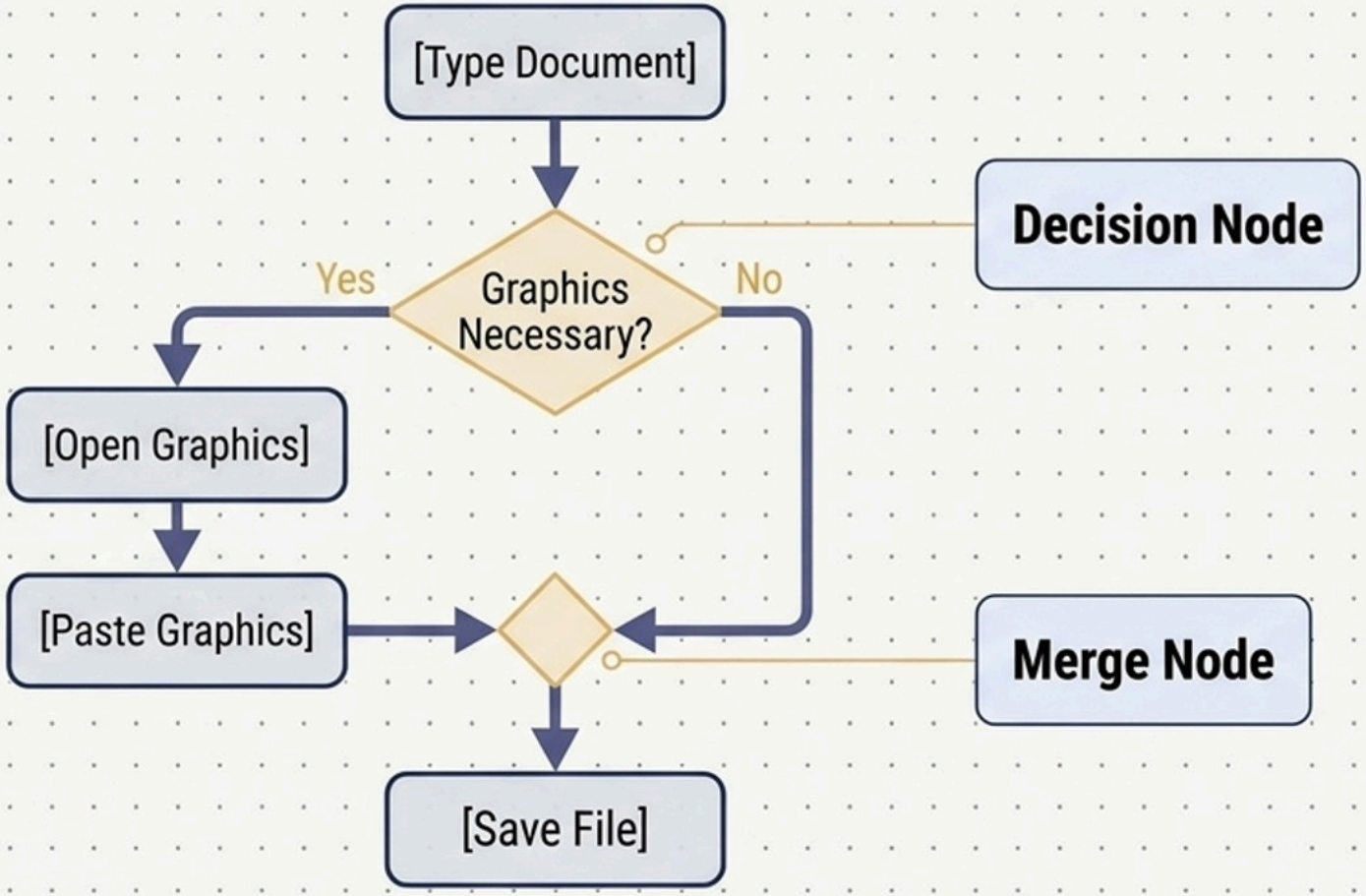


Control Flow
Shows the strict sequence of execution from one node to the next.

Activity Final Node
Stops all control and object flows, terminating the sequence.

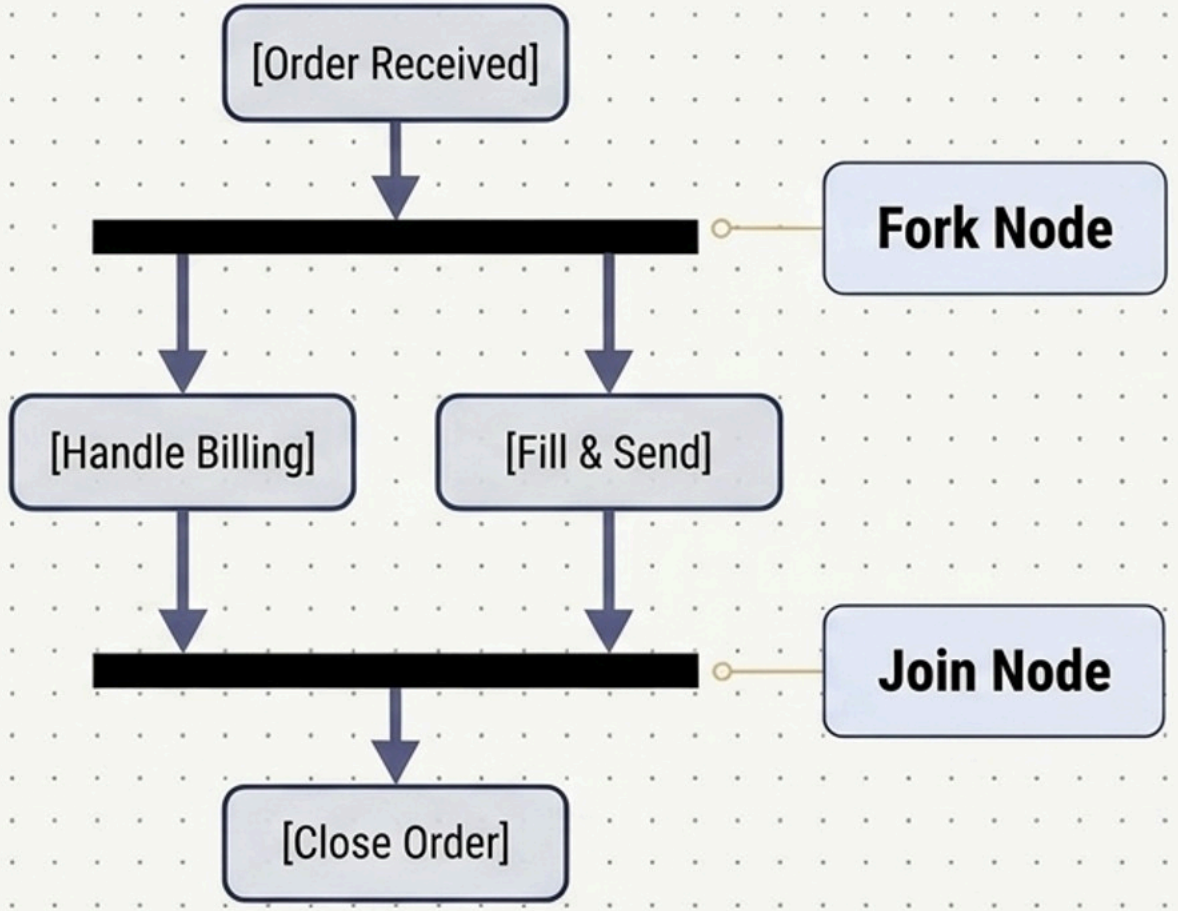
Handling conditional system logic

When a system must evaluate a test condition, a Decision Node ensures the control flow only proceeds down a single valid path. A Merge Node safely brings these disparate decision paths back together.



Orchestrating parallel system actions

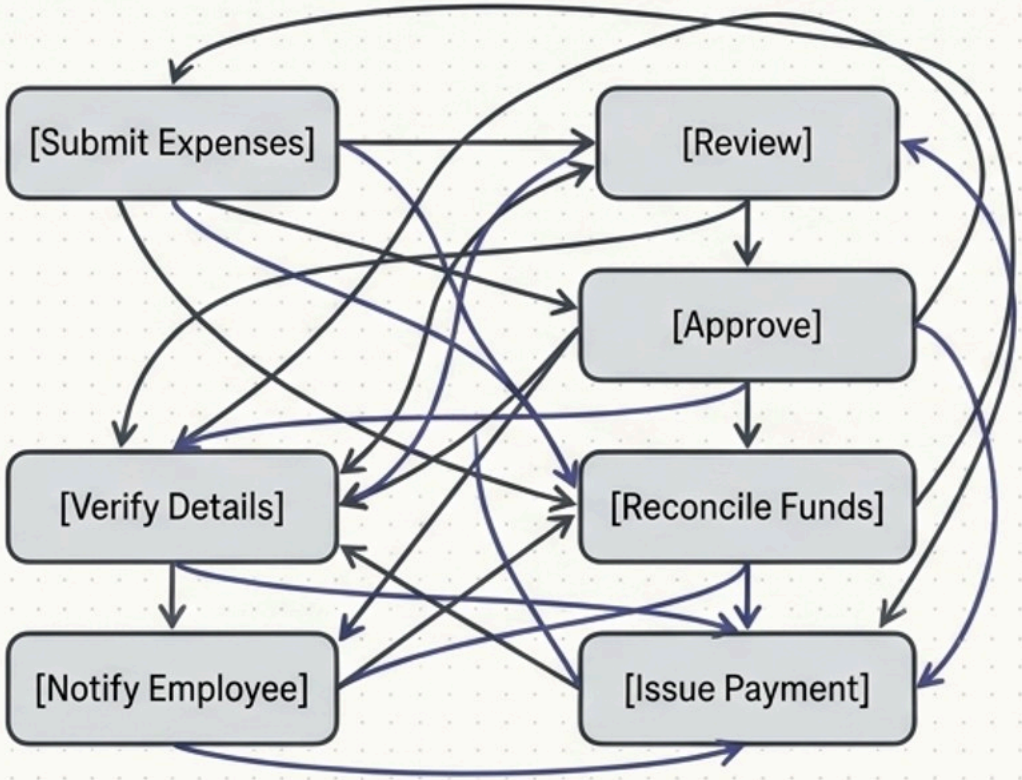
Systems rarely execute one step at a time. The **Fork Node** splits behavior into concurrent actions. The **Join Node** ensures the system waits to bring parallel flows back together before proceeding.



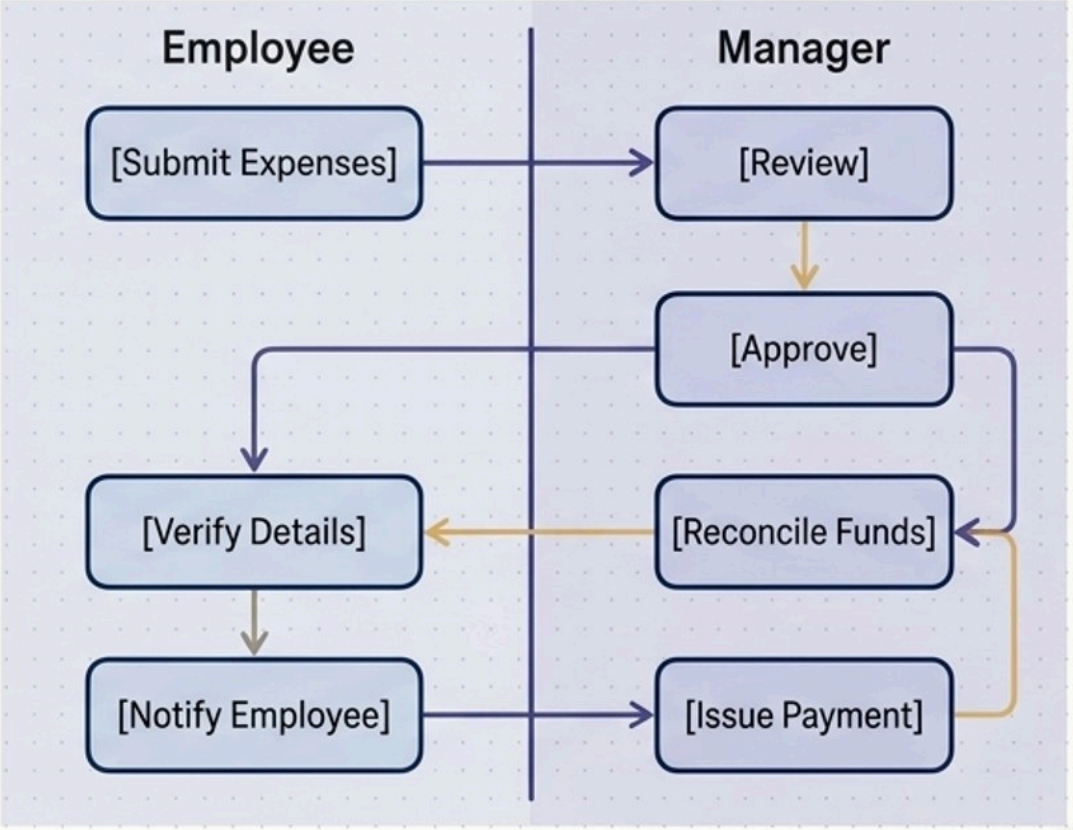
The multi-actor accountability problem

Knowing WHAT happens in a business process is useless if you don't know WHO executes it. Without boundaries, complex workflows degrade into ambiguity.

Without Swimlanes: Ambiguity

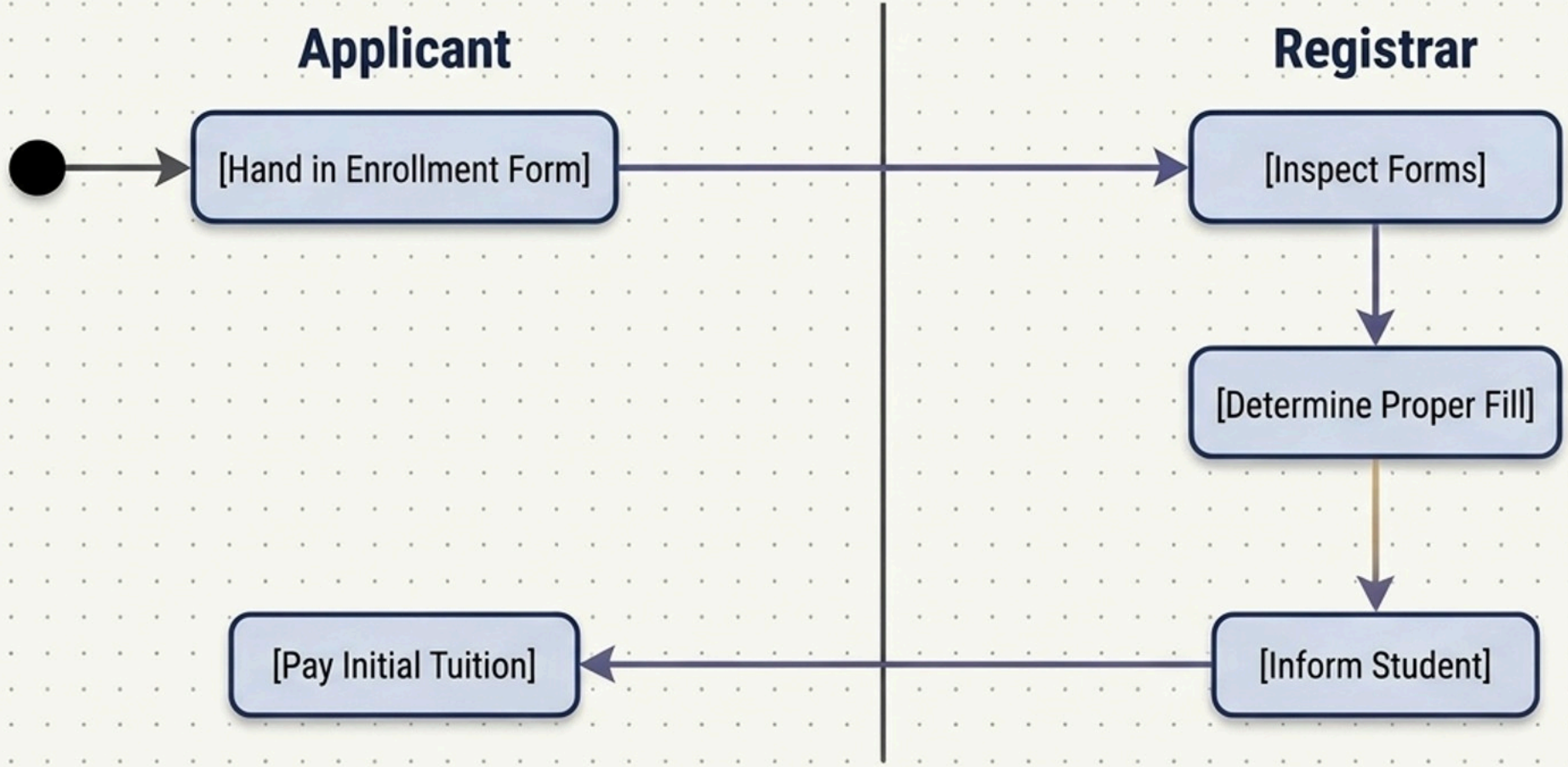


With Swimlanes: Accountability



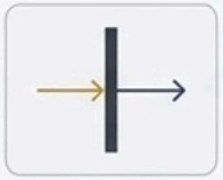
Grouping activities by actor or thread

A Swimlane (or Partition) groups activities performed by the same actor. It visualizes hand-offs seamlessly—such as a student passing paperwork to a registrar and waiting for system validation.



The complete visual taxonomy of dynamic modeling

The Actors



Swimlane / Partition

Groups activities by actor or execution thread.

The Actions

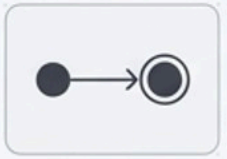
Action Node

A single task to be performed.

Object Node

Represents a connected physical or digital artifact.

The Directors



Initial & Final Nodes

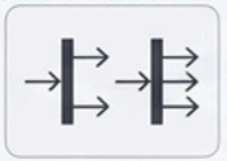
Start and stop all system flows.

Control & Object Flows

Sequence execution and data movement.

Decision & Merge Nodes

Split and return conditional paths.

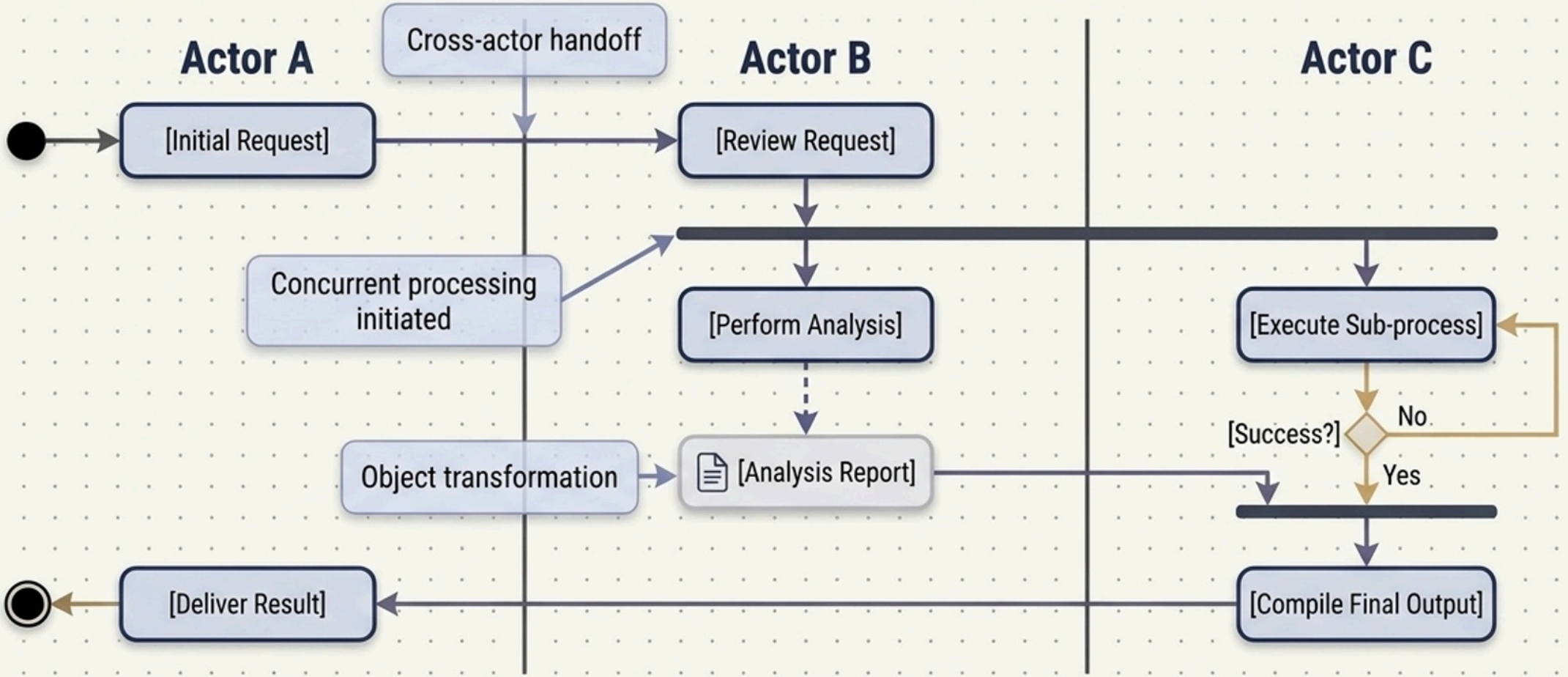


Fork & Join Nodes

Split and return concurrent parallel paths.

The unified workflow model

When Swimlanes, Concurrency, and Conditional logic combine, complex systemic chaos is transformed into a readable, executable blueprint.



Three phases to map your systems

1. Examine the Business Context

Identify candidate use cases by analyzing broader business workflows. Map the precise pre-conditions and post-conditions required.

2. Map the Core Workflows

Build out the activity logic mapping the flow between different use cases, and the sequences contained within them.

3. Zoom In on Complexity

Create detailed, high-level activity diagrams to map out highly complex workflows and intricate operations on specific objects.

AI Assistance: Guidelines for Activity Modeling

- **1. Establish Clear Role Boundaries (Swimlanes):**
  - Partition actions by role or system (e.g., Customer, Kitchen, Courier, Dispatch) so every action has an explicit owner.
- **2. Identify True Concurrency & Synchronization:**
  - **Parallel Workflows:** Guide AI to recognize independent steps that happen at the same time (e.g., cooking food while dispatching a driver).
  - **Synchronization Milestones:** Explicitly specify where parallel branches must wait and join before the workflow can proceed.
- **3. Explicit Decision Points:**
  - Ensure branch conditions are clear and cover all possible outcomes.
- **4. Human Reality Check:**
  - Did AI collapse parallel work into a purely sequential list?
  - Does every split path properly reunite at a synchronization point?

AI Prompt Example: Food Delivery Activity Model

1. Role & Task Instruction:

```
Act as a Workflow & Process Architect.
Generate a clean PlantUML Activity Diagram
using swimlanes and parallel processing.
```

2. Modeling Guidance:

- Divide workflow into 4 roles: Customer, Kitchen, Dispatch, Courier.
- Identify parallel steps after order payment (cooking meal while matching courier).
- Synchronize branches: courier cannot pick up meal until both cooking is finished and courier has arrived.
- Clearly show process start, decision points, and completion.

3. Input Requirements Statement:

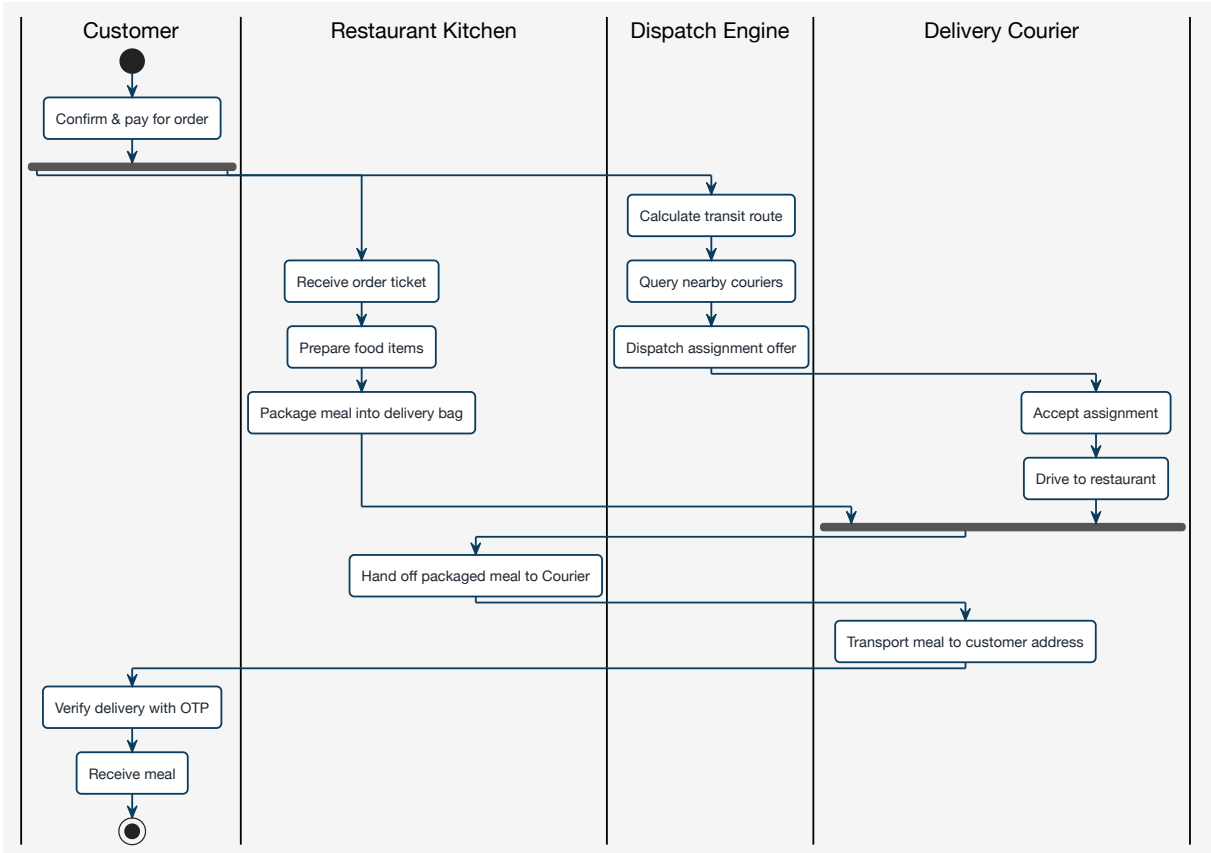
> *"Workflow begins in the **Customer** swimlane when the user confirms and pays for an order.*
> *Once confirmed, two parallel processes initiate:*
>
> - *In **Restaurant Kitchen**: Staff receive order ticket, prepare food items, and package the meal into a delivery bag.*
> - *Simultaneously, in **Dispatch Engine**: The system calculates transit route, queries nearby available couriers, and dispatches a delivery offer.*
>
> *In the **Delivery Courier** swimlane, the courier accepts the assignment and drives to the restaurant.*
> ***Synchronization Milestone:** The courier cannot pick up the order until BOTH the food packaging is complete AND the courier has arrived at the store.*
> *Once synchronized, kitchen hands off food to courier.*
> *Courier transports meal to customer address, customer verifies delivery with OTP, and the process ends."*

Generated PlantUML & Diagram: Food Delivery Activity Model

Generated PlantUML Specification:

```
@startuml
|Customer|
start
:Confirm & pay for order;

fork
  |Restaurant Kitchen|
  :Receive order ticket;
  :Prepare food items;
  :Package meal into delivery bag;
fork again
  |Dispatch Engine|
  :Calculate transit route;
  :Query nearby couriers;
  :Dispatch assignment offer;
  |Delivery Courier|
  :Accept assignment;
  :Drive to restaurant;
end fork

|Restaurant Kitchen|
:Hand off packaged meal to Courier;

|Delivery Courier|
:Transport meal to customer address;

|Customer|
```

Rendered Diagram Output:



Interactive Activity: Evolving Workflows with AI (Pair Hands-On)

Hands-on Experiment: Mutate Process & Audit AI Output (4 Mins)

- **1. Add ONE Process Rule to the Activity Prompt:**
  - *Courier Reassignment:* If matched courier declines, re-dispatch to another courier.
  - *Quality Check:* Kitchen inspects food; if burnt, redo cooking before packaging.
  - *Customer Self-Pickup:* If customer chooses takeaway, bypass courier swimlane.
- **2. Ask AI to Generate:** Prompt an LLM to update the PlantUML activity diagram.
- **3. Render & Critique:** Paste into PlantText and audit:
  - Did AI maintain clear **swimlanes** for all participating roles?
  - Are **parallel splits and joins** properly paired without deadlocks?
  - Are **decision diamonds** fully labeled with complete branch conditions?

Concept Check Question 6

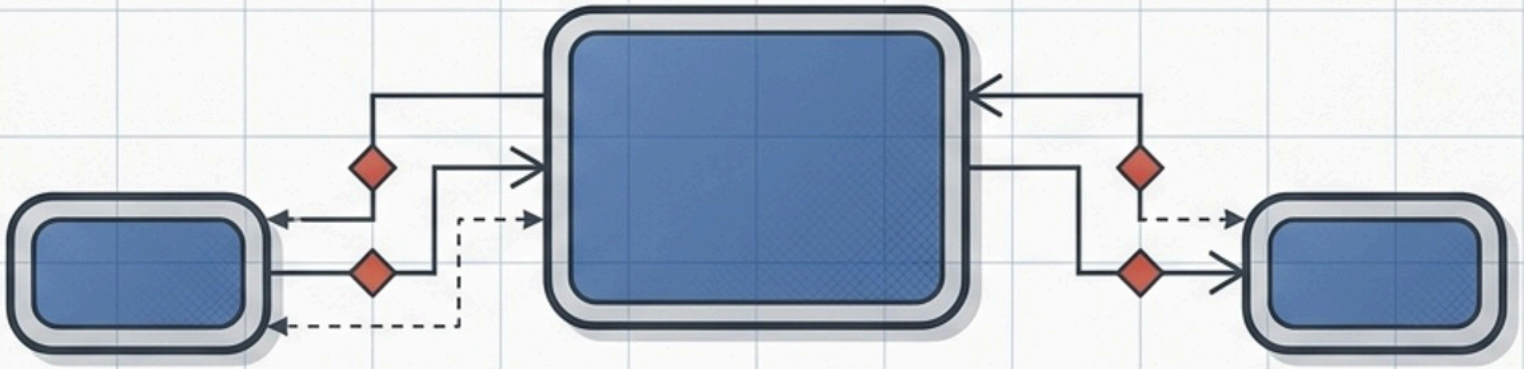
In a UML Activity Diagram, what is the critical behavioral difference between a **Fork/Join** synchronization bar and a **Decision/Merge** diamond?

- **A.** Fork/Join splits and synchronizes concurrent parallel threads; Decision/Merge evaluates guards to pick exactly one mutually exclusive branch.
- **B.** Fork/Join is used for sequential database transactions; Decision/Merge is used for swimlane partitioning.
- **C.** Fork/Join models class inheritance; Decision/Merge models object creation.
- **D.** Fork/Join requires human operator approval; Decision/Merge is executed by automated timers.

4.7 Behavioral & State Machine Models

> *"A system in dynamic execution is defined by the states it occupies and the events that trigger transitions."*

The Anatomy of State-Dependent Behavior

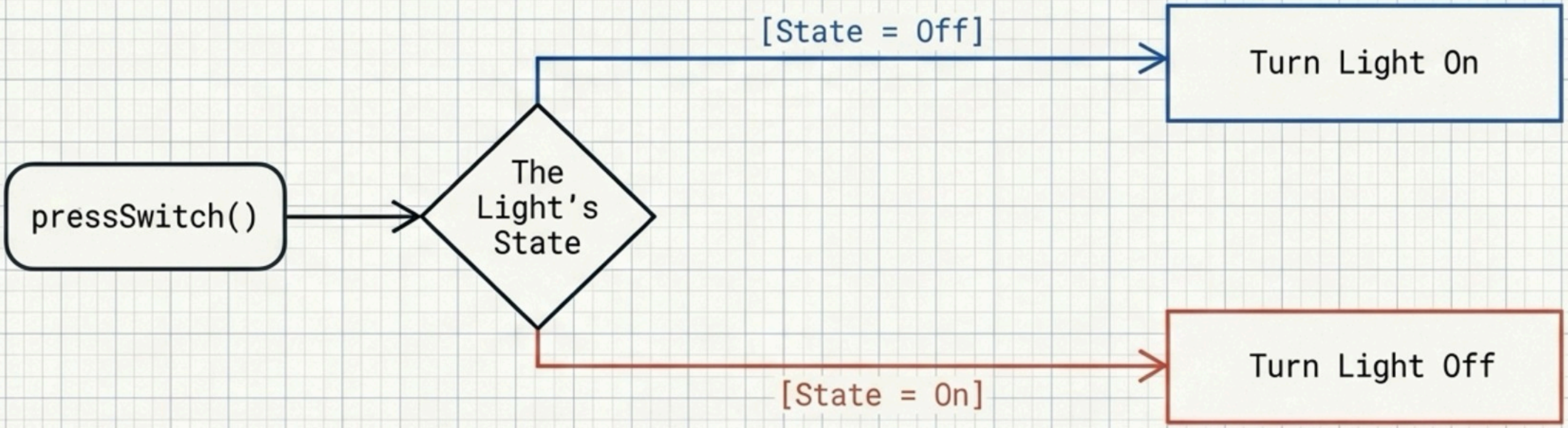


A definitive framework for modeling, deconstructing, and designing UML State Machine Diagrams.

v. Unified Modeling Language | Blueprint Series

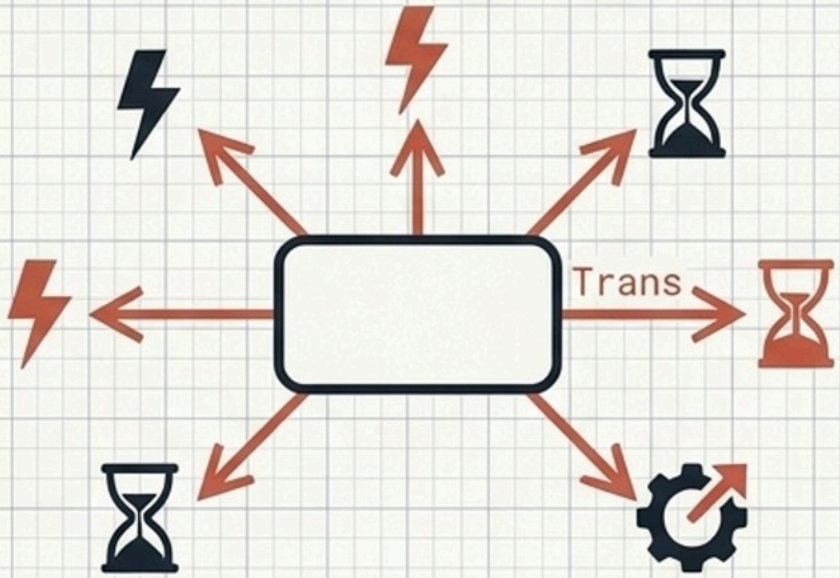
The Same Event Yields Different Results Based on State

A system's behavior is not just a consequence of its current input.
It is entirely dependent on its preceding state and past history.



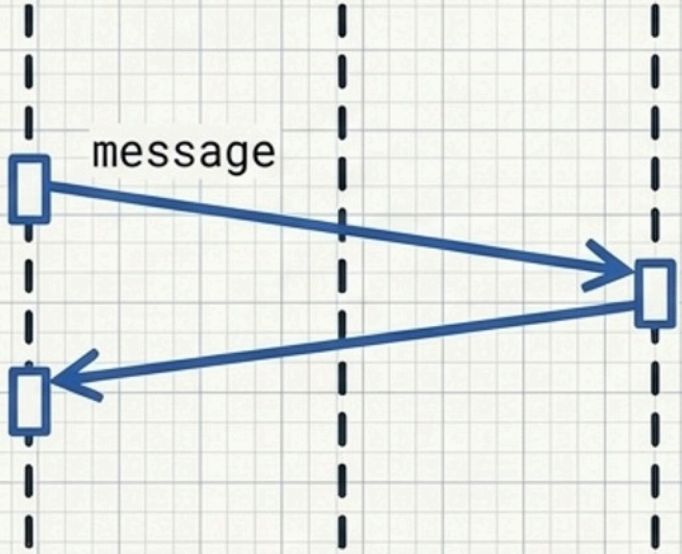
Positioning the Tool Within the UML Ecosystem

State Machine Diagram



Maps all possible events and states for a single isolated object.

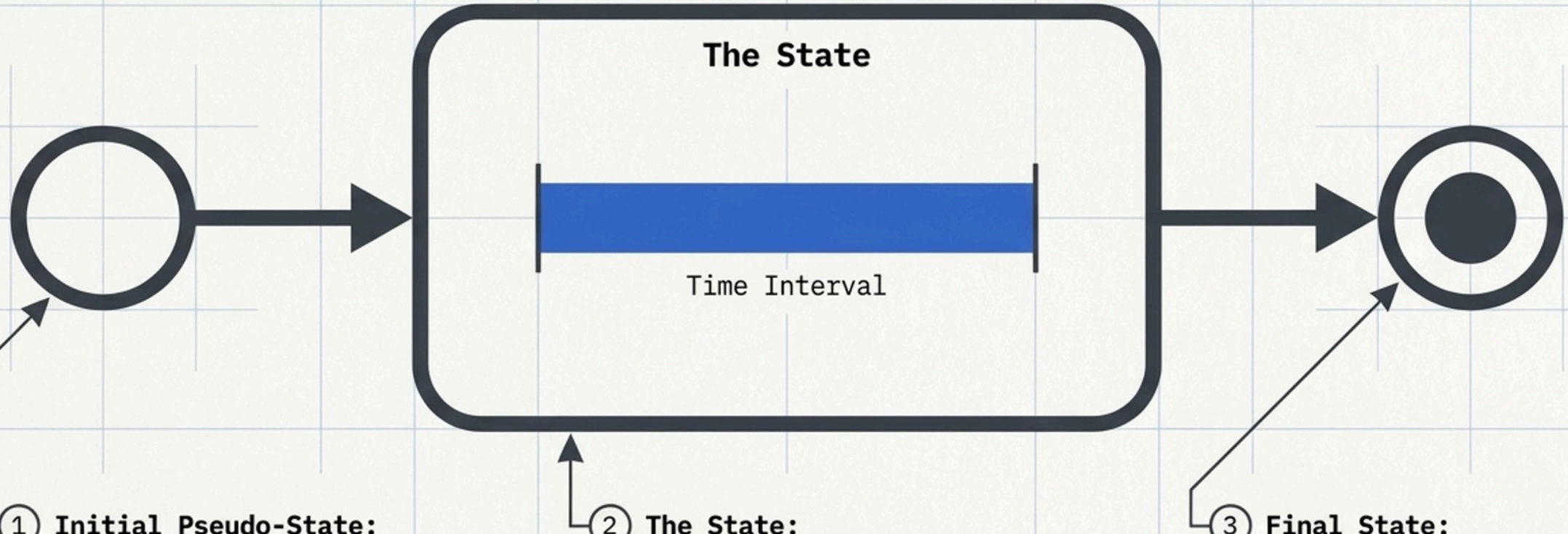
Sequence Diagram



Maps a single linear interaction across multiple connected objects.

A State is a Defined Interval in an Object's Lifecycle

A state represents an abstraction of attribute values where a specific constraint holds true, an activity is executed, or the system waits for an event.



1. **Initial Pseudo-State:** The starting line.
2. **The State:** The active timeframe where conditions are met.
3. **Final State:** The termination point (omitted in closed-loop systems).

The Four Triggers That Initiate State Transitions

Events are the incidents—internal or external—that pass information and force a system to move from one state to another.

Signal Event

Arrival of an asynchronous message or signal.

Call Event

Arrival of a procedural call to an operation.

Time Event

Occurs strictly after a specified time has elapsed.

Change Event

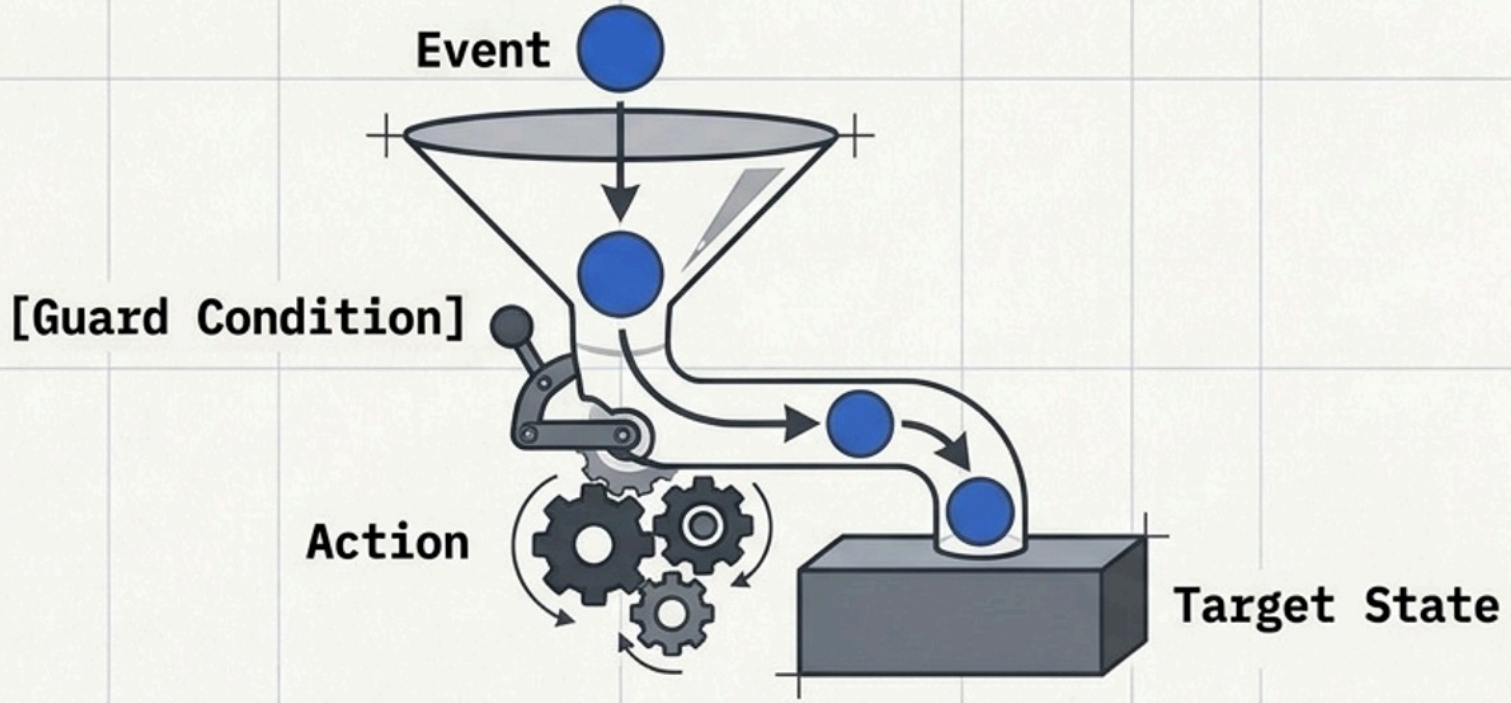
Fires whenever a specified condition is mathematically met.

The Mechanics of a Transition

A transition is the actual movement between states. It requires a source, an event trigger, and an executable action. Automatic transitions skip the event and action entirely.

Transition Equation

[Source State] + (Event) + [Action] = [Target State]



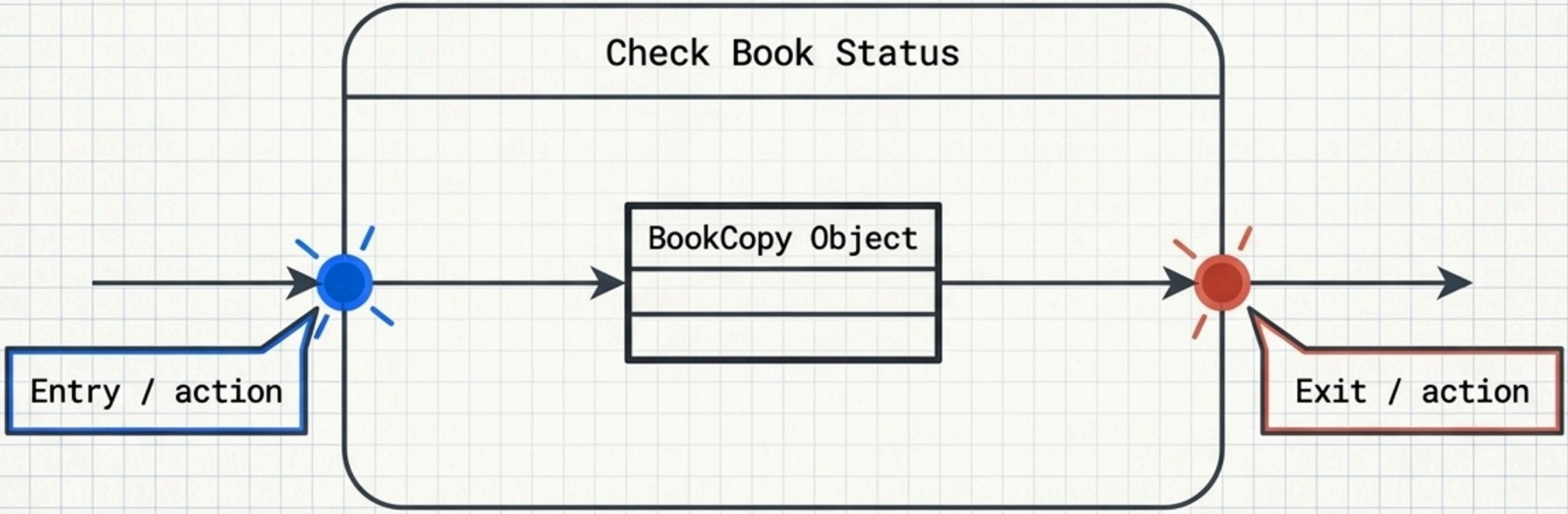
Execution Engines: Actions vs. Activities

States aren't just passive waiting rooms; they trigger computations. Differentiating between instant actions and ongoing activities is critical for system accuracy.

Feature	Action	Activity
Nature	Atomic Computation (Operation call, destroy object)	Non-atomic computation
Duration	Instantaneous	Ongoing (Runs to completion or indefinitely)
Interruptibility	NEVER interruptible	YES, can be interrupted by events
Triggers	On Entry, Do, On Event, On Exit	Sustained state presence

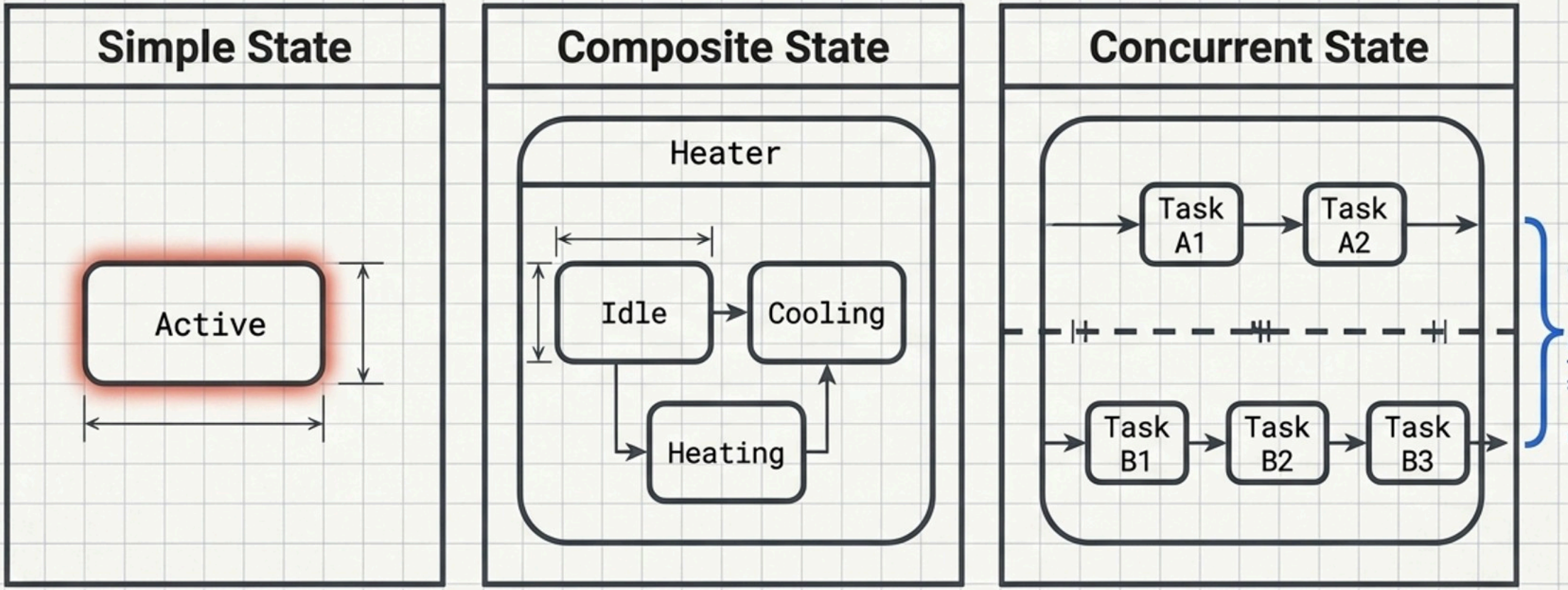
Boundary Executions: Entry and Exit Actions

Actions can be hardwired to the precise millisecond a state is entered or exited. These executions must hold true for every entry or exit occurrence; otherwise, the action belongs on the specific transition arc instead.



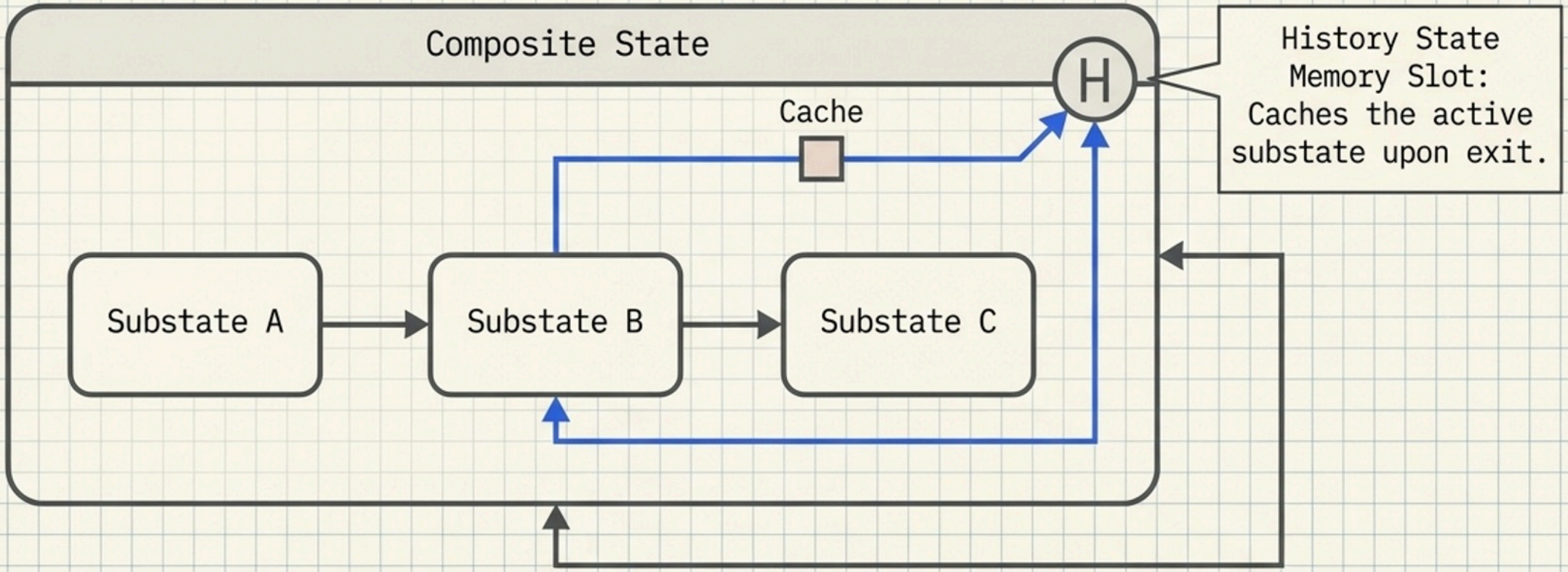
Scaling Architecture: The Complexity Matrix

As system behavior becomes multifaceted, flat state machines are insufficient. Contextual conditions dictate the nesting or parallel execution of states.



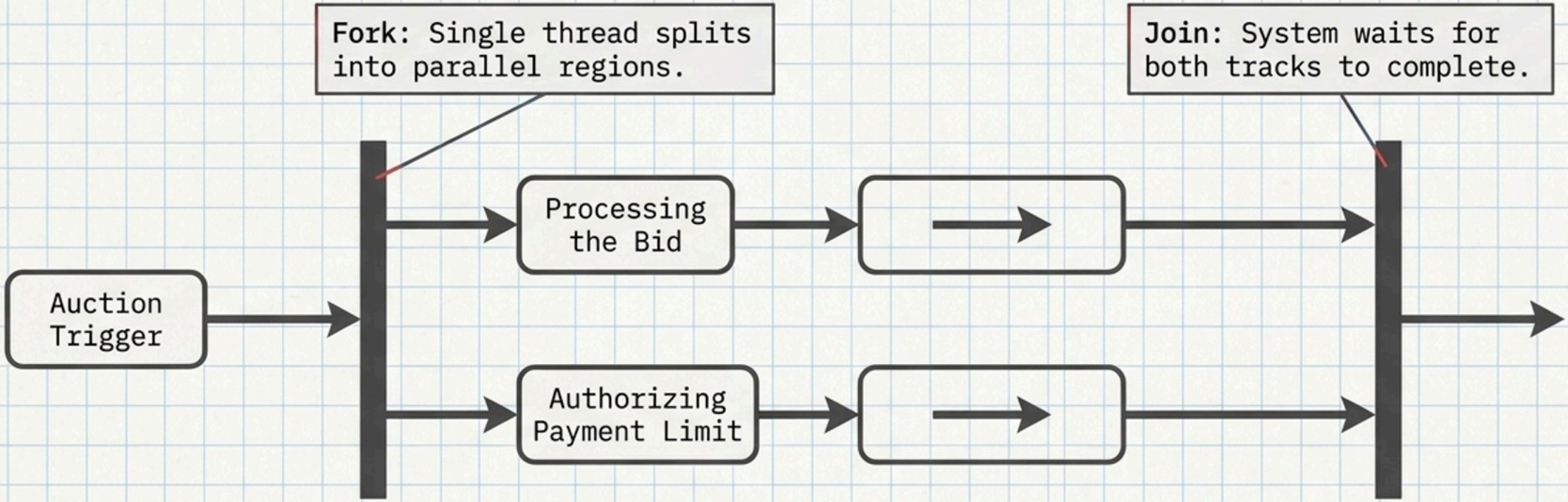
Composite States and System Memory

By default, entering a composite state restarts the nested machine at its initial state. History states act as a cache, allowing the system to remember and re-enter the exact substate that was active prior to leaving.



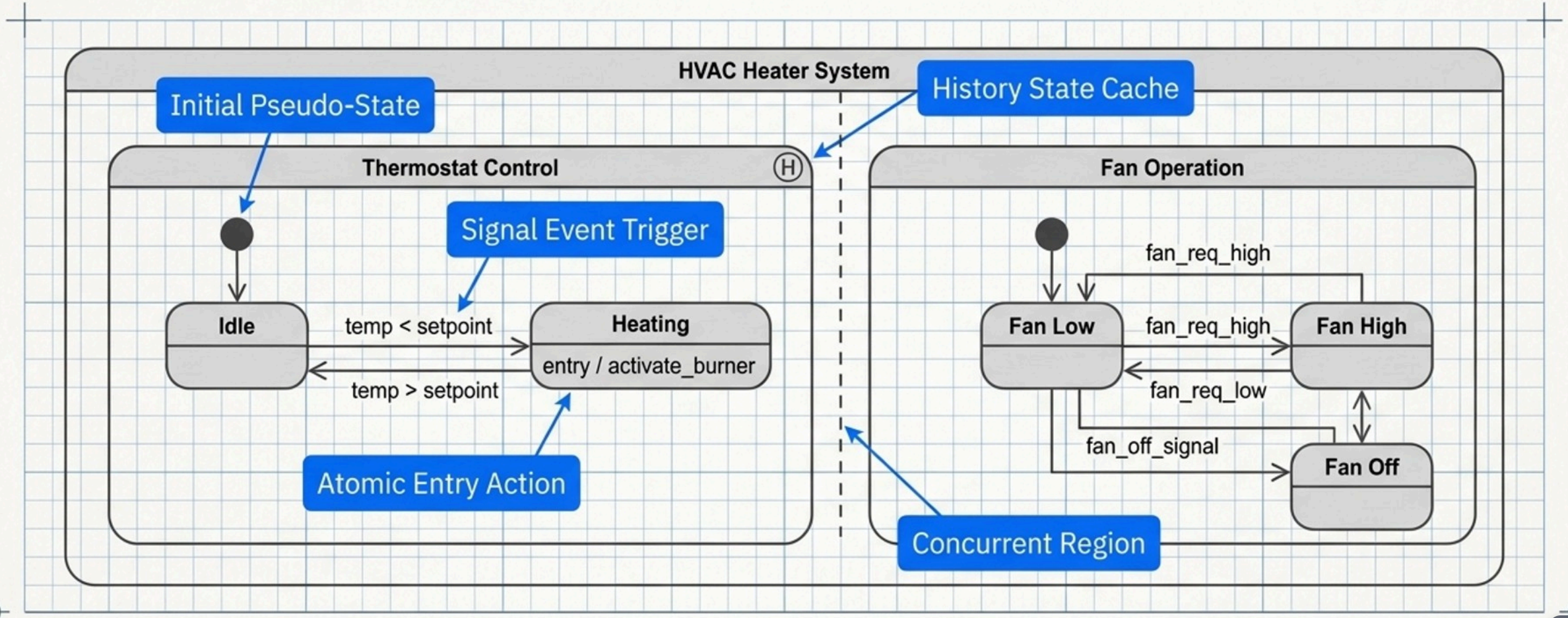
Concurrency: Forking and Joining Execution Threads

For activities requiring concurrent sub-activities, states are nested in parallel regions. The system cannot exit the composite state until all independent substates have successfully terminated.



The Unified Blueprint

By deconstructing the structural variables, complex behavioral models transform from chaotic webs into precise, readable system architectures.



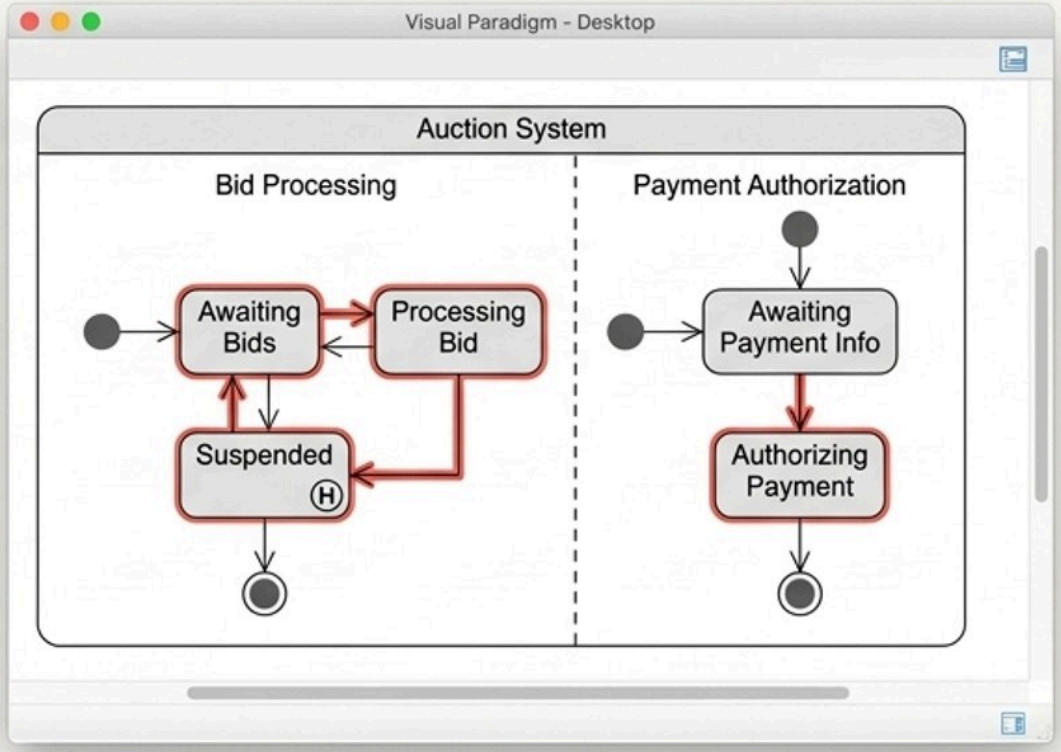
Operationalizing Behavioral Logic with AI

Modern architecture replaces manual drafting with AI-enabled logic generation. Tools like Visual Paradigm transform plain-text behavioral rules into precise, structural UML State Charts in seconds.

The Prompt

Create an auction system with parallel bid processing and payment authorization. Include a history state for suspended auctions.

The Output



[VP Desktop] Integrated AI for real-time model refinement.

[Smart Discovery] Automated identification of nested states and guard conditions.

[Deployment] From system requirement to deployable logic framework instantly.

AI Assistance: Guidelines for State Machine Modeling

- **1. Focus on Entity Lifecycle (States vs. Activities):**
  - Name states as discrete conditions or adjectives (`Placed`, `Preparing`, `Delivered`), never verbs or temporary actions (e.g., avoid "Cooking Meal").
- **2. Define Clear Triggers & Guard Rules:**
  - **Triggers:** Specify the exact business events that cause a state change (e.g., `acceptOrder`, `paymentConfirmed`).
  - **Business Guards:** Explicitly enforce rules that allow or block transitions (e.g., "cancellation allowed only before kitchen starts").
- **3. Account for Terminal & Exception States:**
  - Ensure every lifecycle has a clear starting point, successful completion, and failure/cancellation outcomes.
- **4. Human Reality Check:**
  - Did AI confuse an activity flow with a state machine?
  - Are there dead-end or unreachable states?

AI Prompt Example: Food Delivery State Machine Model

1. Role & Task Instruction:

```
Act as a Systems Verification Architect.
Generate a clean PlantUML State Machine
for a Food Delivery Order lifecycle.
```

2. Modeling Guidance:

- Model discrete lifecycle states: `Placed`, `Accepted`, `Preparing`, `ReadyForPickup`, `OutForDelivery`, `Delivered`, `Cancelled`.
- Enforce business rules: cancellation allowed only while in `Placed` or `Accepted` before cooking begins.
- Trigger automatic actions upon entering `OutForDelivery` (start GPS tracking).
- Ensure clear initial state and terminal states (Delivered, Cancelled).

3. Input Requirements Statement:

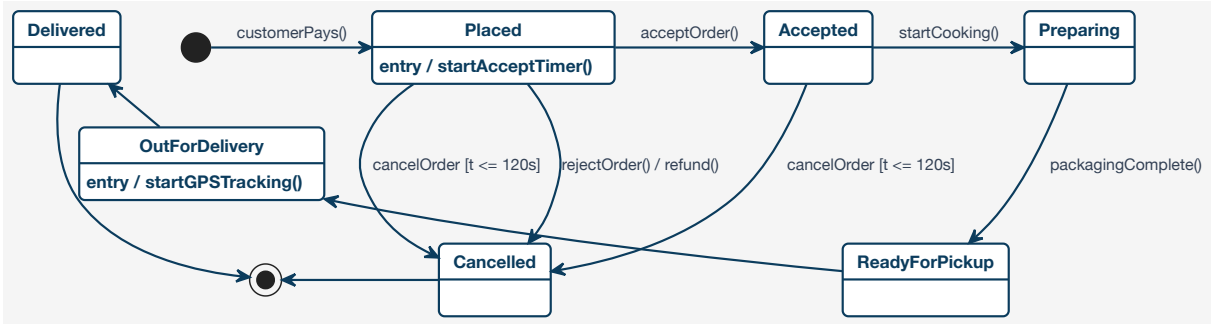
> *"An **Order** lifecycle begins at `[*]` and enters **Placed** upon customer payment.*
> *While in **Placed**, the restaurant receives an `acceptOrder` event moving it to **Accepted**, or a `rejectOrder` event moving it to **Cancelled** `/ issueFullRefund()`.*
> *The customer may trigger `cancelOrder` only while in **Placed** or **Accepted** with guard `[timeElapsed <= 120s]`, transitioning to **Cancelled**.*
> *From **Accepted**, the kitchen fires `startCooking`, transitioning to **Preparing**. Cancellation is now strictly blocked.*
> *When cooking finishes, `packagingComplete` moves order to **ReadyForPickup**.*
> *When the courier scans the pickup QR code, `courierPickedUp` moves order to **OutForDelivery**, which executes `entry / startGPSTracking()`.*
> *Upon arrival and OTP entry, `confirmDropoff` moves order to **Delivered**, which transitions to `[*]`."*
>
> ”

Generated PlantUML & Diagram: Food Delivery State Machine Model

Generated PlantUML Specification:

```
@startuml
[*] -right-> Placed : customerPays()

state Placed {
  Placed : entry / startAcceptTimer()
}
state OutForDelivery {
  OutForDelivery : entry / startGPSTracking()
}

Placed -right-> Accepted : acceptOrder()
Accepted -right-> Preparing : startCooking()
Preparing -down-> ReadyForPickup : packagingComplete()
ReadyForPickup -left-> OutForDelivery : courierPickedUp()
OutForDelivery -left-> Delivered : confirmDropoff()
Delivered -down-> [*]

Placed -down-> Cancelled : rejectOrder() / refund()
Placed -down-> Cancelled : cancelOrder [t <= 120s]
Accepted -down-> Cancelled : cancelOrder [t <= 120s]
Cancelled -left-> [*]
@enduml
```

Rendered Diagram Output:



Interactive Activity: Evolving State Lifecycles with AI (Pair Hands-On)

Hands-on Experiment: Mutate Lifecycle & Audit AI Output (4 Mins)

- **1. Add ONE Lifecycle Invariant to the State Machine Prompt:**
  - *Driver Reassignment:* If courier abandons delivery, transition back to `ReadyForPickup`.
  - *Customer Reschedule:* Allow postponing delivery while in `Preparing`.
  - *Delivery Failed:* If customer unreachable at drop-off, move to `FailedDelivery` and return meal.
- **2. Ask AI to Generate:** Prompt an LLM to update the PlantUML state machine.
- **3. Render & Critique:** Paste into PlantText and audit:
  - Did AI preserve **state naming** (adjectives/conditions, no action verbs)?
  - Are **illegal transitions** (e.g., cancelling hot food en route) strictly blocked?
  - Are **guard conditions and exit/entry actions** logically consistent?

Concept Check Question 7

In a UML State Machine, what is the critical architectural difference between an **Action** (such as `/ refundCharge()` or `entry / startTimer()`) and an **Activity** (`do / playAudio()`)?

- **A.** Actions are instantaneous, atomic, and non-interruptible; Activities are ongoing computations that take time and can be interrupted by incoming events.
- **B.** Actions are written in Python code; Activities are written in SQL queries.
- **C.** Actions only apply to class diagrams; Activities only apply to sequence diagrams.
- **D.** Actions represent manual human tasks; Activities are executed by database servers.

4.8 Text-Based Modeling with PlantUML

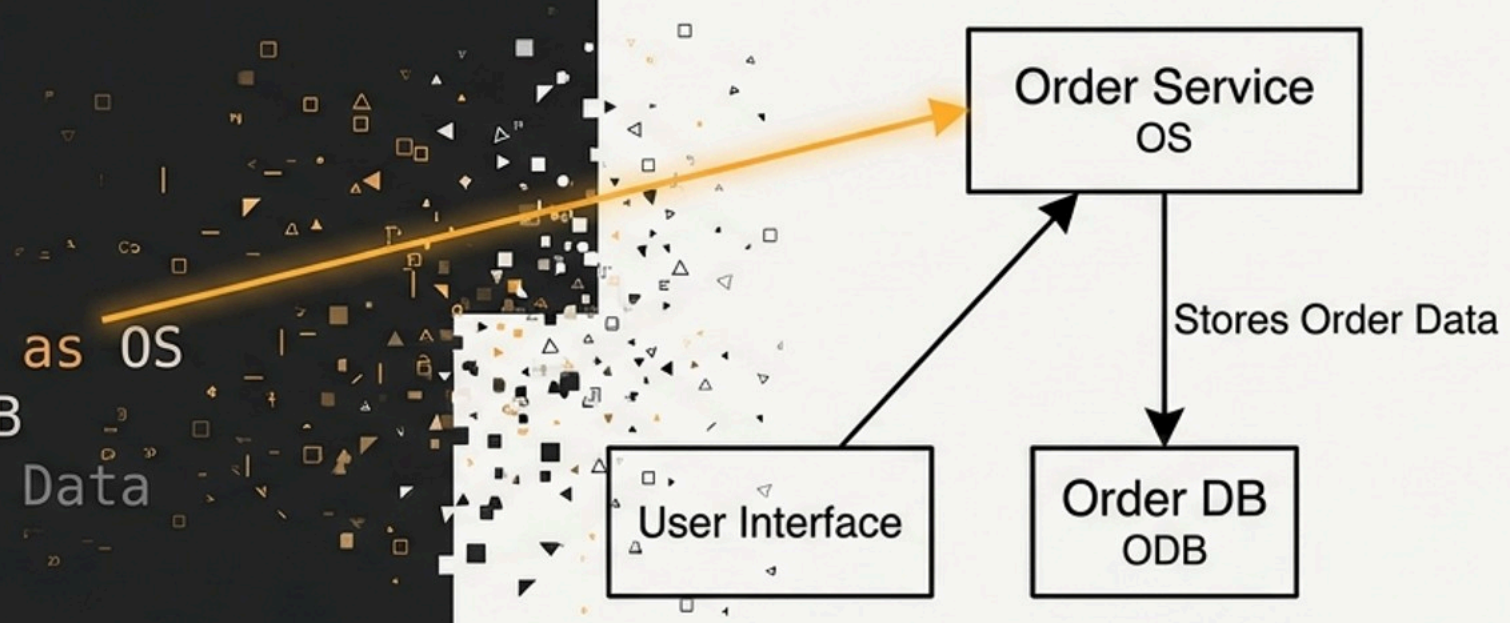
> *"Stop dragging boxes. Start writing architecture."* „

ONLINE EDITOR **PlantText Live Playground (planttext.com) →**

UML Modeling with PlantUML

A visual guide to translating software design from text to architecture.

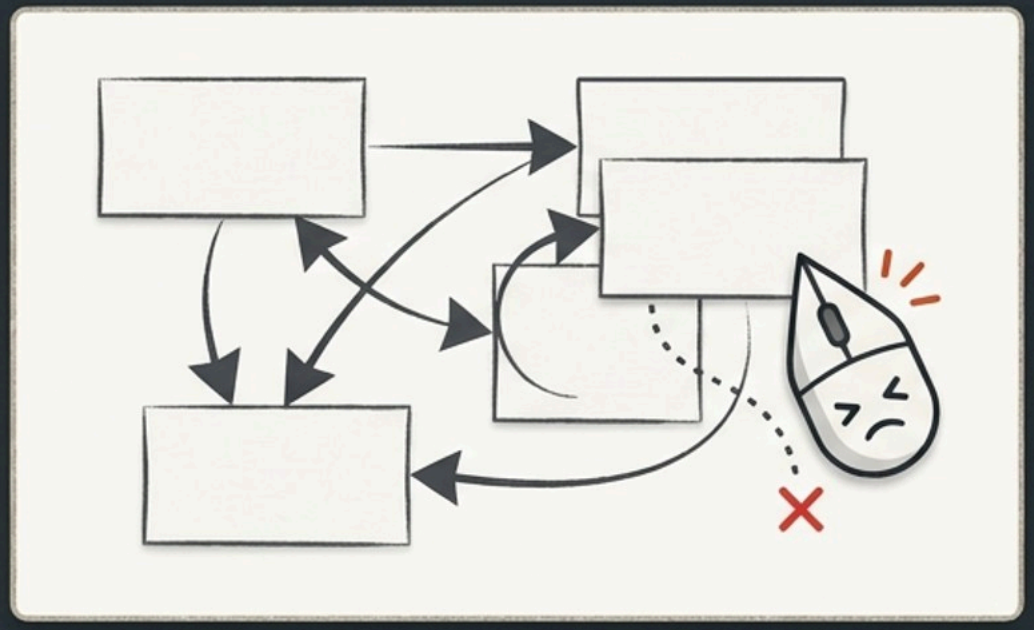
```
@startuml
!theme crt-amber
skinparam monochrome true

component "Order Service" as OS
database "Order DB" as ODB
OS --> ODB : Stores Order Data
@enduml
```



Based on the comprehensive guide by Mohammad Reza Bayat.

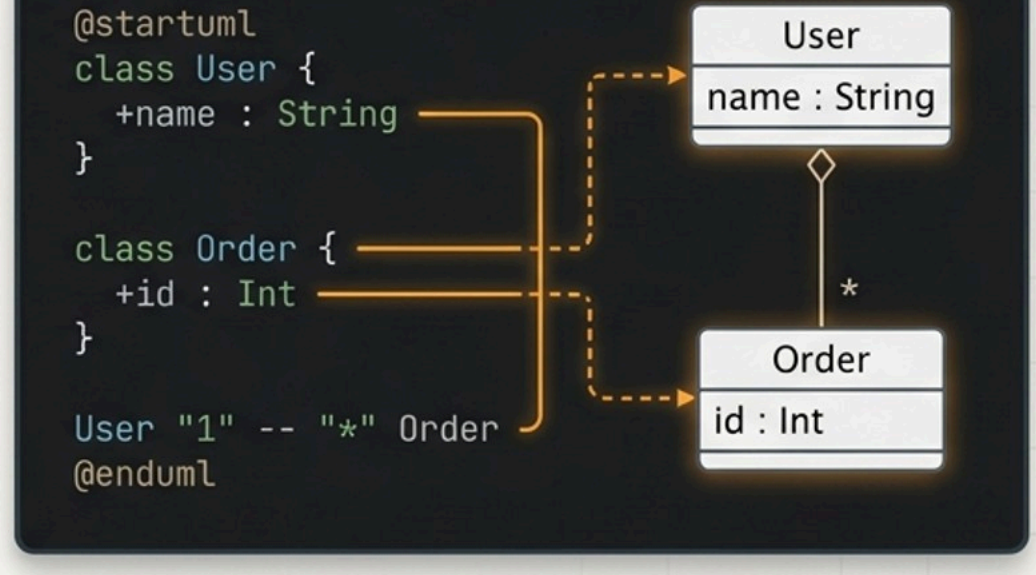
Stop Dragging Boxes. Start Writing Architecture.

The Unified Modeling Language (UML) remains the industry standard for communicating system structure and behavior. PlantUML removes the friction from creating these models. By treating diagrams as simple text syntax, engineers generate complex visual architecture without ever touching a mouse.



Slow. Manual. Disconnected.

```
@startuml
class User {
  +name : String
}

class Order {
  +id : Int
}

User "1" -- "*" Order
@enduml
```



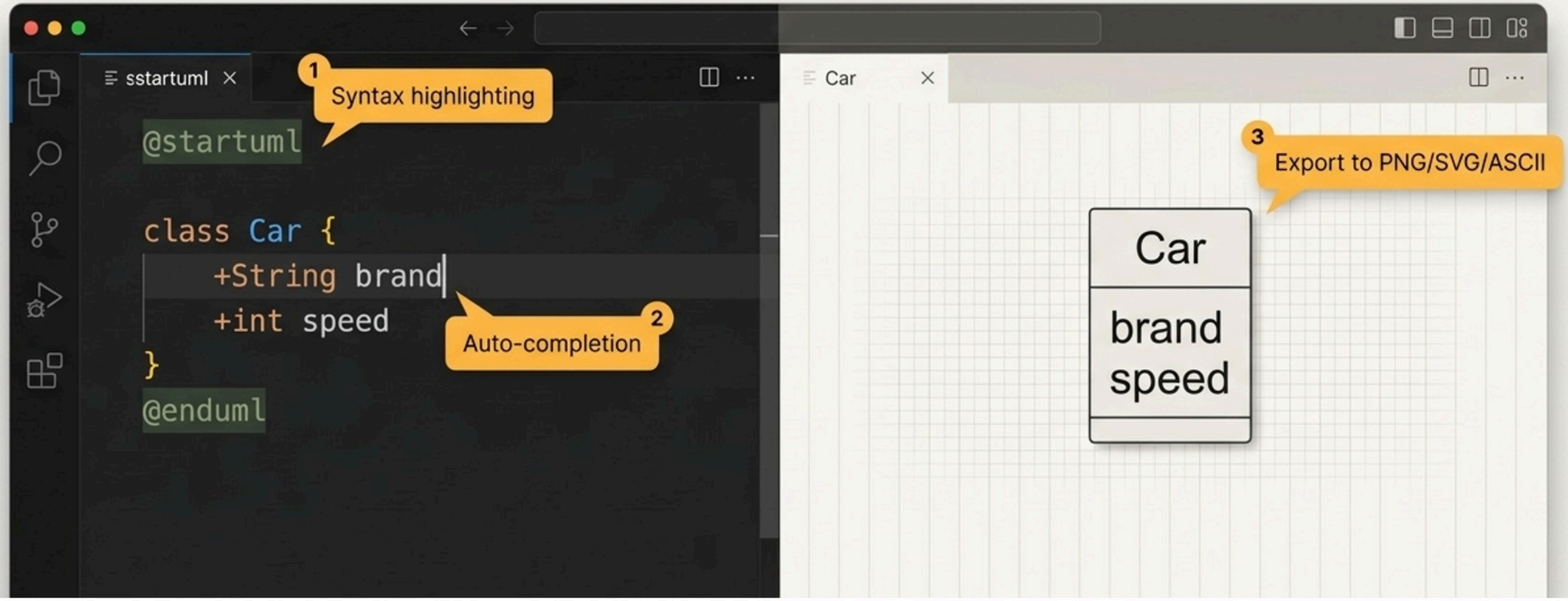
Instant. Version-Controlled. Standardized.

The Frictionless Setup

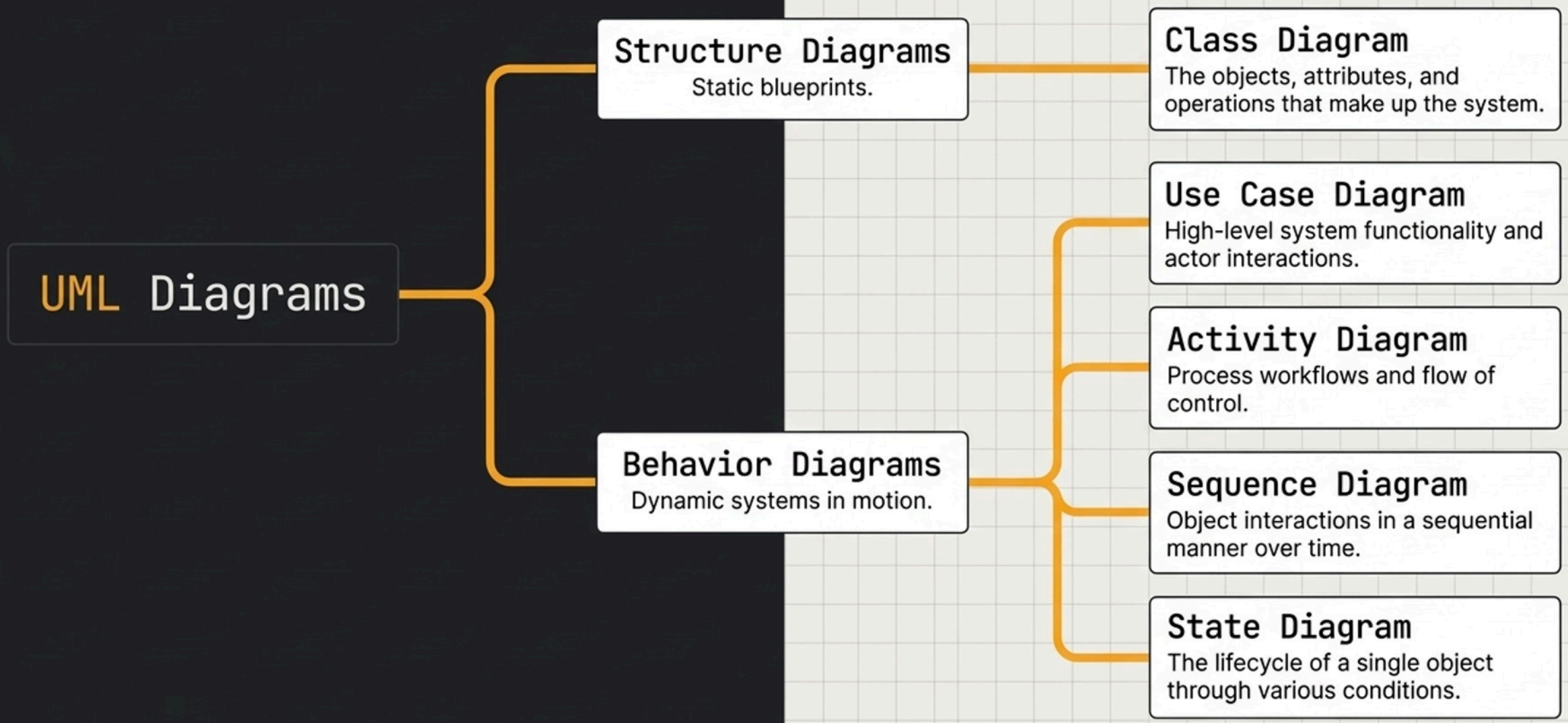
1. Install
Add the PlantUML Extension in the VS Code marketplace.

2. Write
Create a new file with the **.puml** extension.

3. Render
Press **Alt + D** to trigger the live visual preview.



The Architect's Blueprint: 5 Essential Lenses



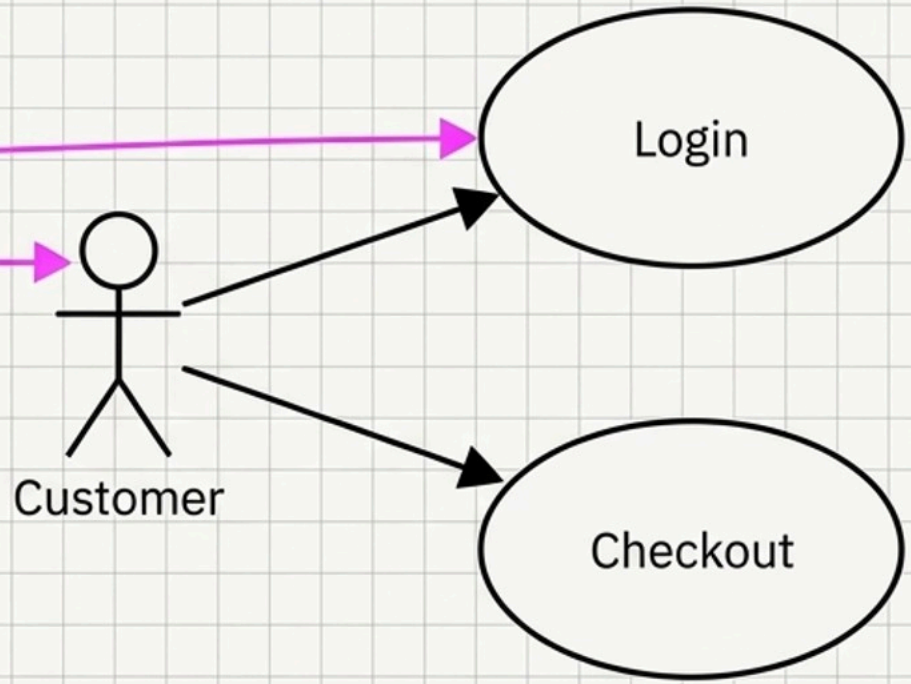
Use Case Diagrams: Mapping System Functionality

Highly effective for Agile methodologies to capture high-level requirements before writing any logic.

```
actor Customer

usecase Login
usecase Checkout

Customer --> Login
Customer --> Checkout
```

External Actors

System Functionalities



Class Diagrams: The Static System Structure

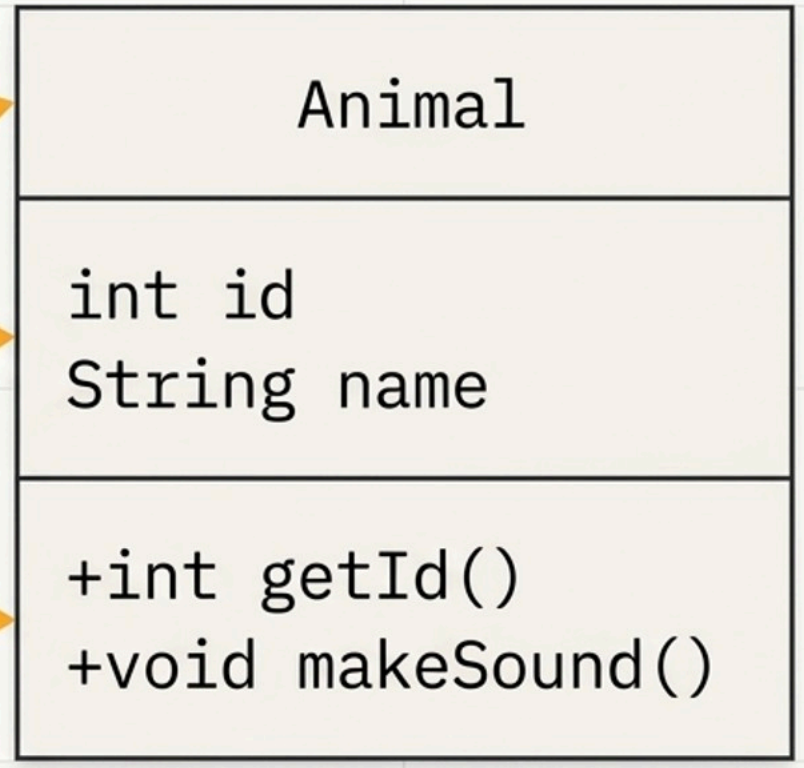
Key Insight: Visibiiity symbols define access:
+ (public), - (private), and # (protected).

```
class Animal {
  int id
  String name
  +int getId()
  +void makeSound()
}
```

Class Name (The Blueprint)

Attributes

Operations/Methods



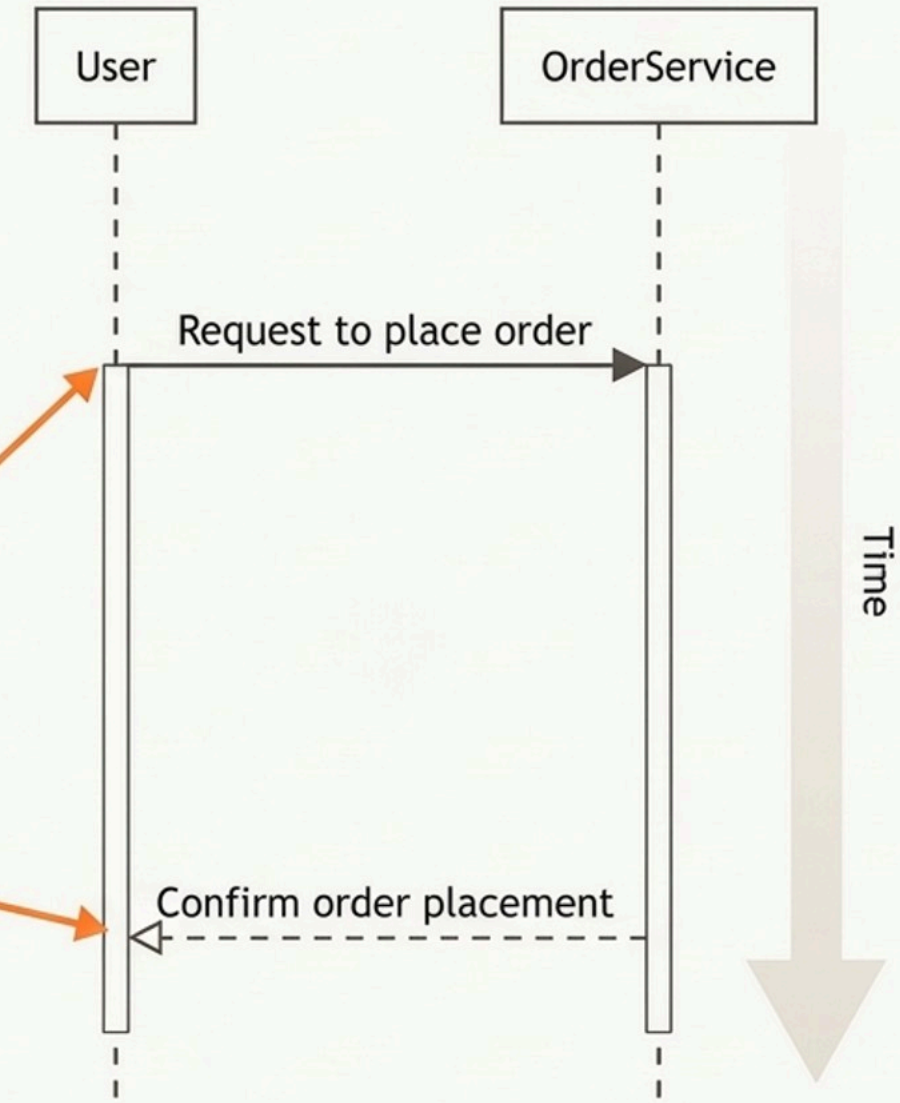
The Class Relationship Matrix

Relationship Type	Definition	PlantUML Syntax	Visual Output
Inheritance	Subclass inherits attributes/operations of a superclass.	`<|--`	
Association	One class uses the services of another.	`--`	
Aggregation	A whole/part relationship where the part can exist independently.	`o--`	
Composition	Strong ownership; the part cannot exist without the whole.	`*--`	

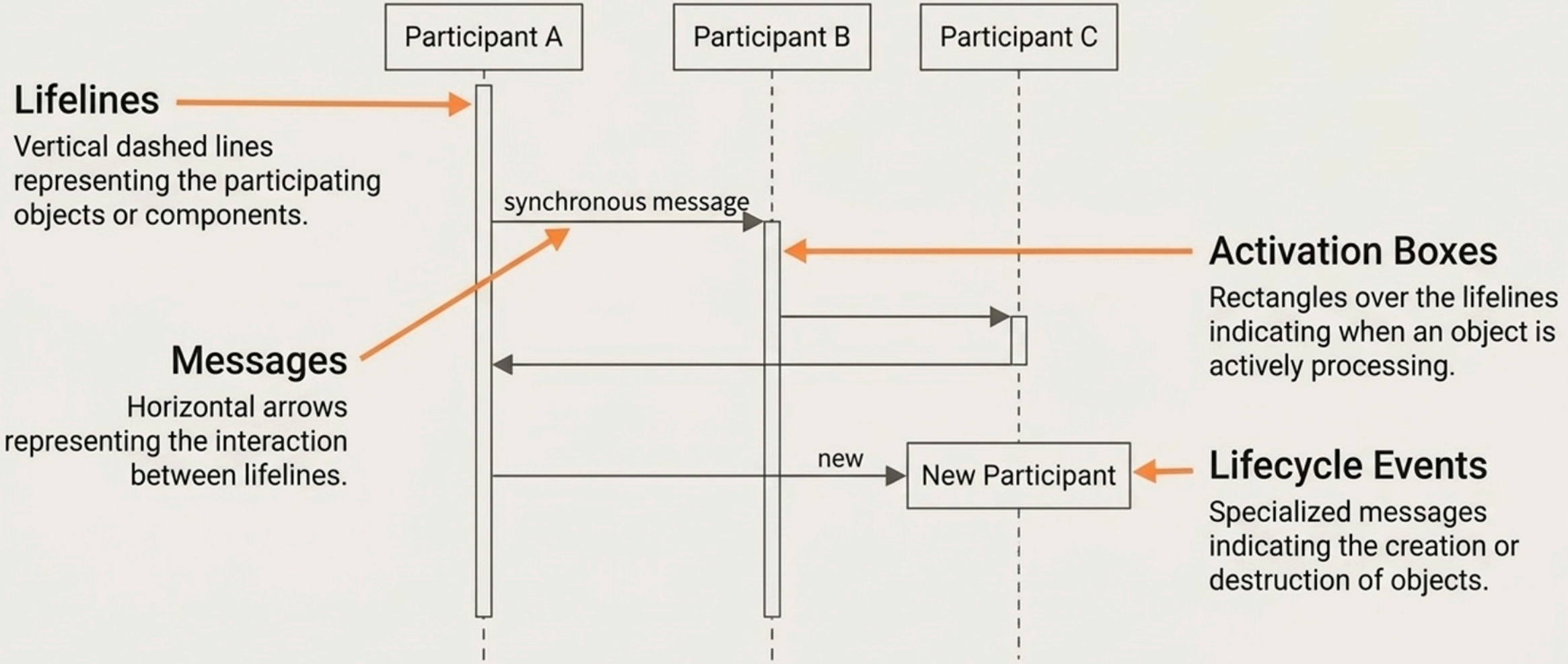
Sequence Diagrams: Interaction Over Time

Essential for modeling dynamic behavior, API calls, and synchronous/asynchronous message flows. The vertical axis represents the flow of time, moving from top to bottom.

```
participant User
participant OrderService

User -> OrderService : Request to place order
OrderService -> User : Confirm order placement
```



Anatomy of a Sequence Diagram



Lifelines

Vertical dashed lines representing the participating objects or components.

Messages

Horizontal arrows representing the interaction between lifelines.

Activation Boxes

Rectangles over the lifelines indicating when an object is actively processing.

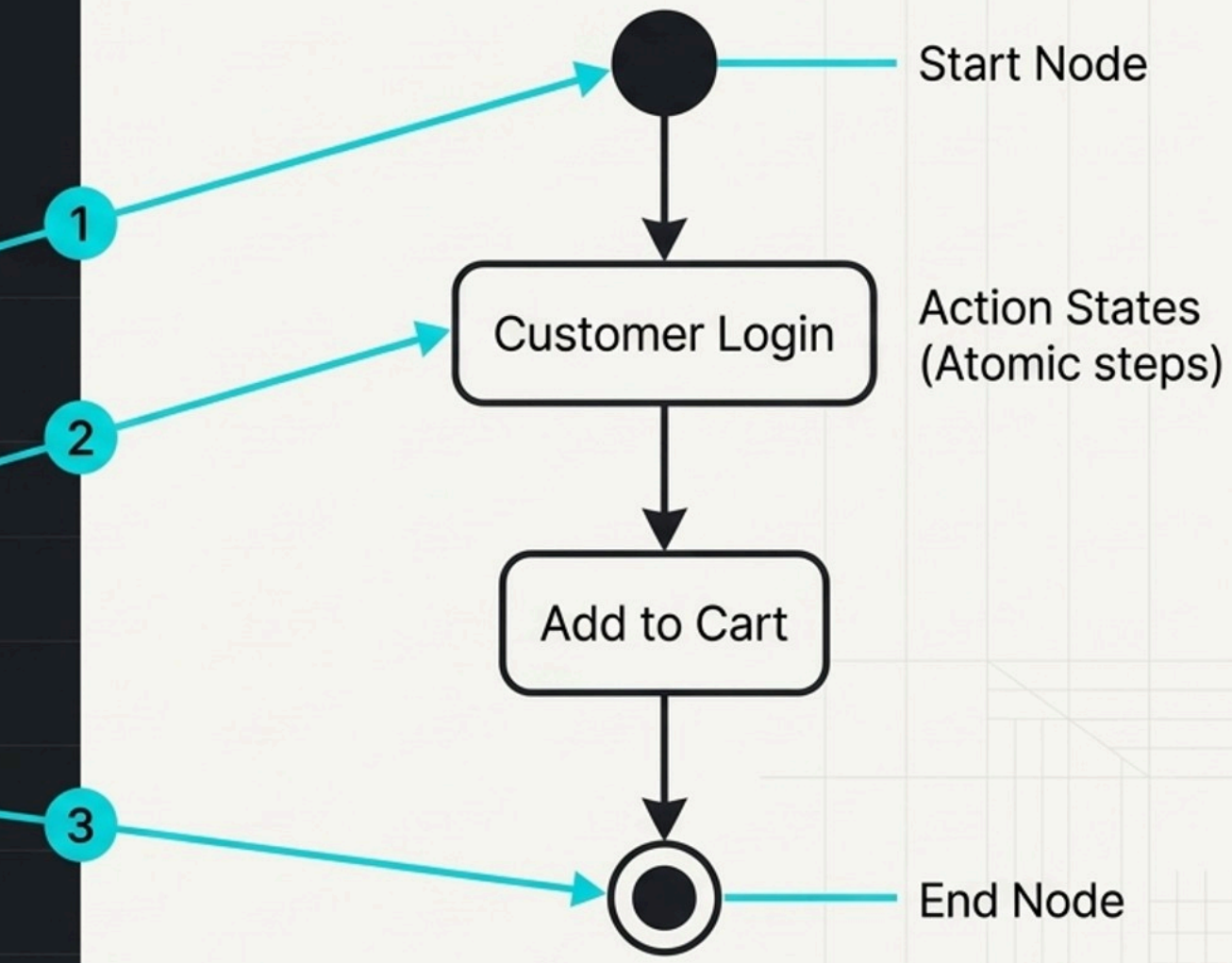
Lifecycle Events

Specialized messages indicating the creation or destruction of objects.

Activity Diagrams: Process Workflows

Powerful for modeling parallel processing via horizontal divisions called swim lanes, and conditional logic via diamond decision nodes.

```
mermaid
start
    :Customer Login;
    :Add to Cart;
stop
```

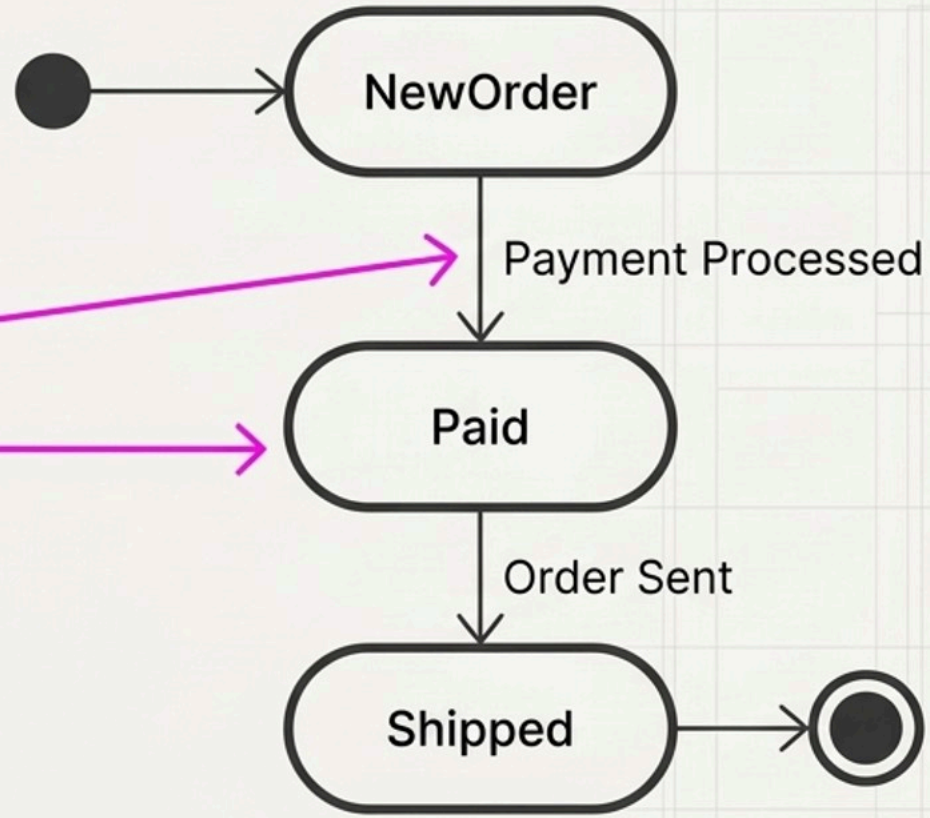


State Diagrams: Object Lifecycles

While Activity diagrams show multi-actor processes, State diagrams focus strictly on the lifecycle of a single object in response to external events.

```
1  [*] --> NewOrder
2  NewOrder --> Paid : Payment Processed
3  Paid --> Shipped : Order Sent
4  Shipped --> [*]
5
```

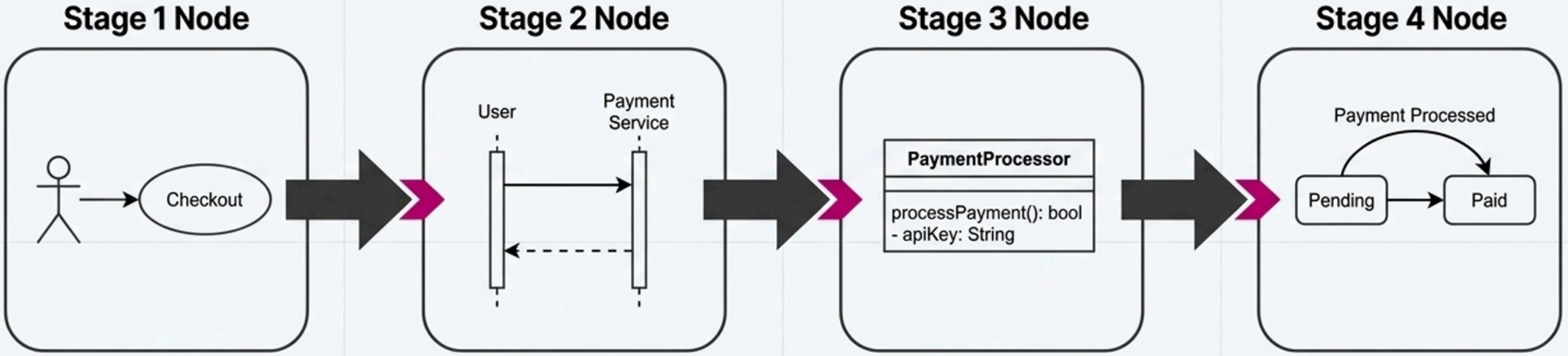
Format: `StateA --> StateB : Event String`



The Architect's Diagnostic: Choosing Your Model

Diagram	Paradigm	Core Focus	When to Use It
Use Case	Use Case	Identification centrol	Use case contains/or slopping, retreats and changes.
Class	Class	Identify fiоce class	Use class ar developers irrvvez of customers and models.
Sequence	Sequence	Example-after sequence	Use the ei tercrpated to larg-en:stiums and conticipants in diagram.
Activity	Activity	Provenional activity	Activity spectureations arupified its complon placemeoz after activity.
State	State	Model aroinotisnt	State from detorning the targeer maricipation of activity.

The Holistic System View



1. A Use Case (Customer Checkout) dictates the requirement...

2. ...which requires a Sequence (User talks to Payment Service)...

3. ...which requires a specific Class (PaymentProcessor) to be structurally built...

4. ...which ultimately changes the State of an Order (from Pending to Paid).

Insight: UML diagrams are not separate documents; they are overlapping viewpoints of the exact same system.

Concept Check Question 8

What is the primary engineering advantage of using text-based diagramming tools like **PlantUML** over traditional proprietary GUI drawing software?

- **A.** PlantUML eliminates the need to understand software architecture and UML principles.
- **B.** Text markup can be version-controlled in Git, diffed in Pull Requests, and automated in CI/CD pipelines.
- **C.** PlantUML automatically generates full production databases and microservices without writing code.
- **D.** PlantUML is strictly limited to class diagrams and cannot model dynamic workflows.

4.9 Conceptual Recap & Synthesis

> *"Models are the lingua franca of software engineering."* „

Conceptual Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. The "Three Amigos" who unified UML at Rational Software were Grady Booch, Jim Rumbaugh, and ___.
2. In system modeling, the 4 foundational perspectives are External, ___, Structural, and Behavioral.
3. In use case modeling, mandatory shared functionality is factored out using the ___ relationship.
4. The architectural pattern that separates user interfaces, business logic, and persistent domain entities is ___.
5. In a UML class diagram, a filled black diamond (◆) represents ___, where parts cannot exist without the whole.
6. In a UML class diagram, an open hollow diamond (◇) represents ___, where parts maintain independent lifecycles.
7. A UML state machine transition label follows the formal syntax: Trigger Event [___] / Action Effect.

References & Further Reading

- **Foundational Textbooks & Standards:**
  - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 5: System Modeling. Pearson.
  - Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
  - Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
  - Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
  - Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.
- **Modern Declarative Diagramming & AI Tooling:**
  - PlantUML Open-Source Standard: plantuml.com
  - Mermaid.js JavaScript Diagramming Documentation: mermaid.js.org